



INSTITUTO SUPERIOR DE ESTUDIOS PSICOLÓGICOS

Mestrado em Business Intelligence and Big Data

Geração
2022-2025

**O ESTUDO DE ALGORITIMOS DE PREDICAO DE PRECOS DO BITCOIN COM
BASE EM BUSSINESSS INTELIGENCE E BIG DATA**

Apresentado por: LUIZ FERNANDO BROGLIATTO FERREIRA

Tutor acessor: Prof., Dr. Aldenir Antunes da Silva.

Toledo-PR, Brasil

2025

LUIZ FERNANDO BROGLIATTO FERREIRA

O ESTUDO DE ALGORITIMOS DE PREDICAO DE PRECOS DO BITCOIN COM BASE EM BUSSINESSS INTELIGENCE E BIG DATA

Monografia apresentada à Banca Examinadora da Univercidade ISEP (Instituto Superior de Estudos Psicológicos) de Culiacán Rosales, Sin., México, como exigência parcial para obtenção do título de Graduação em Economia, sob a orientação do Prof., Dr. Aldenir Antunes da Silva.

Toledo-PR, Brasil

2025

AGRADECIMENTOS

Primeiramente, agradeço a Deus por ter me concedido saúde, inteligência e perseverança para realizar esta obra, permitindo que eu concluísse este projeto com dedicação e clareza de pensamento.

Agradeço profundamente aos meus pais, Luiz Carlos Ferreira e Maria Paulina Brogliatto Ferreira, pelo apoio incondicional ao longo da vida e pela contribuição financeira que tornaram possível a concretização deste trabalho.

Expresso minha gratidão a toda a equipe da Universidade ISEP – Instituto Superior de Estudios Psicológicos, pelos ensinamentos, orientação e ambiente acadêmico que enriqueceram minha formação e subsidiaram esta pesquisa.

Agradeço também aos inúmeros autores cujas pesquisas consultei, que serviram de base sólida e permitiram complementar e aprofundar os temas abordados nesta obra.

Por fim, um agradecimento especial àquele que se identifica como Satoshi Nakamoto, pela criação do conceito de Bitcoin, que revolucionou o entendimento sobre sistemas descentralizados e inspirou reflexões fundamentais neste estudo.

A todos, o meu mais sincero obrigado.

DECICATÓRIA.

Dedico esta obra às futuras pesquisas dedicadas à predição de preços de Bitcoin, criptomoedas e outros ativos financeiros, na esperança de que contribua para o avanço do conhecimento e para o desenvolvimento de modelos mais precisos e inovadores nesta área em constante evolução.

Dedico-a, igualmente, a todo o código escrito em Python aqui apresentado, desenvolvido com o apoio da biblioteca Antigravity, que serviu como base para a criação e o aprimoramento de novas ferramentas com o mesmo objetivo: auxiliar pesquisadores, desenvolvedores e entusiastas na construção de soluções mais robustas e eficientes para o análise e previsão de mercados financeiros digitais.

Que esta obra inspire e facilite o caminho de quem vier depois.

LISTA DE FIGURAS

Figura 1: Representação da Blockchain.....	19
Figura 2: Prova de trabalho	19
Figura 3: Modelo de transações da blockchain)	22
Figura 4: Principais criptomoedas com capitalização de mercado(2024).....	26
Figura 5: Ranking das criptomoedas (out. 2025).....	27
Figura 6: Regressão linear simples	35
Figura 7: Representação de árvore de decisão binária.....	37
Figura 8: Representação simplificada do algoritmo de Random Forest	38
Figura 9: Camadas de uma MLP	41
Figura 10: Camadas cíclicas de uma RNN	42
Figura 11: Diagrama de blocos de uma LSTM	44
Figura 12: Estrutura de uma GRU.....	45

LISTA DE TABELAS

Tabela 1: Exchanges com maior volume e confiabilidade.....	20
Tabela 2: Quatro maiores criptomoedas em valor de mercado e sua variação	24
Tabela 3: Comparação entre estudos na área	46

RESUMO

O objetivo deste trabalho de pesquisa é analisar a aplicabilidade das tecnologias disruptivas, com foco em ferramentas como IA, Blockchain, IoT e Big Data, na predição de preços do bitcoin. Tem-se como objetivos específicos: descrever sobre blockchain, inteligência artificial e criptomoedas; apresentar algumas considerações sobre algoritmos de aprendizado e redes neurais; descrever sobre o estado da arte e predição de preços; e, apresentar uma predição de preços de criptomoedas em um determinado espaço de tempo. Metodologicamente utilizou-se das pesquisas descritiva, qualitativa, bibliográfica e um estudo temporal. Concluiu-se que, em mercados altamente ruidosos e imprevisíveis como o do Bitcoin, a complexidade excessiva não garante superioridade: modelos mais parsimoniosos, como o MLP, generalizam melhor a longo prazo, enquanto o Algoritmo Genético se sobressai na navegação de riscos. Estratégias híbridas, combinando a robustez do MLP e do Genetic Algorithm com dados on-chain enriquecidos, representam o caminho mais promissor para aplicações reais em finanças disruptivas, equilibrando precisão preditiva com preservação de capital em regimes de alta volatilidade.

Palavras chave: Tecnologias disruptivas. Algoritmos de predição. Blockchain. Bitcoin. Inteligência artificial.

ABSTRACT

The objective of this research is to analyze the applicability of disruptive technologies, focusing on tools such as AI, Blockchain, IoT, and Big Data, in predicting Bitcoin prices. Specific objectives include: describing blockchain, artificial intelligence, and cryptocurrencies; presenting some considerations on machine learning algorithms and neural networks; describing the state of the art and price prediction; and presenting a cryptocurrency price prediction within a specific timeframe. Methodologically, descriptive, qualitative, bibliographic research, and a temporal study were used. It was concluded that, in highly noisy and unpredictable markets like Bitcoin, excessive complexity does not guarantee superiority: more parsimonious models, such as MLP, generalize better in the long term, while Genetic Algorithms excel in risk management. Hybrid strategies, combining the robustness of MLP and Genetic Algorithms with enriched on-chain data, represent the most promising path for real-world applications in disruptive finance, balancing predictive accuracy with capital preservation in highly volatile regimes.

Keywords: Disruptive technologies. Prediction algorithms. Blockchain. Bitcoin. Artificial intelligence.

SUMARIO

1	INTRODUÇÃO	11
2	REFERENCIAL	14
2.1	BLOCKCHAIN E MOEDA DIGITAL NO MERCADO FINANCEIRO	14
2.1.1	Riscos e Benefícios do Blockchain	16
2.2	A INTELIGÊNCIA ARTIFICIAL (IA)	17
2.3	AS CRIPTOMOEDAS	18
2.3.1	Tipos de criptomoedas	25
2.3.1.1	Bitcoin	27
2.3.1.2	Ethereum	29
2.3.1.3	Tether	29
2.4	ANÁLISE TÉCNICA	30
2.5	INDICADORES TÉCNICOS	31
2.5.1	Média móvel simples	31
2.5.2	MACD (Moving average convergence divergence)	32
2.5.3	Relative Strength Index	33
2.5.4	Oscilador estocástico	33
2.6	ALGORITMOS DE APRENDIZADO	34
2.6.1	Regressão Linear Múltipla	34
2.6.2	K-Nearest Neighbors	36
2.6.3	Decision Tree	36
2.6.4	Random Forest	37
2.6.5	Histogram-Based Gradient Boosting	37
2.6.6	Autoregressive Integrated Moving Average	39
2.7	REDES NEURAIS	39
2.7.1	Recurrent Neural Networks	41
2.7.2	Long short term memory (LSTM)	42
2.7.3	Bidirectional Long Short-Term Memory	43
2.7.4	Gated Recurrent Unit	44
2.7.5	Séries temporais	44
2.8	ESTADO DA ARTE	45
2.9	PREDIÇÃO DE PREÇOS	46
3	METODOLOGIA	48

4	DISCUSSÃO E ANÁLISE DE DADOS.....	50
5	CONCLUSÃO.....	52
	REFERÊNCIAS.....	54
	ANEXOS	58
	ANEXO A: Resumo executivo: benchmark de IA para criptoativos	58
	ANEXO B: Relatório Técnico Experimental: Previsão de Bitcoin com IA	61
	ANEXO C: Plano de ação: Laboratório de inteligência computacional aplicada	65
	ANEXO D: Relatório científico final	68
	ANEXO E: Documentação do Código Fonte	82

1 INTRODUÇÃO

A Tecnologia Disruptiva é àquela quando inserida no comércio podem mudar ou romper paradigmas constituídos no seio de organizações ou indústrias, sendo nos processos internos, ou na sua forma de produção ou entrega de seus produtos acabados. O célere avanço de tais tecnologias vem transformando expressivamente vários domínios da economia, inclusive nas esferas financeira e contábil. Ferramentas inovadoras vêm tendo rápida efetivação, e temas como Blockchain, Inteligência Artificial (IA), Machine Learning, Automação de Processos Robóticos (RPA) e Big Data circundam debates empresariais, notadamente no tocante à aplicação, segurança das informações e o custo-benefício de tais tecnologias (Stroparo et al., 2024, Stroparo; Bochniak, 2024)

Conforme literatura atual existente percebe-se que tais tecnologias disruptivas vêm tendo aplicabilidade em várias áreas, especialmente nas finanças empresariais. Ferramentas com base em IA vêm sendo largamente utilizadas para automatização de processos, melhoria na precisão de análises e otimização de tomadas de decisões nas organizações (Bahoo et al., 2024; Sharbek, 2024). Designadamente no campo contábil e financeiro, não se restringe a automatização de tarefas rotineiras; mas também na promoção de novos níveis de análise preditiva, gestão de riscos e transparência nas operações financeiras (Ramos; Stroparo; Cordeiro, 2021; Stroparo; Bochniak, 2024)

A tecnologia Blockchain oferece inúmeros reforços nos setores bancário e financeiro, realçando seu potencial para demudá-los, aprimorar a segurança, velocidade e custos nas transações. Uma fundamental contribuição que a Blockchain causou foi realizar transações financeiras sem intermediário, reduzindo a complexidade em contratos, com foco na transparência da informação e segurança de dados (Sousa, 2024).

O uso do Blockchain Accounting (BCA) vem sendo transformada. A tecnologia Blockchain proporciona uma camada de segurança e imutabilidade nos registros contábeis, admitindo auditorias automáticas e continuadas, suprimindo a obrigação de intermediários e afixando forte transparência nos registros financeiros. Igualmente, contratos inteligentes (smart contracts) realizados no blockchain automatizam ações como pagar fornecedores e liquidar contratos, minimizando atrasos e custos operacionais (Stroparo; Bochniak, 2024).

A Inteligência Artificial (IA), também considerada tecnologia disruptiva essencial, se mostra uma força motriz no caminho da automação de processos, seja para tarefas simples ou tomadas de decisões estratégicas. A mesma possui a capacidade de analisar enormes volumes de dados, identificar padrões financeiros e antecipar tendências de mercado, sendo uma ferramenta fundamental para otimizar processos contábeis e financeiros (Carolan, 2024; Stroparo; Bochniak, 2024)

Pagamentos através da rede Blockchain pode institucionalizar criptomoedas e reduzir problemas de lavagem de dinheiro. Essa tecnologia possibilita recuperar valores, reduzir riscos de perdas financeiras. A concepção de moeda digital e sua dependência da tecnologia Blockchain foram introduzidos inicialmente com a criação do Bitcoin, que assinalou o início de uma nova era em transações digitais e sistemas financeiros (Schär, 2020, Sousa, 2024).

A análise e predição de tendências e de risco financeiro é um tema importante na área da contabilidade e finanças, com crescimento nas análises e pesquisas efetivadas nos últimos anos. Esse fato é devido ao aumento dos mercados financeiros junto ao crescimento do número de fraudes e crises financeiras que acontecem conjuntamente. Para tanto, vários algoritmos, métodos, simulações e técnicas computacionais têm sido desenvolvidos e utilizados na área financeira nos últimos anos. Igualmente, crescem os mercados financeiros de criptomoedas, tanto como especulação como para reserva de valor (Santos; Igarashi; Igarashi, 2023).

O Bitcoin surgiu em 2008 como resposta à crise financeira ocorrida, com a concepção de substituir a moeda física e retirar a dependência de intermediação de bancos em operações financeiras. O primeiro artigo sobre Bitcoin foi publicado por Satoshi Nakamoto, descrevendo o funcionamento da moeda digital e o blockchain. O mercado de criptomoedas é marcado pela volatilidade de preços em curtos períodos de tempo. Em 2017, por exemplo, o Bitcoin valorizou 1.300% e teve queda abrupta de preço em maio de 2021, caindo quase 50% frente a maio do ano anterior. Logo, utilizar técnicas estatísticas e modelos computacionais como redes neurais ajudam a identificar momentos mais propícios para compra, venda ou manter Bitcoins (Santos; Igarashi; Igarashi, 2023).

A análise técnica tem base no princípio que preços de ativos em certo momento futuro, tem influência por seus preços passados, assim, é possível prever a tendência de movimentação desses preços em certo período de tempo; utilizando indicadores técnicos para indicar momentos oportunos para realizar transações.

Neste contexto, tem-se a seguinte questão de pesquisa: como se efetua a predição de preços do bitcoin com algoritmos baseados em bussinesss intelligence e big data?

O objetivo da pesquisa é analisar a aplicabilidade das tecnologias disruptivas, com foco em ferramentas como IA, Blockchain, IoT e Big Data, na predição de preços do bitcoin.

O estudo visa contribuir no debate sobre o tema, reunindo dados e material sobre o assunto, movimentações do mercado frente às inovações, bem como avaliar exterioridades que possam colaborar para o entendimento de gestores gerenciais, pois as organizações enfrentam inúmeros desafios, na implementação de soluções robustas e práticas eficazes para integrar com sucesso o Blockchain e as moedas digitais em suas operações (Schär, 2020).

Para tanto, utiliza-se uma pesquisa caracterizada como qualitativa exploratória, a qual, nas palavras de Gil (2008), tem como objetivo esclarecer lacunas de conhecimento, trazendo mais luz ao tema.

Este trabalho se justifica, pois o gerenciamento de riscos no setor corporativo antes uma função defensiva agora versa como elemento central na estratégia de negócios, com foco na qualidade dos ativos e lucros, gerando funções como agentes de risco, que integram a gestão de riscos à cultura organizacional, promovendo valor e progresso, gerenciando riscos de modo eficiente. Logo, o tema se propõe a facilitação de transações diretas sem intermediários, como o Blockchain que atua minimizando expressivamente custos de transação para organizações financeiras, maximizando a eficiência (Barroso, 2023). Sua utilização reduz custos e aumenta a eficiência dos processos, diminuindo os custos de fraude e violação de segurança, propiciando maior eficiência financeira (Schär, 2020; Sousa, 2024).

As redes neurais artificiais (RNAs) tornaram-se peça chave na sociedade contemporânea. Assim, novas técnicas surgem a cada dia para solucionar múltiplos problemas, muitas vezes, insolúveis por humanos. De igual modo, o mercado de ativos com base em criptografia cresce exponencialmente, sendo muito utilizado como reserva de valor, meio de pagamento e modo diversificado de investimentos (SOUSA et al., 2022). As tecnologias disruptivas convergem para um mundo cada vez mais digital. A previsão de preços em mercados financeiros não é recente; no entanto, ativos com alta volatilidade, como o Bitcoin, ainda apresentam potencial para operações lucrativas (Tripathi; Sharma, 2023; Oliveira; 2025).

2 REFERENCIAL

2.1 BLOCKCHAIN E MOEDA DIGITAL NO MERCADO FINANCEIRO

A tecnologia Blockchain ajuda a implementar moedas digitais, deixando transações seguras e descentralizadas, sem interferência de terceiros, trazendo o Bitcoin como precursor, melhor dizendo, moeda digital pioneira adentrada em 2009. Moedas digitais utilizam o sistema Blockchain para afiançar que se façam transações seguras, imutáveis e transparentes. Sua descentralização minimiza o risco de fraude e hacking (preocupação de custos nos sistemas financeiros) (Zhang; Huang, 2022).

A tecnologia Blockchain permite que transações transponham fronteiras, “permitindo que moedas digitais sejam transferidas entre países, sem a necessidade de intermediários, o que é particularmente benéfico para o comércio internacional e para as remessas, reduzindo custos e tempos de transação” (Sousa, 2024, p. 4).

A eficiência de custo ao eliminar intermediários e agilizar processos traz enorme vantagem sobre sistemas financeiros tradicionais, que usualmente abarcam diversas camadas de intermediários e taxas embutidas; estimulando a inovação em serviços financeiros, induzindo a desenvolverem-se novos produtos e serviços mais eficientes e acessíveis (Goghie, 2024).

O setor financeiro vem se beneficiando significativamente do Blockchain, pois o mesmo pode:

[...] revolucionar as operações bancárias ao fornecer pagamentos mais rápidos, reduzir custos e aumentar a segurança e o desempenho das transações. [...] O Bacen vem explorando o uso do Blockchain, colocando em funcionamento a Pier, plataforma que integrados de órgãos reguladores agilizam processos e autorizações de registros, demandados por instituições financeiras brasileiras [...] projeto-piloto da Plataforma do Real Digital (DREX), que tem por objetivo testar a viabilidade e a segurança da moeda digital brasileira, usando a tecnologia de registro distribuído (DLT) [...] proporciona maior auditabilidade, rastreabilidade e transparência, que são importantes para a conformidade regulatória, especialmente no combate à lavagem de dinheiro e ao financiamento do terrorismo, entregando mais segurança às operações (Sousa, 2024, p. 4).

No tocante às expectativas quanto à implementação das tecnologias de plataformas blockchain toma-se como base o survey - Market Survey Report for (Non-financial) Application of Blockchain in China -, consultoria chinesa em 2019, que fez um levantamento com vários ramos empresariais como TI e indústria de alta

tecnologia (28% dos participantes), serviços (15%), indústria manufatureira (11%), indústria de blockchain (11%), mídia (9%), instituições científicas e educacionais (6%), varejo (4%) e outras indústrias (16%). Indicaram expectativa positiva, apontando otimismo com impacto expressivo (52,8% dos participantes) e algum impacto (44%). 87,5% apontaram como principal feito da tecnologia blockchain sua resistência a fraudes e na distribuição (descentralizada) do sistema. Também se consolida em setores que já conhecem ou já implementam a tecnologia. Setores de gestão, como cadeia de abastecimento, de armazenamento e distribuição de dados e de autenticação por identidade, são quem mais vislumbram mudanças e impactos positivos. Quanto à inovação, os setores que conjecturam mais benefícios são os de logística (63%), governamental (47%) e indústria médica (44%). Características mais valorizadas são a segurança e a rastreabilidade de transações e produtos (50%), armazenamento de dados distribuído (26,7%), autenticação da identidade (23,3%), compartilhamento de dados e aplicações (20%) e gerência da cadeia de suprimentos (20%); e, empresas que já implementaram a blockchain, as inovações foram mais intensas (53%), demonstrando abertura e adaptação da tecnologia às demandas particulares das empresas (Gomes; Amorim; Machado, 2022).

O blockchain age como um livro-razão digital composto de blocos de dados encadeados cronologicamente e protegidos por criptografia. Cada bloco possui um número de transações, o horário de registros e um link criptográfico para o bloco anterior (Crosby et al., 2016). Assim, tal estrutura assegura que uma vez adicionada ao blockchain, a informação se torna praticamente imutável (essencial para a confiança na rede) A sua segurança advém:

[...] do uso de criptografia de chave pública, que assegura que apenas os proprietários dos dados possam alterá-los, enquanto a imutabilidade é garantida pela natureza descentralizada da tecnologia. A descentralização é realizada por meio de uma rede de nós que validam e registram transações independentemente, uma configuração que protege contra ataques e fraudes (Hillebrecht, 2024, p. 19).

O blockchain possui variadas aplicações em distintos setores. Na logística, melhora rastreabilidade e eficiência da cadeia de suprimentos. No setor financeiro, promove transações rápidas e seguras, diminuindo dependência de intermediários; também, vem sendo explorado em contratos inteligentes, votação digital e gerenciamento de registros médicos; representando uma mudança de paradigma no modo de gerenciar informações digitais, proporcionando novo nível de transparência

e segurança; podendo vir revolucionar ainda mais indústrias e práticas globais (Hillebrecht, 2024).

2.1.1 Riscos e Benefícios do Blockchain

As instituições financeiras ficam sujeitas a uma série de riscos que podem impactar profundamente suas operações e estabilidade; dentre estes riscos, o sistêmico, que “surge da interconexão das instituições financeiras e do potencial de contágio, em que a falência de uma instituição pode levar a uma cascata de falhas na rede”; que segundo a Resolução nº 4.557/17 do Conselho Monetário Nacional, “instituições financeiras estão expostas a riscos financeiros e não financeiros, que necessitam de uma gestão integrada e uma visão alinhada à estratégia” (Sousa, 2024, p. 6).

A tecnologia Blockchain resulta em oportunidades expressivas e desafios relevantes, nomeadamente nos setores bancário e financeiro. Quanto às oportunidades, ocorre maior segurança, transparência e privacidade nas transações, transformando serviços bancários tradicionais ao promover remessa global de dinheiro, contratos inteligentes e ativos digitais a custos bem mais baixos e céleres. Mesmo oportunizando benefícios frente à mitigação de riscos e redução de custos, o Blockchain possui enormes desafios como problemas técnicos de escalabilidade, interoperabilidade e falta de padrões universais (Mishra et al., 2023).

A tecnologia Blockchain ainda apresenta muitas iniciativas em desenvolvimento e a falta de estruturas regulatórias aumenta sua complexidade. Blockchain e moedas digitais trazem inúmeros riscos para as instituições financeiras, que precisam ser meticulosamente gerenciados para aproveitar seus benefícios. O risco operacional inclui possíveis perdas frente a deficiências de confiabilidade e integridade do sistema (falhas de hardware ou software, interrupções e comprometimento do banco de dados). Esse risco fica acentuado com ameaças externas, hackers podem manejar ou modificar saldos de contas (roubos e/ou fraudes), comprometendo a reputação do banco, que depende da confiança. As fintechs são disruptoras na difusão do Blockchain, já bancos tradicionais são rígidos e hesitantes em adotar totalmente a tecnologia (Kaniadakis; Foster, 2024; Sousa, 2024).

2.2 A INTELIGÊNCIA ARTIFICIAL (IA)

A Inteligência Artificial (IA) é a área da Ciência da Computação que procura replicar funções cognitivas na resolução de problemas e gerar sistemas que podem aprender e raciocinar como os humanos. A mesma se apresenta como uma tecnologia que pode processar dados e tomar decisões de modo a superar em muito as capacidades humanas (Rigotti et al., 2024) e vem se tornando o principal meio para aprimorar os negócios (Matos, 2025).

A integração da IA nas empresas se destaca como força transformadora que remodela empreendedorismo, estratégias, dinâmica do mercado e expectativas dos consumidores. Seu célere avanço torna mais relevante o modo como empreendimentos e inovações são concebidos, desenvolvidos e gerenciados (Jeremiah, 2025; Matos, 2025).

Com a globalização, as organizações ingressaram na corrida pela IA; se renovando, sendo proativas, se preparando para as mudanças, antecipando processos futuros. Esse cenário acontece por força dos clientes, concorrentes atuais e potenciais, estruturação de cada setor, relações com fornecedores e empresas. Mudanças nas expectativas do quadro funcional, questões financeiras, modos de se investir, visão dos gestores e o ambiente tecnológico permeiam as estratégias das empresas e, nesse caminho, desenvolver e/ou adquirir novas ferramentas tecnológicas, know-how, IA, sistemas de virtualização e automação se mostraram elementos decisivos na competição (Yeke 2023).

A IA ampliou as oportunidades de trabalho com a inovação em robótica (envolvendo IA e Internet das Coisas - IoT). A IA pode funcionar ser uma ferramenta poderosa frente desenvolver alternativas de marketing para produtos (criação de material para postagens nas mídias sociais) (Jatmika et al, 2024). No setor financeiro e mercado de ações, a Long Short-Term Memory (LSTM - tipo de rede neural artificial) pode prever eficazmente preços do mercado de ações, cruzando dados em inúmeros intervalos de tempo; algoritmos metaheurísticos como o Artificial Rabbits Optimization (ARO), podem otimizar os hiper parâmetros de um modelo LSTM e melhorar as previsões do mercado de ações (Matos, 2025).

A indústria, mídia e organizações políticas têm interesse crucial no potencial da IA. De 2011 a 2016, empresas de capital aberto como Google, Microsoft, Twitter, Intel, Apple e outros gigantes da tecnologia da informação compraram cerca de 140

empresas que pesquisavam a IA. No primeiro semestre de 2016, 200 empresas que trabalham com IA operaram 1,5 bilhão de dólares no mercado de ações. Múltiplas fusões e aquisições aceleram o uso da IA e, o aprendizado profundo (Deep Learning) vem sendo uma tecnologia relevante no mercado industrial. A IA tornou-se tecnologia principal em uso no Google Glass, veículos não tripulados e outros (Matos, 2025).

2.3 AS CRIPTOMOEDAS

Em 2008, Nakamoto expôs o Bitcoin, uma criptomoeda para resolver demandas como custo de mediação, possibilidade de transações irreversíveis, valor mínimo de transação, anonimato na transação e garantia de impossibilidade de fraudes; atraindo a atenção, sobretudo na área de investimentos, por sua descentralização e autorregulagem, através do sistema peer to peer, ou P2P. O uso dessa arquitetura permite que transações com criptomoedas sejam de acesso comum a todos, com verificação de toda transação efetivada, devido a tecnologia Blockchain, também apresentado por Nakamoto (Santos; Igarashi; Igarashi, 2023).

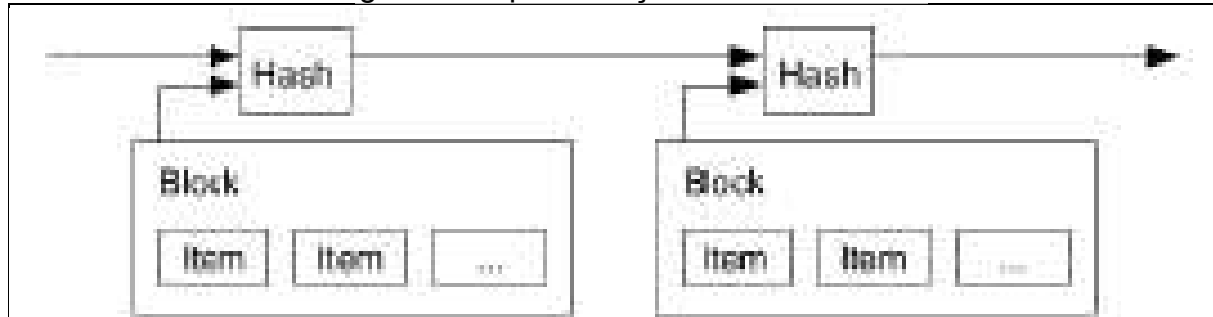
Acontecem infinitas possibilidades e fins para o uso do Bitcoin, sendo forte alternativa para investidores frente reserva de valor; que se trata de um bem com atributo de preservar seu poder de compra no decorrer do tempo. O Bitcoin é uma moeda autorregulada, descentralizada com circulação limitada e valorização em longo prazo sendo positiva. Investidores consideram o Bitcoin uma moeda como bom modo de lucrar através da especulação, pois sendo volátil e sofrendo variações em um curto período de tempo; a ação de compra e venda de ativos em curto ou médio prazo, geram lucro; para tanto, costuma envolver a análise técnica de gráficos e de indicadores estatísticos (Santos; Igarashi; Igarashi, 2023).

A concepção de ativos digitais descentralizados com base em criptografia, ou criptomoedas, teve inúmeras tentativas; mas, só ocorreu com o advento da Blockchain (Mollin et al., 2020). Essa ferramenta é:

[...] um registro compartilhado distribuído, no qual a verificação, o armazenamento, a manutenção e a transmissão dos dados são baseados na confiança mútua entre as partes, estabelecida por meio de algoritmos matemáticos. Tal estrutura é formada por blocos que contêm um conjunto de transações, sendo que cada um possui um identificador exclusivo chamado Hash, gerado a partir das informações presentes no bloco, incluindo as do

anterior. Esse encadeamento de blocos cria uma sequência contínua e cronológica na qual os blocos são ligados ao anterior, formando uma cadeia (Oliveira, 2025, p. 18).

Figura 1: Representação da Blockchain



Fonte: Oliveira (2025, p. 18)

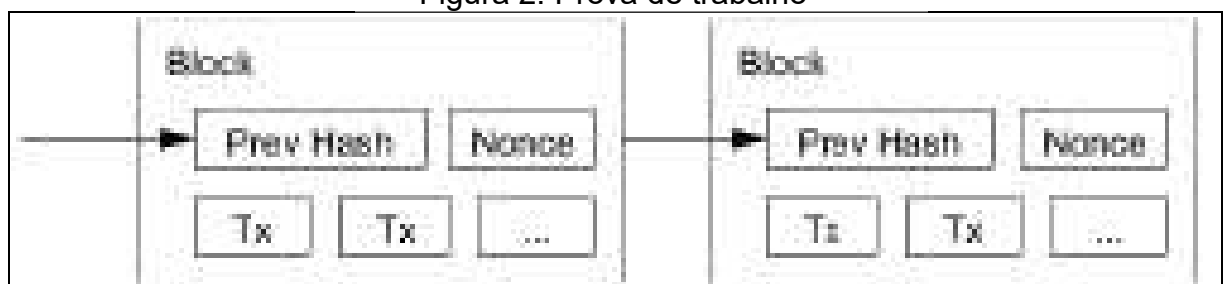
O Bitcoin é o primeiro e mais conhecido ativo, conhecido como um dinheiro eletrônico negociado direto entre as partes, sem passar por instituição financeira. Para Sousa et al. (2022), as tecnologias com base em Blockchain são mais céleres, ágeis e seguras, auferindo muito espaço na mídia, e atraindo interesse de investidores individuais e fundos.

Segundo Oliveira (2025, p. 18-19):

Cada Bitcoin é subdividido em 100 milhões de unidades menores chamadas Satoshis, ou, simplesmente, "Sats". Seu armazenamento é feito em uma carteira digital criptografada de maneira assimétrica Offline, chamada Cold Wallets, ou Online, em serviços de Hot Wallets [...]. A transferência ocorre por prova de trabalho; um validador chamado de minerador é responsável por resolver um problema matemático de dificuldade variável. Além disso, serve como nó que confere a operação de outros mineradores e garante a integridade da rede. Quando o problema é resolvido, adiciona-se um bloco de transação na Blockchain; em troca, o minerador recebe uma recompensa em Bitcoins [...]. A dificuldade da prova de trabalho é ajustada automaticamente pelo Hashrate, indicador que mede a quantidade de poder computacional necessário à mineração. Quanto maior o Hashrate, mais difícil é encontrar o próximo bloco, regulando o tempo entre a adição de novos blocos à rede, mantendo uma média de 10 minutos.

A figura a seguir representa a prova de trabalho.

Figura 2: Prova de trabalho



Fonte: Oliveira (2025, p. 18)

A cada quatro anos, a quantidade de Bitcoins criadas por bloco é reduzido pela metade (evento chamado Halving). O fator de redução confere à moeda uma escassez programada (limitada a 21 milhões de unidades), que a torna deflacionária. Contratos inteligentes, DeFi (Finanças Descentralizadas), NFTs (Tokens Não Fungíveis) e a IA generativa cominam uma nova era de aplicações e serviços (Books, 2024).

No cenário de negociação do Bitcoin, a Exchange mais conhecida é a Binance, que opera em vários países com volume de transações diário de bilhões de dólares. Com uma alta liquidez, o Spread (diferença entre preço de compra e venda) é baixo (valor bem acurado frente o oferecido no mercado). Há vários sites de comparação de preços, como o CoinGecko e CoinMarketCap (portais que classificam plataformas, moedas e posição atual do mercado), por meio de índices (Oliveira, 2025), como apresenta a Tabela a seguir.

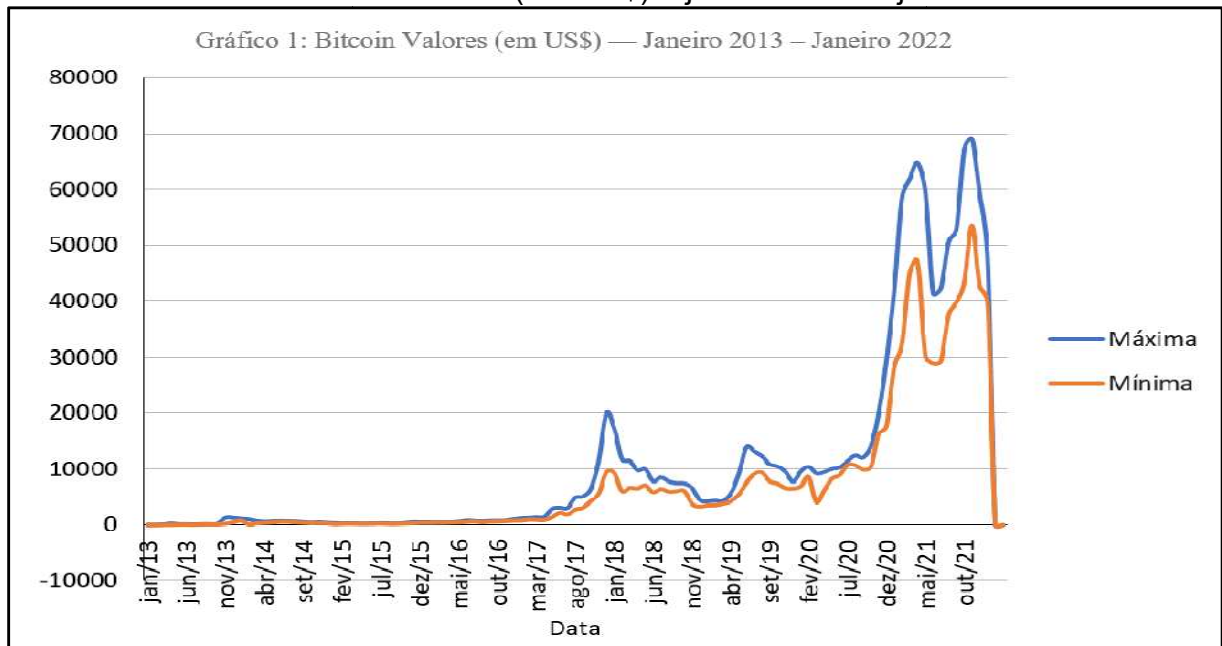
Tabela 1: Exchanges com maior volume e confiabilidade

Exchange	Volume 24h (BTC)	Score Coingecko	Score CoinMarketCap
Binance (Global)	135.157	10/10	9,9/10
Bybit	44.720	10/10	7.6/10
HTX (Huobi)	34.250	9/10	6.9/10

Fonte: Oliveira (2025, p. 20)

Bitcoin e blockchain permitem uma relação de trocas financeiras descentralizadas de fácil acesso e com processos rigorosos de segurança e criptografia. Assim, os mercados financeiros coadunaram com a implementação desse novo sistema de pagamentos, fortalecendo o mercado de ativos financeiros. Prova disso é o aumento de investimentos empresariais e individuais, trabalhadores profissionais e não profissionais (traders e day-traders) para esse mercado, atraídos por suas oportunidades (Almada; Costa, 2023), como mostra o gráfico a seguir.

Gráfico 1: Bitcoin valores (em US\$) – janeiro 2013 – janeiro 2022



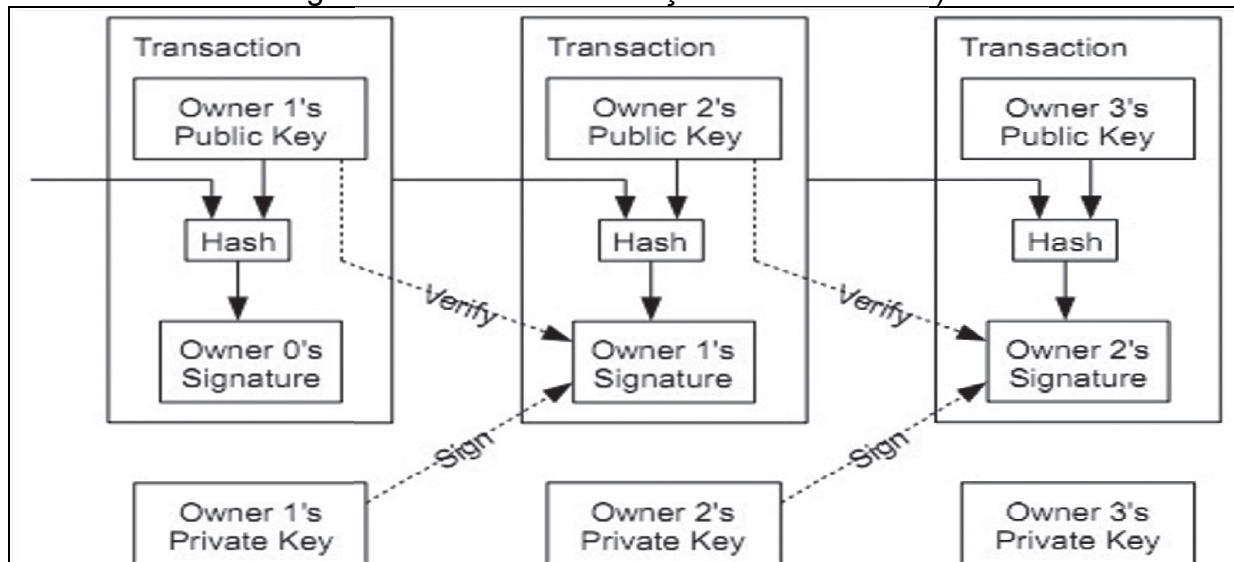
Fonte: Almada e Costa (2023, p. 6)

A estrutura blockchain está documentada na literatura científica e analisados os algoritmos, contratos inteligentes (smart contracts) e as criptomoedas. existem fases em torno da blockchain: a primeira fase (blockchain 1.0), com ênfase no desenvolvimento de criptomoedas e Bitcoin, a segunda fase (blockchain 2.0), com ênfase na criação de contratos inteligentes e comandos para gradual expansão das áreas de aplicação. Na área industrial, por exemplo, a integração da blockchain com o sistema de Internet das Coisas (IoT), gerou a Blockchain das Coisas (BCoT), novo paradigma de arquitetura, que interopera dispositivos de IoT com setores industriais que interage com sistemas físicos, oferecendo trocas de informações, que permite rastrear, confiar na qualidade dos dados, interações autônomas, satisfazendo cláusulas predispostas nos contratos inteligentes e ausência de violações (Almada; Costa, 2023).

Internamente ao protocolo, a integração entre as transações resulta em cumular as criptografias: cada transação é criptografada por um método (SHA-256), que gera algoritmos de criptografia. Os algoritmos da operação anterior se adicionam à operação posterior e, assim, sucessivamente; gerando uma chave de proteção para cada operação, repassada ao agente e/ou cliente que fez a operação; por conseguinte, só com essa chave se permite verificar o conteúdo da operação. Com operações seguidas e acumuladas, geram-se novos algoritmos; assim, para

fraudar esse sistema, é preciso que todos os pacotes anteriores sejam abertos com suas respectivas chaves de acesso, implicando uso de alto poder computacional, gastos de energia e tempo (Almada; Costa, 2023). A estrutura básica desse modelo se descreve conforme a Figura a seguir.

Figura 3: Modelo de transações da blockchain)



Fonte: Almada e Costa (2023, p. 7).

Criptomoeda é o “nome genérico para moedas digitais descentralizadas, criadas em uma rede blockchain a partir de sistemas avançados de criptografia que protegem as transações, suas informações e os dados de quem transaciona” (Gomes; Amorim; Machado, 2022, p. 2).

De início, as criptomoedas eram apenas curiosidade tecnológica sem comércio organizado. Contudo, a concepção inovadora logo se propagou fora do círculo original de geeks de computador, estimulando o setor financeiro e, também, devido ao anonimato para os círculos criminosos. A primeira corretora a possibilitar troca de Bitcoins por moedas tradicionais, a Mt.Gox, foi lançada em julho de 2010. Em seguida foi criado o primeiro mercado negro online, Silk Road, primeira aplicação prática de criptomoedas para fins comerciais, que permitiu negociar fora de regulamentação qualquer mercadoria (Watoreket al., 2021).

A tecnologia blockchain na qual as criptomoedas são baseadas, permite o uso de um registro descentralizado não apenas para fins financeiros. Assim como tão revolucionária foi a internet, essa nova tecnologia está evoluindo de um sistema seguro de transações monetárias para parte de um ecossistema de tecnologias

emergentes. Espera-se que a tecnologia blockchain “melhore a colaboração entre empresas e indivíduos, a transparência dos processos e dados de negócios e, em última análise, a produtividade e a sustentabilidade da economia” (Gomes; Amorim; Machado, 2022, p. 2).

O crescimento do Bitcoin e outras criptomoedas confirmam a atenção e relevância do tema para o cenário dos negócios. Conforme Gomes, Amorim e Machado (2022, p. 2-3):

[...] desde o início da possibilidade de negociações regulares de troca entre Bitcoin e dólar americano em julho de 2010 pela Mt. Gox, o número de transações começou a crescer exponencialmente. [...] embora o Bitcoin seja transacionado desde 03 de janeiro de 2009, apenas configurou um comércio ativo em 2013. [...] o número de transações confirmadas por dia de Bitcoins, que destaca o valor da rede Bitcoin como uma forma de transferir fundos em segurança sem uma terceira parte, em fevereiro de 2009 era cerca de 124.000, tendo aumentado 368% até agosto de 2010, quando se realizavam 580.000 transações/dia. De agosto de 2010 a agosto de 2021, o número diário de transações usando Bitcoin aumentou 40.000% de 580 mil para 230 milhões, implicando uma taxa de crescimento anual composto para o período de 72% ao ano [...]. O valor de mercado do total de criptomoedas disponíveis que era de US\$ 1,6 bilhões em abril de 2013, aumentou cerca de 140.000% em 8 anos, chegando a US\$ 2,2 trilhões, em abril de 2021 [...]. Já quanto ao número de criptomoedas transacionadas, que em 2013 era de apenas 66, já figurava 7.557 no final de 2021, embora as 20 principais criptomoedas representem quase 90% do mercado total.

Diante desses dados, as criptomoedas apontam forte potencial de impactar expressivamente a conduta futura dos negócios, assim como diversas disciplinas acadêmicas demandam pesquisas para compreender esse fenômeno. Existem quatro principais correntes de pesquisa sobre criptomoedas: questões tecnológicas visando investigar problemas criptográficos, segurança do sistema e vulnerabilidade a ataques; questões públicas e legais para examinar como as criptomoedas são tratadas em distintas jurisdições legais; questões políticas, sociológicas e éticas frente ao fenômeno das criptomoedas; e, questões econômicas (Gomes; Amorim; Machado, 2022).

A tabela a seguir apresenta as quatro maiores criptomoedas por ordem de valor de mercado. Evidencia-se para cada criptomoeda o valor de mercado e o preço em dólar, desde a primeira informação de suas séries históricas até 31 de janeiro de 2022, evidenciando o potencial de crescimento em valor de mercado, preço e estoque de circulação, comprovando sua expansão no sistema financeiro internacional (Gomes; Amorim; Machado, 2022).

Tabela 2: Quatro maiores criptomoedas em valor de mercado e sua variação

			Início da série histórica		31 de janeiro de 2022			
	Nome	Dados desde	Valor de mercado (USD)	Preço (USD)	Valor de mercado (USD)	Preço (USD)	Estoque em circulação	Variação valor do mercado
1	Bitcoin (BTC)	28/04/13	1.600mi	144,54	730.500mi	38.632,00	19mi BTC	45.556%
2	Ethereum (ETH)	07/08/15	45,49mi	0,75	330.000mi	2.771,00	120mi ETH	725.175%
3	Tether (USDT)	25/02/15	0,3mi	1,20	70.000mi	1,00	78.000mi USDT	25.999.900%
4	BNB (BNB)	25/07/17	10,51mi	0,1	63.200mi	384,00	165mi BNB	601.232%

Fonte: Gomes, Amorim e Machado (2022, p. 7).

Para Nakamoto (2009), criptomoedas (moedas eletrônicas) se define como “uma cadeia de assinaturas digitais, onde cada proprietário transfere a moeda para um destinatário, assinando digitalmente um hash da transação anterior e a chave pública do próximo proprietário e adicionando-os ao final da moeda” (apud Leite, 2023, p. 15).

Segundo a plataforma de mercados financeiros Investing, criptomoeda é “um ativo digital criptografado que pode ser usado como meio de troca ou reserva de valor”.¹ Para o banco digital Nubank, criptomoeda é o “nome genérico para moedas digitais descentralizadas, criadas em uma rede blockchain a partir de sistemas avançados de criptografia que protegem as transações, suas informações e os dados de quem transaciona”².

Conforme Leite (2023, p. 19):

A criptografia única de cada criptomoeda controla sua oferta monetária para evitar sua desvalorização, bem como o meio pelo qual as unidades podem ser criadas, armazenadas e transferidas com segurança [...] não são respaldadas por governos, bancos ou ativos subjacentes, como o ouro. Em vez disso, a moeda é controlada através de um sistema descentralizado de livros contábeis públicos, chamados blockchains. Esses livros contábeis são mantidos por particulares em computadores individuais, que formam o chamado sistema peer-to-peer, responsável por evitar que o valor do ativo seja afetado por qualquer entidade, ou seja, significa que as transações são gerenciadas por cada um dos usuários que fazem parte do sistema.

O funcionamento do envio de criptomoedas pode ser resumido em:

1. Uma pessoa deseja enviar uma criptomoeda para outra;

¹ Disponível em: <https://br.investing.com/education/terms/criptomoeda-200226261> Acesso em out. 2025.

² Disponível em: <https://blog.nubank.com.br/o-que-e-criptomoeda/> Acesso em out. 2025.

2. O ativo é representado online como um bloco onde os detalhes estão armazenados;
3. O bloco é distribuído pela rede e cada máquina fica com uma cópia da transação em tempo real;
4. A rede verifica se o ativo é válido em questão de minutos;
5. Se aprovado, o bloco é adicionado a uma corrente de blocos e ganha um registro permanente na rede. Isso significa que ele não pode ser alterado;
6. A propriedade do ativo agora fica registrada na rede como sendo da pessoa que recebeu. Toda essa movimentação é validada pela rede, mas ninguém sabe quem está fazendo a transação (Leite, 2023, p. 20).

O Bitcoin e várias outras criptomoedas usam criptografia de chaves pública e privada. Nela, a rede da criptomoeda emite a todo usuário uma chave privada (senha muito forte) e, a partir desta, gera uma chave pública por hashing. Esta última pode ser passada para outros sujeitos, sendo a única informação precisa para que um usuário envie unidades da criptomoeda para outro. Com as chaves pública e privada vinculadas, a rede sabe as unidades de uma criptomoeda que pertencem a certo usuário (Coinbase, 2022).

As cotações das criptomoedas se fazem em um par, representado por uma moeda digital e uma moeda física. Por exemplo, a compra de Bitcoin (BTC) com moeda brasileira (o real - BRL), expresso por BTC/BRL seguido de um valor, exibe a quantidade de reais necessária para comprar uma unidade de Bitcoin (Leite, 2023).

O cenário marcando o surgimento das criptomoedas descentralizadas deixa evidente: deslocar o controle das moedas (estatal ou privado), das instituições financeiras e Bancos Centrais, objetivando um sistema monetário mais rigoroso e autogerido; refletindo aspirações de grupos anarquistas das décadas de 1980 e 1990, que viram na criptografia um modo de combater a regulação e vigilância estatais. Logo, o Bitcoin e as criptomoedas se transacionam num sistema de pagamentos descentralizado (o blockchain), sem a presença de intermediários financeiros (Hillebrecht, 2024).

2.3.1 Tipos de criptomoedas

Depois do advento no mundo das moedas virtuais, verificou-se um avanço significativo na criação de várias moedas digitais com os mais variados fins. Nesse panorama diverso, as criptomoedas alcançaram destaque frente sua valorização e estabilidade no decorrer dos anos como o Bitcoin, Ethereum e Tether

(Hillebrecht, 2024). Na figura a seguir verificam-se as principais moedas virtuais do mercado, segundo o Market Cap.

Figura 4: Principais criptomoedas com capitalização de mercado (2024)

#	Name	Price	1h %	24h %	7d %	Market Cap ⓘ	Volume(24h) ⓘ	Circulating Supply ⓘ	
☆ 1	 Bitcoin BTC	\$64,234.97	▼ 0.14%	▲ 0.55%	▼ 3.13%	\$1,266,410,712,112	\$15,073,795,400 234,644 BTC	19,715,284 BTC	
☆ 2	 Ethereum ETH	\$3,497.49	▼ 0.23%	▼ 0.02%	▼ 2.20%	\$427,659,387,319	\$10,319,439,330 2,949,634 ETH	122,276,217 ETH	
☆ 3	 Tether USDT	\$0.9996	▲ 0.01%	▲ 0.02%	▼ 0.01%	\$112,892,022,492	\$34,960,329,705 34,977,577,210 USDT	112,941,691,563 USDT	
☆ 4	 BNB BNB	\$586.41	▼ 0.09%	▲ 0.61%	▼ 3.50%	\$86,534,012,454	\$1,440,232,763 2,456,082 BNB	147,565,989 BNB	
☆ 5	 Solana SOL	\$134.14	▼ 0.30%	▲ 1.77%	▼ 7.54%	\$61,974,168,703	\$1,472,751,399 10,947,681 SOL	462,001,830 SOL	
☆ 6	 USDC USDC	\$1.00	▼ 0.00%	▲ 0.01%	▼ 0.02%	\$32,754,747,734	\$3,547,816,870 3,547,785,055 USDC	32,753,231,418 USDC	
☆ 7	 XRP XRP	\$0.4856	▼ 0.35%	▼ 0.06%	▲ 0.33%	\$27,008,486,360	\$623,380,797 1,283,529,625 XRP	55,618,185,850 XRP	
☆ 8	 Dogecoin DOGE	\$0.1251	▲ 0.17%	▲ 1.73%	▼ 8.74%	\$18,120,905,921	\$396,225,255 3,166,891,598 DOGE	144,815,776,384 DOGE	
☆ 9	 Toncoin TON	\$7.29	▲ 0.84%	▲ 1.85%	▼ 9.13%	\$17,915,702,243	\$210,483,361 28,917,713 TON	2,458,203,288 TON	
☆ 10	 Cardano ADA	\$0.3855	▼ 0.88%	▲ 1.05%	▼ 6.47%	\$13,776,901,924	\$269,761,545 699,401,249 ADA	35,740,253,204 ADA	

Fonte Hillebrecht (2024, p. 17).

A figura a seguir mostra o ranking das criptomoedas em 29 de outubro de 2025.

Figura 5: Ranking das criptomoedas (out. 2025)

Nome ▾	Código ▾	Preço (USD) ▾	Vol. (24h) ▾	Vol. Total ▾	Var (24h) ▾	Var (7d) ▾	Valor de Mercado ▾
 Bitcoin	BTC	111.160,6	\$67,63B	37,81%	-1,47%	+2,79%	\$2,21T
 Ethereum	ETH	3.927,97	\$40,16B	20,64%	-1,31%	+3,21%	\$472,04B
 Tether USDt	USDT	1,0003	\$144,62B	79,23%	+0,03%	+0,02%	\$183,37B
 XRP	XRP	2,5869	\$5,49B	2,98%	-0,30%	+8,82%	\$154,62B
 BNB	BNB	1.113,91	\$4,78B	2,75%	+2,04%	+3,89%	\$152,80B
 Solana	SOL	195,317	\$8,67B	4,61%	+1,26%	+7,63%	\$106,23B
 USDC	USDC	0,9997	\$19,72B	10,30%	-0,01%	+0,01%	\$76,24B
 Dogecoin	DOGE	0,195301	\$2,93B	1,56%	+1,16%	+2,19%	\$29,44B
 TRON	TRX	0,295785	\$1,01B	0,51%	+0,32%	-7,87%	\$27,96B
 Cardano	ADA	0,6541	\$1,37B	0,76%	+1,60%	+3,96%	\$23,33B
 Hyperliquid	HYPE	48,8069	\$718,94M	0,40%	+3,33%	+32,93%	\$16,01B
 Chainlink	LINK	18,36	\$1,04B	0,54%	+3,89%	+6,26%	\$12,73B
 Bitcoin Cash	BCH	559,83	\$627,61M	0,35%	+0,85%	+17,93%	\$11,13B
 Stellar	XLM	0,32150	\$492,66M	0,32%	+1,55%	+3,22%	\$10,23B
 Ethena USDe	eUSDe	0,999	\$276,80M	0,15%	0,00%	+0,03%	\$9,65B
 Sui	SUI	2,5732	\$1,10B	0,60%	+3,03%	+5,19%	\$9,23B
 UNUS SED LEO	LEOu	9,6400	\$969,86K	0,00%	+1,06%	+7,09%	\$8,88B
 Hedera	HBAR	0,20242	\$605,93M	0,34%	+3,85%	+20,43%	\$8,52B

Fonte: Investing (2025).³

2.3.1.1 Bitcoin

Anterior ao Bitcoin em 2008, criado por Satoshi Nakamoto, as transações online dependiam de intermediários terceirizados como PayPal ou Mastercard; registrando os saldos das contas de seus usuários, debitando e creditando valores segundo as transações. Sem essa intermediação, existia o risco de valores digitais serem usados mais de uma vez (gasto duplo). O Bitcoin inovou trazendo a solução para esse gasto duplo sem precisar de terceiros, por meio da tecnologia blockchain, que inaugurou a primeira rede de pagamento global totalmente descentralizada (Ulrich, 2014).

Distintamente dos sistemas tradicionais de pagamento, as transações na rede Bitcoin (bitcoins), se apresentou como moeda virtual cujo valor não se baseia em

³³ Disponível em: <https://br.investing.com/crypto> Acesso em 29 out. 2025.

ouro ou decretos governamentais, e sim na percepção de valor pelos usuários. O preço do Bitcoin se determina no mercado aberto, similar às taxas de câmbio entre moedas convencionais, o que torna o Bitcoin uma rede de pagamentos descentralizada e uma moeda virtual (Hillebrecht, 2024).

O Bitcoin criado por Nakamoto (2009) com a publicação chamada “Bitcoin: A Peer-to-Peer Electronic Cash System”, é “um sistema de caixa eletrônico totalmente descentralizado que não depende de uma autoridade central para circulação de moeda e autenticação de transações” (Leite, 2023, p. 20). O nome Satoshi Nakamoto é um pseudônimo e se desconhece a identidade do indivíduo ou indivíduos por trás desta invenção. Contudo, nem o autor, nem ninguém, exerce controle sobre o sistema Bitcoin; sendo uma invenção inovadora que gerou novas ciências nas áreas de computação distribuída, economia e econometria (Leite, 2023).

Bitcoin é uma criptomoeda “criada em 2008 por um indivíduo ou grupo de indivíduos sob o pseudônimo de Satoshi Nakamoto [...] tem ganhado cada vez mais atenção de investidores e traders em todo o mundo devido ao seu potencial de lucro e à sua natureza descentralizada” (Calderaro, 2023, p. 25).

Diferentemente das moedas tradicionais, o Bitcoin funciona em uma rede de computadores peer-to-peer, onde usuários realizam transações diretas entre si, sem intermediários; tendo como vantagem a segurança. Conforme Calderaro (2023, p. 25):

Todas as transações são registradas em um livro contábil digital, chamado de blockchain, que é distribuído em vários computadores ao redor do mundo. Isso significa que é muito difícil para qualquer pessoa ou entidade adulterar as transações, tornando o Bitcoin resistente a fraudes e hackeamentos. Além disso, o Bitcoin também oferece privacidade aos seus usuários, pois as transações são realizadas de forma pseudônima, ou seja, sem a necessidade de identificação pessoal. Isso permite que as pessoas mantenham sua privacidade financeira e protejam suas informações pessoais.

No cenário macroeconômico, o Bitcoin se faz um ativo de refúgio seguro contra a inflação e desvalorização. Como não tem controle de governo ou autoridade central, não se sujeita às políticas monetárias (nem inflação ou desvalorização). Sendo considerado uma reserva de valor, semelhante ao ouro, frente a oferta limitada e à crescente demanda (Narayanan et al., 2016).

Mesmo sendo considerado uma moeda volátil e de alto risco, sua popularidade e adoção são fortes globalmente, sobretudo por investidores e entusiastas de tecnologia. Destaca-se que, no período entre 16/08/2022 a

14/08/2023, o Bitcoin teve volume médio diário de negociação de mais 24 bilhões de dólares e valor de mercado médio de mais de 462 bilhões de dólares, sendo a moeda mais negociada e com maior valor de mercado no período (Calderaro, 2023).

2.3.1.2 Ethereum

A segunda criptomoeda de grande estabilidade mundial continua sendo o Ethereum, criado por Vitalik Buterin em 2013, que aumenta as capacidades do Bitcoin ao lidar com transações monetárias e executar contratos inteligentes e aplicações descentralizadas (DApps). Conforme Antonopoulos e Wood (2018), esta plataforma global admite criar e operar aplicações sem intermediários, usando um sistema mais livre e flexível do que blockchains tradicionais. A proposta de Buterin foi ter-se um ambiente onde diferentes scripts de programação podem ser executados, aceitando um rol mais extenso de transações e contratos digitais.

Segundo Hillebrecht (2024, p. 18):

Lançado oficialmente em 30 de julho de 2015, o Ethereum introduziu o conceito de “contratos inteligentes”, programas que executam automaticamente os termos de um contrato quando condições predefinidas são cumpridas. Essa inovação promove a automação e transparência em diversas aplicações, transformando a forma como transações e acordos são realizados digitalmente.

A criptomoeda associada, Ether, é utilizada para gerenciar operações computacionais na rede, afirmando que os recursos sejam alocados eficientemente e impulsionando o procedimento correto dentro da plataforma (Hillebrecht, 2024).

2.3.1.3 Tether

A criptomoeda tether (USDT) mais conhecida como stablecoin (moeda estável) foi projetada para sustentar valor estável, comumente indexada a uma moeda fiduciária, como o dólar americano (USD), onde para cada USDT em circulação é afiançado por um dólar americano mantido em reserva pela empresa emissora (Tether, 2024).

Nesse caminho, “busca-se manter uma ponte entre o setor financeiro tradicional e o mercado de criptomoedas, facilitando assim, a realização de transações

entre grandes empresas simplificando a conversão de dólares para os investidores individuais” (Hillebrecht, 2024, p. 19).

2.4 ANÁLISE TÉCNICA

Conforme Santos, Igarashi e Igarashi (2023, p. 184), a análise técnica:

[...] consiste no estudo dos movimentos do mercado e tem como propósito a previsão de tendências de preços futuros. Portanto, busca-se por meio desta análise a identificação de padrões e tendências possíveis de serem exploradas em busca de melhores rentabilidades do que as da média do mercado. A análise técnica utiliza diversas ferramentas, entre as quais há o grupo dos indicadores técnicos. Os indicadores técnicos permitem identificar pontos de entrada e saída do mercado, auxiliando os investidores na maximização do retorno de seu capital. Um aspecto importante dos indicadores técnicos é a objetividade por trás desses sistemas, pois é eliminada a possibilidade de interferência emocional na tomada de decisões, fazendo com que o operador aja conforme os sinais indicam.

Destacam-se três fundamentais fontes de informações disponíveis e importantes para tal análise: o preço, o volume e o interesse dos investidores. Segundo Kikuti, Igarashi e Igarashi (2022, p. 3), a análise técnica é “o estudo de ativos (como ações e criptomoedas) do mercado, primordialmente a partir do uso de gráficos com o propósito de prever a tendência futura de preços”. A frase “market actions discounts everything”⁴ forma a base para se compreender a teoria por trás da análise técnica. Assim, a análise do preço de certo ativo se apresenta suficiente para prever seu comportamento. “A análise técnica se baseia na análise gráfica dos preços de um ativo e na utilização de indicadores estatísticos como ferramenta para a previsão de momentos de compra, venda e manutenção desse ativo” (Santos; Igarashi; Igarashi, 2023, p. 184).

A análise técnica se categoriza em dois grupos principais:

- Análise técnica empírica: Esta abordagem tradicional se baseia em métodos que identificam padrões repetitivos no mercado. Apesar de não haver uma explicação racional clara ou uma relação de causa e efeito para tais padrões, eles são considerados confiáveis devido à sua consistência e recorrência ao longo do tempo.

⁴O princípio do desconto de mercado afirma que os preços atuais das ações já refletem todos os eventos futuros e passados. Esse princípio leva em consideração eventos que vão desde as emoções dos investidores até fatores macroeconômicos, como a inflação. O desconto de mercado é um dos seis princípios da negociação. A frase em questão, forma o pilar base para a aceitação e compreensão de toda teoria por trás da análise técnica. Os investidores que utilizam a análise técnica para fazer seus investimentos e especulações, acreditam que tudo pode possivelmente influir no preço de um ativo no mercado financeiro (Kikuti; Igarashi; Igarashi, 2022, p. 3-4).

- Análise técnica estatística: Esta abordagem mais moderna utiliza modelos matemáticos e estatísticos que são desenvolvidos a partir de dados históricos do mercado, como preços e volumes de negociação (Hillebrecht, 2024, p. 19-20).

Para analisar a tendência do mercado, utiliza-se a análise técnica, que é um rol de métodos e ferramentas para identificar padrões futuros baseado no comportamento histórico do mercado. “Utilizando indicadores específicos, os investidores podem formular estratégias de negociação, prevendo quais ativos estão propensos a valorizar ou desvalorizar” (Hillebrecht, 2024, p. 19).

2.5 INDICADORES TÉCNICOS

Conforme Hillebrecht (2024, p. 20), um indicador técnico se entende por:

[...] um conjunto de dados obtidos pela aplicação de uma fórmula aos preços de um ativo financeiro, que podem incluir qualquer combinação de preços de abertura, máxima, mínima ou fechamento em um determinado período. Alguns indicadores utilizam somente os preços de fechamento, enquanto outros também incorporam dados de volume e contratos em aberto em suas fórmulas. Os preços são inseridos na fórmula, resultando em um dado específico [...]. Indicadores técnicos na análise de preços fornecem perspectivas variadas, desde fórmulas simples, como médias móveis, até complexas, como o estocástico.

Analistas técnicos habitam utilizar indicadores estatísticos e financeiros para facilitar a identificação de certos padrões nos ativos. Tais indicadores foram desenvolvidos para auxiliar na identificação de determinadas tendências de mercado em manter preços em alta, em baixa ou neutro. Há vários tipos de indicadores na análise técnica, tais como:

- Indicadores de tendência: buscam por padrões para indicar a direção do mercado assim como momentos de reversão;
- Osciladores: ajudam identificar pontos de mudança com antecedência; e,
- Indicadores de volume: representam o volume transacionado (Santos; Igarashi; Igarashi, 2023).

2.5.1 Média móvel simples

Uma média móvel simples (MMS ou simple moving average – SMA) é, conforme Santos, Igarashi e Igarashi (2023, p. 184-185), “uma série temporal

composta de valores obtidos ao calcular a média de outra sequência de valores, sendo definida como uma forma de convolução matemática”. A MMS é um dos indicadores mais versáteis e mais usados, pois é de fácil uso; “enquanto a análise de gráficos é subjetiva e difícil de ser feita, MMS podem facilmente ser programadas em um computador. Técnicos podem discordar na leitura de um gráfico, mas não do sinal emitido por uma MMS”.

O tamanho da MMS (a quantidade de períodos considerados) precisater variações de acordo com o mercado. “Uma MMS maior irá produzir um atraso para mudanças bruscas na tendência, mas em contrapartida produzirá um sinal mais conservador. Quanto maior a média, maior o atraso para se verificar uma nova tendência” (Santos; Igarashi; Igarashi, 2023, p. 185)

Partindo do indicador MMS aparecem outros indicadores, como a média móvel exponencial (MME ou Exponential Moving Average - EMA), uma alternativa para certas defasagens deixadas pelas MMS. O mesmo atribui um peso maior aos preços mais recentes. O cálculo de uma MME ocorre da seguinte maneira:

MME: {Fechamento – MME (dia anterior)} × fator + MME (dia anterior)

onde: fator = $2/(n+1)$, e n é o número de períodos.

2.5.2 MACD (Moving average convergence divergence)

Outro indicador técnico que usa as médias móveis é o MACD (Moving average convergence divergence) largamente usado por analistas e traders. O mesmo transforma duas médias móveis em um indicador de momento, subtraindo a média móvel de maior período pela média móvel de menor período (Santos; Igarashi; Igarashi, 2023).

O MACD trabalha com três parâmetros. Segundo Santos, Igarashi e Igarashi (2023, p. 185):

[...] a linha do MACD, a linha do sinal e o histograma do MACD. A linha do MACD trata-se de uma MME de período de 12 dias subtraída de uma MME de período de 26 dias, para os preços de fechamento. A linha de sinal é definida por uma MME da linha MACD (geralmente de 9 períodos) e é utilizada para identificar pontos de virada. O histograma é o resultado da subtração da linha do MACD e a linha de sinal. Quando a linha do MACD está acima da linha de sinal o histograma é positivo, do contrário é negativo.

2.5.3 Relative Strength Index

O Relative Strength Index (RSI), criado Welles (1978) avalia a tendência de força de um ativo dada sua variação de preço. O mesmo é usualmente utilizado junto com outros indicadores, podendo sugerir possíveis zonas de sobrecompra e sobrevenda. Exibido como um gráfico pode variar de 0 a 100 (Santos; Igarashi; Igarashi, 2023). Calcula-se o RSI da seguinte forma:

$$RSI = 100 - 100 / (1 + RS)$$

onde: $RS = \text{Média de } (fechamentosAlta) / \text{Média de } (fechamentosBaixa)$

Ao obter o valor do indicador é possível realizar algumas inferências sobre o valor calculado: Entre 0 e 30 o RSI indica uma zona de sobrevenda. Indica boa oportunidade para compra e evitar vendas no período; Entre 30 e 70 não há informação específica; Entre 70 e 100 o RSI sinaliza uma zona de sobrecompra, boa oportunidade para venda e evitar realização de compras no período (Santos; Igarashi; Igarashi, 2023).

2.5.4 Oscilador estocástico

O oscilador estocástico é um indicador de volume criado por Lane (1984), que avalia a posição relativa do preço de fechamento comparada ao ponto mais baixo, em determinados dias, frente à diferença entre o ponto mais alto e o ponto mais baixo. É caracterizada pelo uso de duas linhas que se movimentam simultaneamente, a linha %K (estocástico bruto), dada pela divisão entre a diferença do preço de fechamento corrente e a menor mínima do período definido e a diferença do fechamento corrente e a maior máxima do período definido; e, a %D dada pela média móvel de %K em dado período (Santos; Igarashi; Igarashi, 2023).

$$\%K = (C - Ll(n)) / (Hh - Ll(n))$$

onde C = preço de fechamento, $Ll(n)$ = mínimo dos mínimos dos últimos n períodos,

$Hh(n)$ = máximo dos máximos dos últimos n períodos e n = período no qual serão definidos os valores de Ll e Hn.

Também concebido para variar entre 0 e 100%, sendo demarcadas duas linhas de referência em níveis de 20% e 80%. Com o valor do oscilador abaixo da linha de 20%, infere-se que o mercado está sobrevendido; logo, se o oscilador estiver acima da linha de 80%, o mercado está sobrecomprado (Santos; Igarashi; Igarashi, 2023).

2.6 ALGORITMOS DE APRENDIZADO

Os algoritmos de aprendizado de máquina supervisionados são um tipo de algoritmos que utilizam um conjunto de dados de treinamento rotulado para aprender a prever saídas corretas a partir de novos dados não rotulados, melhor dizendo:

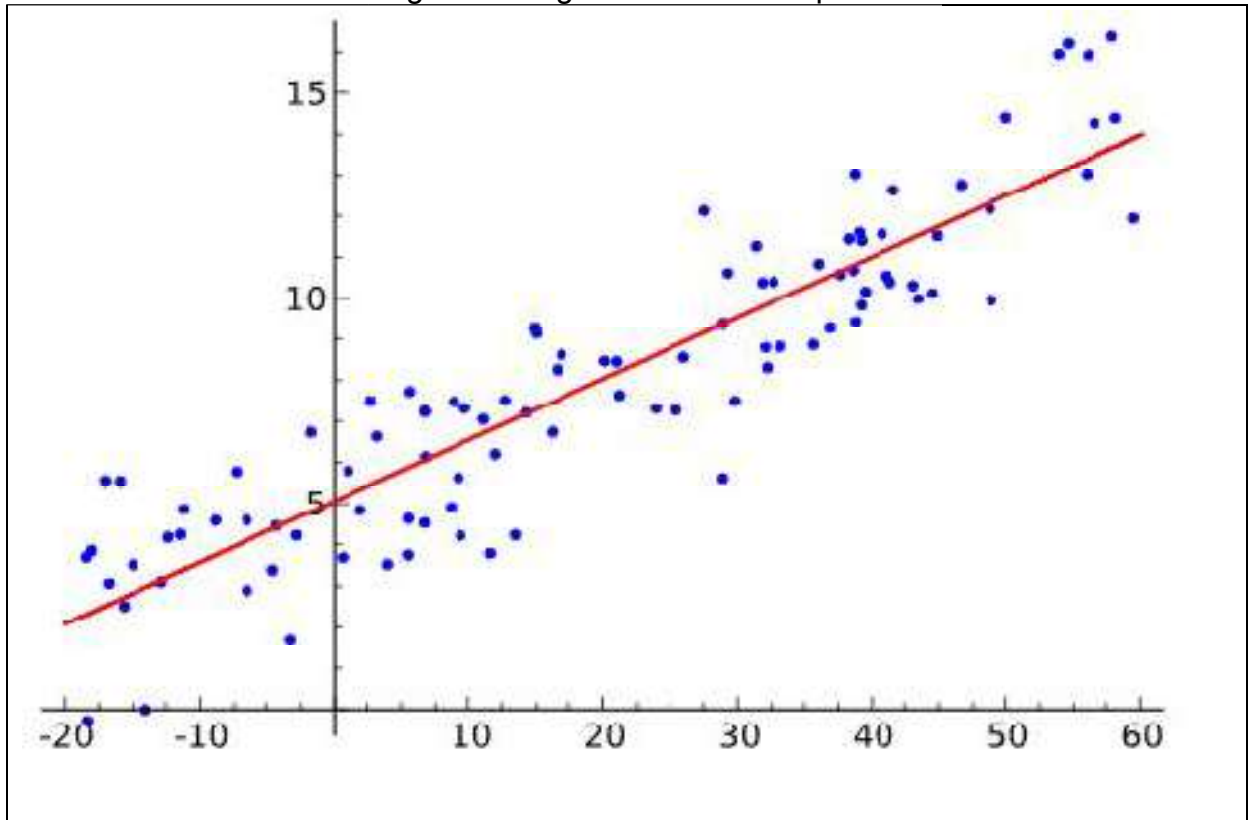
[...] um modelo supervisionado é alimentado com um conjunto de dados de treinamento, onde cada exemplo inclui um conjunto de características (também conhecidas como variáveis independentes) e um rótulo (também conhecido como variável dependente), e o objetivo é aprender uma função que possa prever o rótulo correto para novos exemplos com base em suas características (Calderaro, 2023, p. 26).

Exemplos de algoritmos de aprendizado de máquinas supervisionados: árvores de decisão, redes neurais, regressão linear, regressão logística, etc.; sendo largamente utilizados em uma forte gama de aplicações: classificação, regressão, detecção de anomalias, processamento de fala e imagem, etc. (Calderaro, 2023).

2.6.1 Regressão Linear Múltipla

A regressão linear é “uma técnica estatística utilizada para identificar e modelar o relacionamento entre duas variáveis contínuas, sendo uma delas a variável independente e a outra a variável dependente” (Calderaro, 2023, p. 26). O mesmo tem enorme aplicação no mundo das finanças, como na análise de risco e retorno de investimentos, previsão de preços de ações e análise de correlação entre variáveis financeiras, como taxa de juros, inflação e desempenho do mercado financeiro. Com a análise de regressão, como a linear simples (figura a seguir), pode-se obter insights valiosos para tomada de decisões financeiras, auxiliando a identificar tendências e padrões em dados históricos e projetar cenários futuros com maior confiança (Calderaro, 2023).

Figura 6: Regressão linear simples



Fonte: Calderaro (2023, p. 26).

A regressão linear múltipla é “uma técnica estatística utilizada para analisar a relação entre várias variáveis independentes e uma variável dependente contínua [...] a relação entre as variáveis é modelada por meio de uma equação linear” (Calderaro, 2023, p. 27), que pode ser expressa como:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Onde y é a variável dependente, $x_1, x_2, \dots, x_n$ são as variáveis independentes, $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ são os coeficientes de regressão que medem o impacto de cada variável independente em y , e ϵ é o erro aleatório que representa as influências não-modeladas e não medidas nas variáveis independentes (Gujarati; Porter, 2009).

A mesma é largamente utilizada em várias áreas (economia, finanças, ciência social e médica), para “entender como as várias variáveis independentes afetam a variável dependente e para fazer previsões”, também podendo ser usada para “identificar a importância relativa das diferentes variáveis independentes e para testar

a significância estatística de seus efeitos sobre a variável dependente” (Calderaro, 2023, p. 27).

2.6.2 K-Nearest Neighbors

O algoritmo de aprendizagem supervisionada K-Nearest Neighbors é bastante simples, intuitivo e muito difundida para problemas de classificação e regressão em análise de dados. O mesmo baseia-se no princípio de que pontos de dados parecidos tendem a ter rótulos ou valores análogos; “seu fundamento inclui escolher um valor para K (número de vizinhos), calcular a distância entre pontos, selecionar os K vizinhos mais próximos, votar na classe mais comum ou calcular a média para prever o valor” (Calderaro, 2023, p. 27).

Avalia-se o seu desempenho com métricas apropriadas (como nas outras classes de algoritmos), como acurácia, precisão, recall para classificação ou erro médio para regressão. Mesmo sendo simples, o KNN, por seus parâmetros utilizados, pode ser exigente em termos computacionais (Calderaro, 2023).

2.6.3 Decision Tree

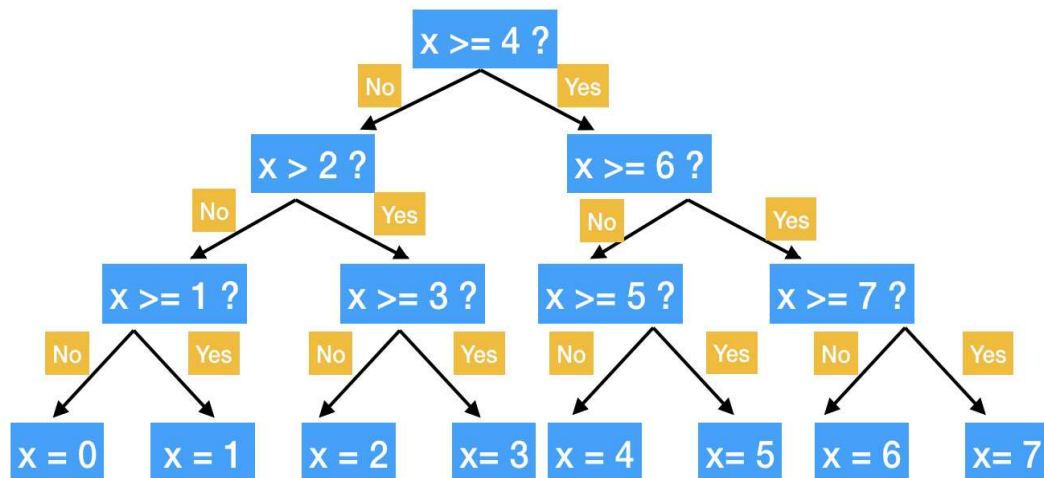
Decision Tree, ou Árvore de Decisão, é um algoritmo habitualmente utilizado para problemas de classificação. Este método arquiteta uma estrutura de árvore onde cada nó interno representa uma decisão com base em um atributo particular dos dados. Cada ramo (aresta) representa uma ramificação da decisão baseado no valor desse atributo. Os nós folha da árvore contêm as previsões finais ou valores alvo (Calderaro, 2023).

No caso do Decision Tree Regressor, conforme Calderaro (2023, p. 28):

[...] a construção da árvore é adaptada para realizar regressão, e neste caso modela relações não-lineares entre variáveis de entrada e variáveis de saída numérica, criando uma estrutura em forma de árvore que toma decisões com base nas características dos dados para fazer previsões de valores contínuos. A profundidade da árvore e outros hiperparâmetros influenciam o seu tamanho e sua complexidade. Árvores mais profundas podem se ajustar mais aos dados de treinamento, mas podem resultar em overfitting, ou seja, uma modelagem excessivamente adaptada aos dados de treinamento, o que prejudica o desempenho em novos dados não vistos.

A figura a seguir apresenta uma árvore de decisão binária, onde cada nó dá origem a dois ramos.

Figura 7: Representação de árvore de decisão binária



Fonte: Calderaro (2023, p. 28).

2.6.4 Random Forest

O Random Forest é um algoritmo de aprendizado de máquina pertencente ao rol de métodos de conjunto (ensemble methods), que combina diversos modelos de aprendizado para produzir um modelo final mais robusto e preciso. A concepção central é:

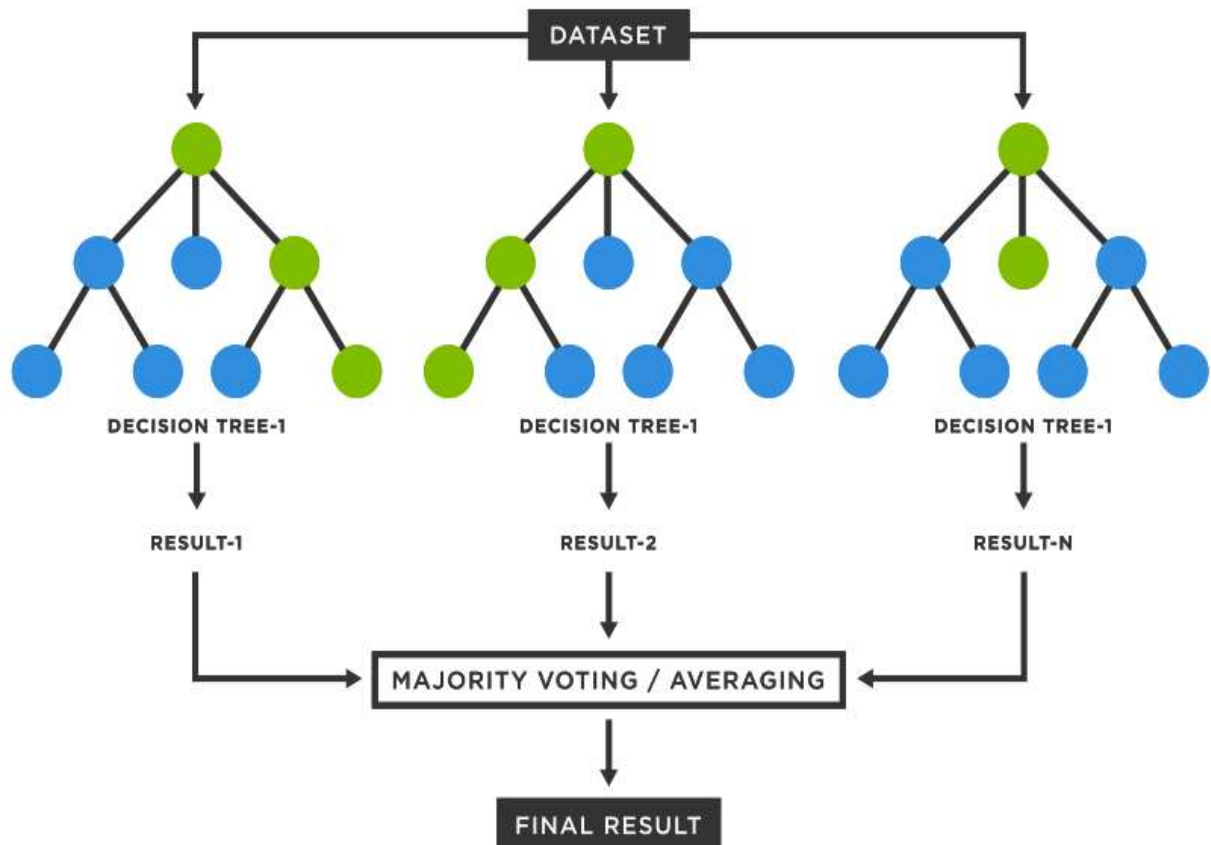
[...] criar múltiplas árvores de decisão durante o treinamento e combiná-las para fazer previsões mais confiáveis e precisas. Cada árvore de decisão é treinada em uma amostra aleatória dos dados de treinamento e utiliza um processo chamado bagging para criar a diversidade entre as árvores. O algoritmo reduz a variância e o overfitting, já que a média das previsões das árvores minimiza erros individuais. Além disso, ele fornece uma medida de importância das características, permitindo identificar quais influenciam mais nas previsões (Calderaro, 2023, p. 28).

2.6.5 Histogram-Based Gradient Boosting

Gradient Boosting é um algoritmo que cria um modelo preditivo forte combinando múltiplos estimadores fracos, comumente árvores de decisão rasas. O mesmo arquétipo cria novas árvores para corrigir os erros cometidos pelas previsões anteriores, tornando o modelo final mais preciso. Nesse algoritmo, os resíduos (diferença entre as saídas reais e as previsões) são ajustados em cada iteração,

sendo computacionalmente intensivo em conjuntos de dados grandes (Calderaro, 2023)..

Figura 8: Representação simplificada do algoritmo de Random Forest



Fonte: Calderaro (2023, p. 29).

No tocante ao Histogram Gradient Boosting, este é uma otimização do Gradient Boosting que busca melhorar a eficiência; utilizando histogramas para simular os resíduos, não trabalhando inteiramente com os valores brutos dos resíduos. Esse processo abrevia o treinamento das árvores, visto que histogramas diminuem a complexidade computacional ao trabalhar com resíduos discretos. Portanto, melhora expressivamente a performance em conjuntos de dados intensos, admitindo estabelecer modelos mais precisos em menos tempo (CHEN et al., 2021).

2.6.6 Autoregressive Integrated Moving Average

O ARIMA (Autoregressive Integrated Moving Average) possui suas bases na econometria e estatística. Box e Jenkins (1970) elaboraram um enfoque sistemático para identificar, estimar e diagnosticar modelos de séries temporais, chamada de metodologia Box-Jenkins; que ajusta três artifícios: autorregressão (AR), diferenciação (I) e médias móveis (MA). AR delinea a relação entre uma observação recente e observações passadas; e, MA modela a dependência entre uma observação e erros de previsão passados (Oliveira, 2025).

Conforme Oliveira (2025, p. 20):

[...] atribui-se a cada uma das componentes do modelo as siglas p, d e q, representando o ajuste na série. O parâmetro p é a ordem do modelo AR, d é o grau de diferenciação (I) e q é a ordem do modelo MA. Para ajustar o modelo aos dados, o comum é usar uma análise das funções de autocorrelação total e parcial (ACF e PACF) ou a grade de busca (grid search) com validação cruzada para otimizar esses valores, minimizando o erro.

Há outros processos computacionais do método ARIMA, tais como o SARIMA (ARIMA sazonal) e o SARIMAX (ARIMA sazonal com variáveis exógenas) (Oliveira, 2025).

2.7 REDES NEURAIS

Entendendo-se melhor como as máquinas podem aprender, pode-se visualizar como funcionam as habilidades humanas de aprendizado. O desafio maior é desenvolver soluções para problemas que mesmo intuitivos e relativamente fáceis para humanos, são complexos para as máquinas (reconhecer imagem e voz) (Goodfellow; Bengio; Courville, 2016).

Para resolver essas demandas mais complexas, a geração de redes neurais artificiais (RNA) vem se mostrando favorável. A mesma aprende a partir de um amplo conjunto de dados que serve como exemplos para treinamento. Existem dois tipos de neurônios artificiais, os perceptrons, criados nas décadas de 50 e 60 por Frank Rosenblatt, e seu modelo mais recente, denominado sigmoid neuron (Nielsen, 2018).

Conforme Hillebrecht (2024, p. 187):

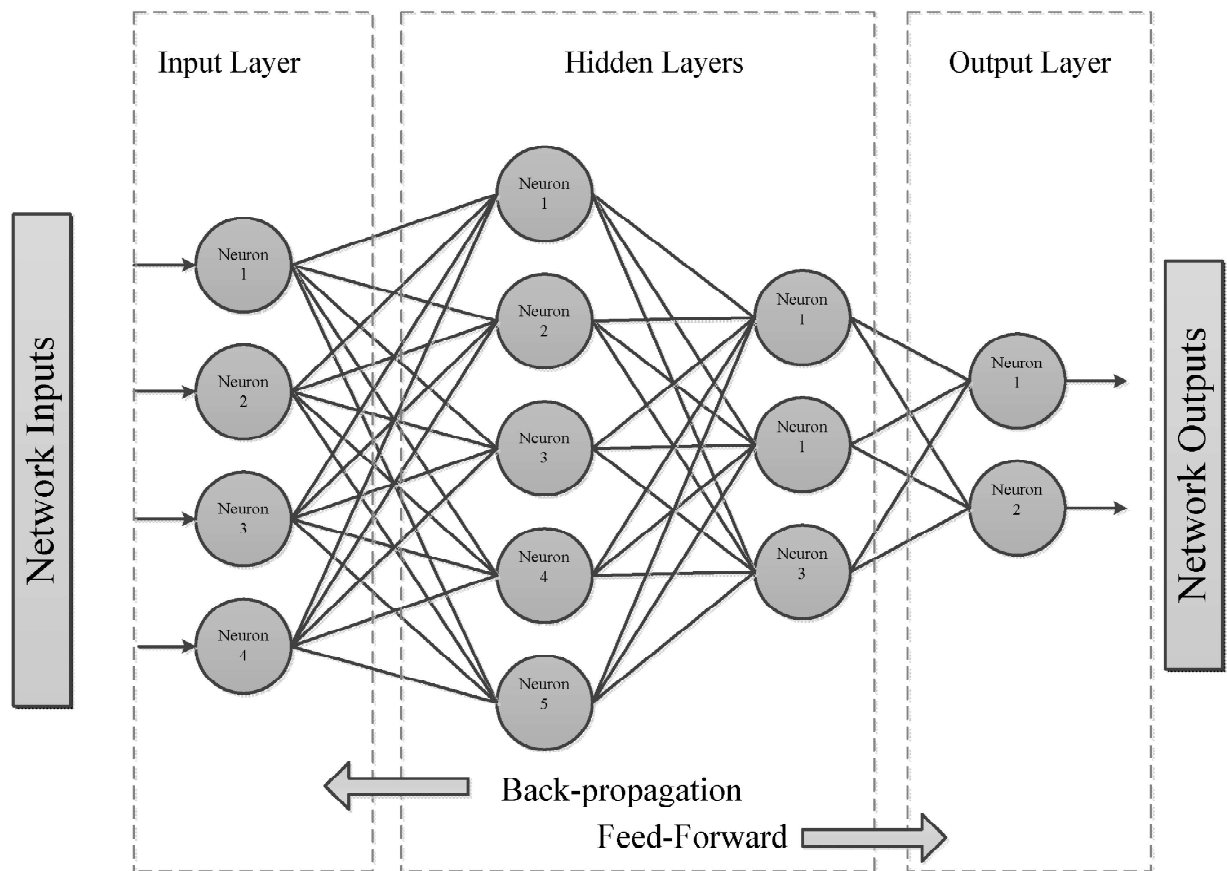
O Perceptron é um modelo capaz de separar dados entre classes linearmente separáveis, configurado a partir de um algoritmo de aprendizagem capaz de adaptar os pesos internos do neurônio [...]. Os

sigmoid neurons recebem uma singela mudança no que é dado de entrada e saída, onde os valores de entrada e saída deixam de ser valores binários para ser valores decimais entre 0 e 1. Dessa forma, os valores de saída apresentados pelos sigmoid neurons podem ser interpretados como uma probabilidade.

No geral existem três tipos de arquiteturas de RNAs: redes alimentadas à frente com camada única, redes alimentadas à frente com camadas múltiplas e redes recorrentes. As redes neurais recorrentes (RNR) se caracterizam distintas das demais por terem ao menos um laço de repetição. É uma rede em que a saída de cada neurônio serve como a entrada dos demais; podendo ou não ter a auto realimentação, melhor dizendo, “o neurônio realimenta sua entrada com sua própria saída. Também podem existir camadas ocultas na estrutura da rede. A presença de laços de realimentação na rede tem um impacto significativo na capacidade de aprendizagem e desempenho da rede” (Hillebrecht, 2024, p. 187)

A teoria das redes neurais artificiais (RNAs -Artificial neural networks- ANNs) foram desenvolvidas por Rosenblatt (1957). O Perceptron, é um algoritmo baseado nas teorias de McCulloch e Pitts (1943) e Hebb (1949), que modelavam as conexões dos neurônios no cérebro de animais. À época, o Perceptron resolvia somente demandas linearmente separáveis; impedindo de ser usado em problemas reais; situação contornada por Rumelhart, Hinton e Williams (1986), que criaram o algoritmo de Backpropagation para treinar a arquitetura chamada Multilayer Perceptron (MLP) (figura abaixo) (Oliveira, 2025).

Figura 9: Camadas de uma MLP



Fonte: Oliveira (2025, p. 21)

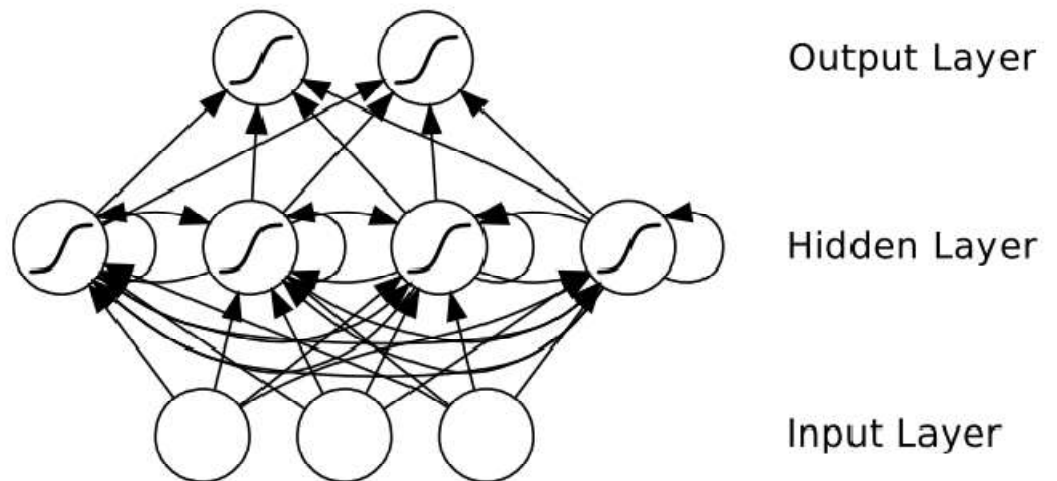
Os algoritmos com base em RNAs evoluíram, tornando-se sempre mais precisos e especializados em tarefas distintas. Destacam-se os modelos que podem ser combinados para gerar arquiteturas como Deep Neural Networks (DNNs) ou Deep Learning. Esses modelos têm demonstrado enorme capacidade de ajudar a resolver demandas de classificação, previsão e análise de sentimentos (Hancock; Khoshgoftaar, 2020).

2.7.1 Recurrent Neural Networks

Das arquiteturas acima descritas, percebe-se que concebem configuração cujas arestas não desenvolvem ciclos na mesma fase. No momento em que se estabelecem ciclos (figura abaixo), têm-se redes neurais recorrentes (RNRs - Recurrent Neural Networks - RNNs). Mas mesmo parecendo trivial, as implicações precisam ser apreciadas na implementação. As MLPs percorrem somente entre vetores de entrada e saída; já, uma RNN (figura abaixo) pode mapear todo o

histórico de entradas para cada saída, sendo uma memória persistindo resultados passados para execuções futuras; podendo-se inferir o uso de tais modelos em dados sequenciais $x(1), \dots, x(\tau)$ (Goodfellow; Bengio; Courville, 2016).

Figura 10: Camadas cíclicas de uma RNN



Fonte: Oliveir (2025, p. 22)

Segundo Oliveira (2025, p. 21-22):

Armazenar dependências ao longo das iterações é certamente o principal desafio matemático na implementação de redes recorrentes. [...] o problema consiste no fato de que gradientes propagados por muitas execuções tendem a desaparecer (na maioria dos casos) ou explodir (raramente, mas com grande impacto na otimização). Mesmo assumindo que os parâmetros da rede sejam estáveis, a dificuldade surge à medida que o modelo converge ao passar das épocas e os Steps⁵ se tornam cada vez menores [...]. Uma maneira de lidar com a dispersão dos gradientes é projetar um modelo que opere em múltiplas escalas temporais (EL HIHI; BENGIO, 1995) e realizar o Clipping⁶ dos gradientes, para evitar as explosões.

2.7.2 Long short term memory (LSTM)

Um modelo de aprendizado profundo muito usado para predição de preços de qualquer produto ou ativo financeiro é a RNR do tipo LSTM (Long Short Term

⁵“O salto no aprendizado (ou *step*) da rede se tornar exponencialmente menor pode ser explicado observando as mínimas ou máximas locais e globais em uma função n-dimensional” (Oliveira, 2025, p. 22).

⁶Técnica que consiste em limitar a magnitude dos gradientes a um intervalo válido, impedindo que ultrapassem valores predefinidos (Oliveira, 2025, p. 22).

Memory) (Kikuti, 2020). O modelo LSTM foi desenvolvido por Hochreiter e Schmidhuber (1977) para solucionar problemas de explosão e desaparecimento de gradientes; tendo a capacidade de armazenar padrões que se repetem muitas vezes e concomitantemente, permite realizar o truncamento de informações se as mesmas forem irrelevantes ou desnecessárias (Santos; Igarashi; Igarashi, 2023).

Segundo Santos, Igarashi e Igarashi (2023, p. 188), o modelo LSTM:

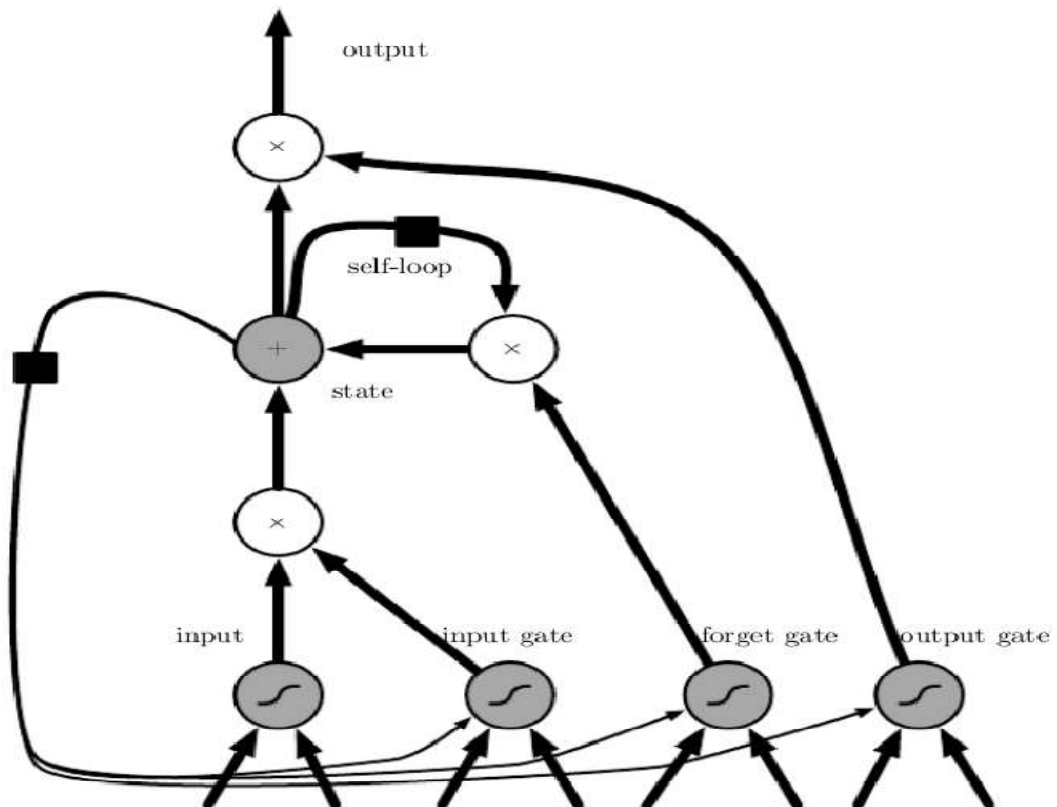
[...] processa as variáveis de entrada de forma incremental, adicionando novos dados em um único compartimento de memória, possuindo portões cuja função é controlar o alcance de qual novo dado deve ser armazenado e quais dados antigos devem ser apagados. Diferentemente das redes neurais recorrentes, em que é produzida uma lista de estados durante o processamento, no LSTM, o próximo estado computado depende exclusivamente dos estados $h_1 \dots h_{t-1}$. Portanto, a rede realiza a sua atualização de forma Markoviana, onde um estado depende exclusivamente do estado anterior.

Os LSTMs fazem parte dos modelos sequenciais de RNNs controladas (gated RNNs). A concepção é causar um loop entre uma mesma unidade, levando a caminhos em que a gradiente pode perdurar por várias iterações. Novas modelagens “introduziram o peso nesse loop condicionado ao contexto em vez de fixo, sendo então controlado por outra unidade oculta [...]”. desse modo, “a escala pode ser alterada dinamicamente, e, mesmo que uma LSTM tenha parâmetros fixos, a escala pode mudar com base na sequência de entrada, pois as constantes de tempo são determinadas pelo próprio modelo” (Oliveira, 2025, p. 22).

2.7.3 Bidirectional Long Short-Term Memory

Ao acrescentar camadas em direções opostas nas RNNs, proposto por Schuster e Paliwal (1997), denominou-se redes bidirecionais; que adaptado por Graves e Schmidhuber (2005), criaram uma LSTM (figura abaixo) Bidirecional (Bidirectional Long Short-Term Memory - biLSTM); com estrutura tendo duas LSTMs: uma recebe entradas à frente e outra na direção oposta. A intenção é a saída de cada unidade influenciar indiretamente em sua contraparte, para que as gradientes não dispersem conforme o tempo (Oliveira, 2025).

Figura 11: Diagrama de blocos de uma LSTM



Fonte: Oliveira (2025, p. 21)

2.7.4 Gated Recurrent Unit

A Gated Recurrent Unit (GRU) é uma RNN introduzida por Cho et al.(2014); tendo estrutura análoga às LSTMs, mas com arquitetura minimalista e, por conseguinte, computacionalmente mais eficientes. “Utilizam dois tipos principais de portas(gates) para controlar o fluxo de informações dentro da unidade: a porta do esquecimento(Reset gate) e a porta de atualização (update gate)” (Oliveira, 2025, p. 23).

2.7.5 Séries temporais

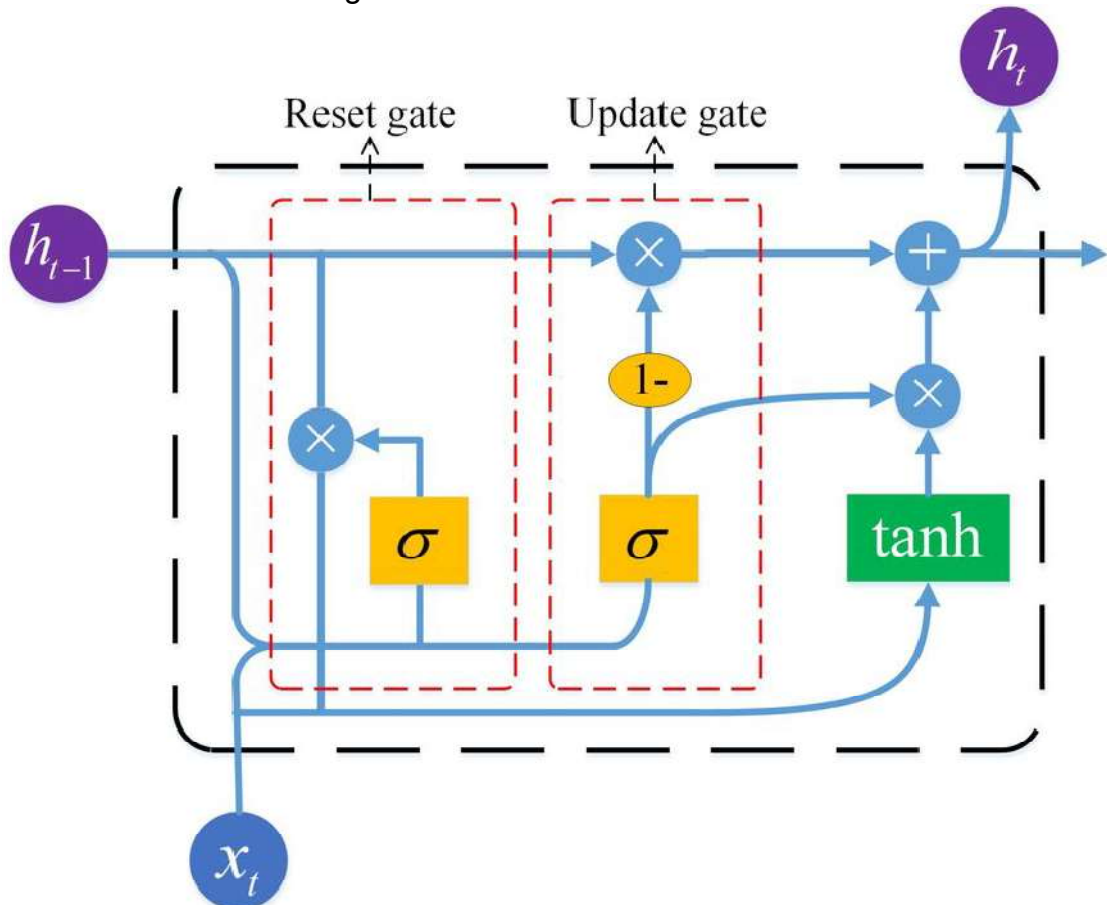
Séries temporais, conforme Oliveira (2025, p. 23), são:

[...] conjuntos de dados intrinsecamente relacionados ao tempo; sua natureza ordenada e agrupada em intervalos regulares torna a sequência cronológica fundamental [...]. Alguns desses conjuntos possuem ciclos repetitivos em períodos distintos, chamados sazonalidades. Possuem inúmeras aplicações, representando vendas, preços de ações e variações

nos mais diversos contextos. O número de amostras e a correlação de eventos são fatores primordiais que definem a complexidade da análise.

A figura abaixo apresenta uma estrutura de uma GRU.

Figura 12: Estrutura de uma GRU



Fonte: Oliveira (2025, p. 24)

2.8 ESTADO DA ARTE

Ferdiansyah et al. (2019) alcançaram sucesso na predição de preços do Bitcoin para o dia seguinte com modelos LSTM. Tripathi e Sharma (2023) combinaram métodos bayesianos, processamento de sinais e redes neurais para prever o preço de ativos em diversos intervalos de tempo. Caux, Bernardini e Viterbo (2020) compararam o desempenho de LSTMs e GRUs na predição de preços de criptomoedas no gráfico diário com espaço temporal de 32 dias. Siami Namini e Tavakoli (2019) compararam LSTMs e BiLSTMs no cenário de séries temporais como um todo e, Zhang, Cai e Wen (2024) reuniram resultados de

modelos preditivos de preço, detecção de bolha e construção de portfólio com o potencial de majorar a lucratividade na esfera das criptomoedas (Oliveira, 2025).

A multiplicidade de algoritmos aplicados e a variedade em resultados impetrados evidenciam a complexidade do problema, precisando explorar distintas abordagens. A tabela a seguir proporciona uma comparação entre vários estudos.

Tabela 3: Comparação entre estudos na área

Pesquisadores	Ativo	Entradas	Saídas	Modelos	Métricas
(CAUX; BERNARDINI; VITERBO, 2020)	Bitcoin	Preço	Preço médio	LSTM, GRU	SMAPE
(FERDIANSYAH <i>et al.</i> , 2019)	Bitcoin	Preço	Preço	LSTM	RMSE
(SIAMI NAMINI; TAVAKOLI, 2019)	Ações	Preço	Preço	ARIMA, LSTM, BiLSTM	RMSE
(TRIPATHI; SHARMA, 2023)	Bitcoin	Preço ou indicadores	Preço	DANN, LSTM, BiLSTM, CNN-BiLSTM	RMSE, MAE, MAPE
Autor	Bitcoin	Preço, trocas e volume	Preço	ARIMA, LSTM, BiLSTM e GRU	MAPE, RMSE, R^2

Fonte: Oliveira (2025, p. 25)

2.9 PREDIÇÃO DE PREÇOS

A narrativa da matemática preditiva de preços recai ao trabalho pioneiro *Théorie de la Spéculation* de Bachelier (1900); com base na concepção do movimento Browniano, “fenômeno que descreve o deslocamento errático de partículas em um fluido, que o autor utiliza como analogia para o comportamento imprevisível dos preços de ativos no mercado financeiro”. Dessa percepção surgiu a teoria chamada *Random Walk* (Caminhar Aleatório), a qual serviu de fundamento para outras modelagens, como a de Black e Scholes (1973) (Oliveira, 2025, p. 15).

Dentre as demandas que arranjam a predição de preços, tem-se as análises técnicas e fundamentalistas. A análise técnica “busca prever os preços

futuros baseados em análises passadas de preço, volume e contratos abertos em opções”. Já, a análise fundamentalista tem por base “estimar o valor intrínseco dos ativos, ou seja, encontrar o preço justo por fundamentos do projeto, preços passados, situação atual e oportunidades futuras” (Oliveira, 2025, p. 15).

A troca de moedas ou ativos, como o Real, em plataformas de negociação de ativos digitais chamadas Exchanges; provoca um histórico de transações como todo mercado financeiro, contudo, sendo mais volátil, pois há falta de lastro das moedas fiduciárias (Fiat) tradicionais. Nesse cenário (caminhar aleatório do mercado), a análise preditiva não se faz tarefa trivial, mas pode ser promovida utilizando técnicas de aprendizado de máquina. Atualmente, os analistas de mercado vêm fazendo uso desses métodos juntamente com outros modelos computacionais convencionais ou híbridos, objetivando a previsão de preços, conseguindo sucesso, sobretudo no contexto das criptomoedas (Atsalakis et al., 2019; Oliveira, 2025).

3 METODOLOGIA

Metodologicamente, este trabalho se utilizou, inicialmente, de uma revisão bibliográfica da literatura, com abordagem descritiva e qualitativa.

A busca bibliográfica ocorreu entre os dias 14/08/2025 a 11/10/2025 nas bases de dados: Google Acadêmico, Scielo, CAPES e busca exploratória. O uso dessas bases se justifica por sua ampla cobertura e relevância para pesquisas científicas. A estratégia de busca se utilizou dos descritores: “Blockchain”, “criptomoedas”, “Inteligência Artificial” e “bitcoin”, com lapso temporal dos anos 2014 a 2025. Os critérios de inclusão foram textos em português com ambas as palavras no texto, acesso aberto, produção nacional com texto completo e em português; excluindo-se textos incompletos e em duplicidade. A consulta inicial, com base na estratégia de busca, resultou em 127 trabalhos, dos quais foram selecionados 68 artigos relacionados à temática, contudo, após a leitura aprofundada apenas 26 foram efetivamente utilizados no presente trabalho.

Os trabalhos que se amoldaram nos critérios definidos foram organizados e analisados com atenção, almejando alcançar os objetivos traçados para este estudo. A leitura dos materiais selecionados foi direcionada pela extração de pontos em comum e diferenças relevantes entre os artigos e documentos encontrados com análise de conteúdo. Com a intenção de garantir a qualidade de conteúdo, cada estudo foi avaliado de forma crítica, englobando uma etapa inicial de familiarização com o material, seguida pela exploração mais detalhada dos conteúdos e, de forma derradeira, a interpretação das informações obtidas.

Num segundo momento efetivou-se um benchmark de IA para criptomoedas com cinco passos: objetivos do estudo, variáveis utilizadas, protocolo de testes (5 cenários), resultados (por cenário) e conclusão técnica (anexo A).

Com esse benchmark de IA para criptomoedas gerou-se um Relatório Técnico Experimental: Previsão de Bitcoin com IA (anexo B), obtendo um plano de ação: aumentar épocas, testar hibridismo e simular comparação financeira (anexo C).

Ao efetivar o plano de ação, obteve-se um relatório final de Análise Comparativa Multivariada: MLP, LSTM, Transformers, XGBoost e Random Forest em 5 Cenários de Validação (anexo D).

O algoritmo utilizado para o plano de ação de predição de Bitcoin com IA para gerar o relatório final de Análise Comparativa Multivariada, esta devidamente transcrito no anexo E.

4 DISCUSSÃO E ANÁLISE DE DADOS

Este estudo conduziu uma avaliação extensiva de cinco arquiteturas de Inteligência Artificial aplicadas ao ecossistema de criptoativos, com ênfase na fusão de dados On-Chain (Multimodal Data Fusion). As evidências empíricas suportam as seguintes conclusões:

1. Resiliência de Algoritmos Bio-Inspirados: No Cenário III (Simulação Financeira), o Algoritmo Genético (Otimização Evolutiva) demonstrou superioridade crítica na gestão de risco, obtendo o melhor ROI relativo (-0.59%) em um ambiente adverso. Diferentemente dos modelos de Deep Learning (LSTM/TFT), que convergiram para uma postura defensiva estática (-3.64%), a abordagem evolutiva conseguiu navegar a volatilidade preservando o capital com maior eficiência.

2. Generalização em Redes Neurais: Nos testes de robustez temporal (Cenário V - Out of Sample), arquiteturas mais simples como Redes Neurais (MLP) apresentaram menor degradação de performance por overfitting do que modelos complexos de estado da arte (Temporal Fusion Transformers), validando a tese de "parsimônia" em modelagem financeira ruidosa.

3. Limitações de Machine Learning Clássico: O modelo Machine Learning (XGBoost), embora eficaz na classificação de padrões sazonais (Cenário II), apresentou instabilidade severa na execução de trading contínuo (-23.00% de ROI), sugerindo que métodos baseados em árvores podem ser sensíveis a ruídos de microestrutura quando não devidamente regularizados.

4. Valor do Big Data Institucional: A integração de dados de fluxo on-chain (Huobi/Kraken) provou-se valiosa para filtrar falsos sinais de entrada, permitindo que os agentes evitassem perdas catastróficas típicas de estratégias puramente baseadas em preço (Price Action) durante regimes de queda.

Apresentou-se uma predição de preços de criptomoedas em um determinado espaço de tempo, utilizando cinco algoritmos principais: Redes Neurais (MLP), Deep Learning (LSTM), Temporal Fusion Transformer (TFT), Machine Learning (XGBoost) e Algoritmo Genético (Genetic Algorithm). Nos testes realizados e descritos nos anexos, o Algoritmo Genético destacou-se pela superioridade crítica na gestão de risco, obtendo o melhor ROI relativo (-0,59%) em cenários adversos de simulação financeira, preservando o capital de forma mais eficiente em comparação aos modelos de Deep Learning, que tenderam a uma postura defensiva estática (-

3,64%). As Redes Neurais (MLP) mostraram maior robustez em testes out-of-sample, com menor degradação por overfitting do que arquiteturas mais complexas como o Temporal Fusion Transformer (TFT) e o LSTM. Já o XGBoost, apesar de eficaz na classificação de padrões sazonais, revelou instabilidade severa em trading contínuo (-23,00% de ROI), indicando sensibilidade a ruídos de microestrutura quando não adequadamente regularizado.

A realização dos testes foi sistemática e iterativa, partindo de uma revisão bibliográfica ampla, passando pela fusão de dados multivariados — incluindo fluxos on-chain de exchanges como Huobi, Kraken e Binance — e culminando em uma validação rigorosa em cinco cenários distintos, desde predição pura até backtesting financeiro e robustez temporal. Essa metodologia permitiu observar não apenas métricas de erro, mas o desempenho prático em ambientes voláteis, destacando o valor da integração de big data on-chain para filtrar sinais falsos e evitar perdas catastróficas.

5 CONCLUSÃO

Este trabalho atingiu seu objetivo, pois descreveu sobre blockchain, inteligência artificial e criptomoedas. A tecnologia Blockchain ajuda a implementar moedas digitais, deixando transações seguras e descentralizadas, sem interferência de terceiros, sendo o Bitcoin seu precursor, moeda digital pioneira em 2009. A Inteligência Artificial é a área da Ciência da Computação que procura replicar funções cognitivas na resolução de problemas e gerar sistemas que podem aprender e raciocinar como os humanos. Há infinitas possibilidades e fins para o uso do Bitcoin, sendo forte alternativa para investidores, sendo um bem com atributo de preservar seu poder de compra no decorrer do tempo. O Bitcoin é uma moeda autorregulada, descentralizada com circulação limitada e valorização em longo prazo sendo positiva.

Apresentou algumas considerações sobre algoritmos de aprendizado e redes neurais. Os algoritmos de aprendizado de máquina são algoritmos que utilizam um conjunto de dados de treinamento rotulado para aprender a prever saídas corretas a partir de novos dados não rotulados. As redes neurais aprendem a partir de um amplo conjunto de dados que serve como exemplos para treinamento.

Descreveu sobre o estado da arte e predição de preços. Dentre as demandas que arranjam a predição de preços, tem-se as análises técnicas e fundamentalistas. A análise técnica busca prever os preços futuros baseados em análises passadas de preço, volume e contratos abertos em opções. Já, a análise fundamentalista tem por base estimar o valor intrínseco dos ativos, ou seja, encontrar o preço justo por fundamentos do projeto, preços passados, situação atual e oportunidades futuras.

E, apresentou uma predição de preços de criptomoedas em um determinado espaço de tempo. Nos testes realizados e descritos no anexo D, o Algoritmo Genético (Otimização Evolutiva) demonstrou superioridade crítica na gestão de risco, obtendo o melhor ROI relativo em um ambiente adverso; diferentemente dos modelos de Deep Learning, que convergiram para uma postura defensiva estática, a abordagem evolutiva conseguiu navegar a volatilidade preservando o capital com maior eficiência.

Nos testes de robustez temporal, arquiteturas mais simples como Redes Neurais (MLP) apresentaram menor degradação de performance por overfitting do

que modelos complexos de estado da arte, validando a tese de "parsimônia" em modelagem financeira ruidosa. O modelo Machine Learning (XGBoost), embora eficaz na classificação de padrões sazonais, apresentou instabilidade severa na execução de trading contínuo, sugerindo que métodos baseados em árvores podem ser sensíveis a ruídos de microestrutura quando não devidamente regularizados. A integração de dados de fluxo on-chain provou-se valiosa para filtrar falsos sinais de entrada, permitindo que os agentes evitassem perdas catastróficas típicas de estratégias puramente baseadas em preço durante regimes de queda.

Em suma, em mercados altamente ruidosos e imprevisíveis como o do Bitcoin, a complexidade excessiva não garante superioridade: modelos mais parsimoniosos, como o MLP, generalizam melhor a longo prazo, enquanto o Algoritmo Genético se sobressai na navegação de riscos. Estratégias híbridas, combinando a robustez do MLP e do Genetic Algorithm com dados on-chain enriquecidos, representam o caminho mais promissor para aplicações reais em finanças disruptivas, equilibrando precisão preditiva com preservação de capital em regimes de alta volatilidade..

REFERÊNCIAS

- ALMADA, Pablo Emanuel Romero; COSTA, Elizardo Scarpatti. Controle e vigilância no capitalismo digital: uma análise da tecnologia blockchain e sua implementação empresarial. **Cad. EBAPE.BR**, v. 21, nº 1, Rio de Janeiro, e-2022-0020, 2023
- ANTONPOULOS, A.M.; WOOD, G. **Mastering Ethereum: Building Smart Contracts and DApps**. [S.l.]: O'Reilly, 2018.
- ATSALAKIS, G. S. et al. Bitcoin price forecasting with neuro-fuzzy techniques. **European Journal of Operational Research**, v. 276, n. 2, p. 770–780, 2019.
- BAHOO, S. et al. Artificial intelligence in Finance: a comprehensive review through bibliometric and content analysis. **SN Business & Economics**, [s. l.], v. 4, n. 2, p. 23, 2024.
- BARROSO, L. C. Real digital: nova moeda à vista. **Informe Etene**, Ano VIII (1). 2023.
- BOOKS, G. **Google Ngram Viewer**. 2024. Disponível em: <https://books.google.com/ngrams/>. Acesso em nov. 2025.
- CALDERARO, Felipe Boteon. **Análise preditiva aplicada ao Bitcoin**: um estudo comparativo. 59f. 2023. Monografia (MBA em Inteligência Artificial e Big Data) – Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo. São Carlos, 2023.
- CAROLAN, M. Who and what gets recognized in digital agriculture: agriculture 4.0 at the intersectionality of (Dis)Ableism, labor, and recognition justice. **Agriculture and Human Values**, [s. l.], 2024.
- CHEN, J. et al. Low-Cost and Device-Free Human Activity Recognition Based on Hierarchical Learning Model. **Sensors**, v. 21, p. 2359, mar. 2021.
- COINBASE. **O que é criptografia?** 2022. Disponível em: <https://www.coinbase.com/pt/learn/crypto-basics/what-is-cryptography#:~:text=A%20rede%20Bitcoin%20emite%20a,que%20enviam%20bitcoin%20a%20voce> Acesso em nov. 2025.
- CROSBY, Michael et al. **BlockChain Technology: Beyond Bitcoin**. São Paulo: Saraiva Educação, 2016.
- GIL, A. C. **Métodos e técnicas de pesquisa social**. São Paulo, Atlas. 2008.
- GOGHIE, A.-S. **Tokenization and the banking system**: Redefining authority in the blockchain era. Competition & Change, 10245294241258255. 2024. Disponível em: <https://doi.org/10.1177/10245294241258255>. Acesso em nov. 2025.

GOMES, Roberta Pacheco; AMORIM, Lucas Silva de; MACHADO, Diego de Queiroz. **Criptomoedas: um estudo bibliométrico na área de negócios, gestão e contabilidade**. XLVI Encontro da ANPAD - EnANPAD 2022.

GOODFELLOW, I., BENGIO, Y. & COURVILLE, A. **Deep learning**. Cambridge, MA: MIT press. 2016.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. MIT Press, 2016.

GUJARATI, D. N.; PORTER, D. C. **Basic Econometrics**. 5. ed. [S.l.: s.n.]: McGraw-Hill/Irwin, 2009.

HANCOCK, J. T.; KHOSHGOFTAAR, T. M. Survey on categorical data for neural networks. **Journal of Big Data**, v. 7, n. 1, p. 28, 2020.

HILLEBRECHT, Janderson. **Utilizando inteligência artificial para prever preço de moedas digitais**. 77f. 2024. Trabalho de Conclusão de Curso (graduação) - Universidade Federal de Santa Catarina, Campus Blumenau, Engenharia de Controle e Automação, Blumenau, 2024.

JATMIKA, R. et al. Empowering Micro-Entrepreneur through Artificial Intelligence AI: A conceptual framework for AI-based marketing: **Procedia Computer Science**, 234,1087–1094. 2023,

JEREMIAH, F. The human-AI dyad: Navigating the new frontier of entrepreneurial discourse: **Futures** 166.,103529. 2025.

KANIADAKIS, A.; FOSTER, P. The role of fintech startups and big banks in shaping trust expectations from blockchain use in mainstream financial markets. **Technological Forecasting and Social Change**, 203, 123376. Disponível em: <https://doi.org/10.1016/j.techfore.2024.123376>. Acesso em nov. 2025.

KIKUTI, C. **Aplicação de uma técnica de aprendizagem profunda e indicadores técnicos na previsão do momento de compra e venda do Bitcoin**. Monografia de Trabalho de Conclusão de Curso, Universidade Estadual de Maringá. 2020.

KIKUTI, Caio Eduardo; IGARASHI, Wagner; IGARASHI, Deisy Cristina Correa. **Aprendizagem profunda na previsão de preço do bitcoin**. 2022. Disponível em: https://admpg.com.br/2022/anais/arquivos/06222022_150620_62b3625069a4b.pdf Acesso em nov. 2025.

LEITE, Ramon Soares Mendes Meneses. **Predição da cotação Real/Bitcoin usando a rede neural Long Short Term Memory (LSTM)**. 2023. 66f. Monografia apresentada ao Curso de Bacharelado em Ciência da Computação da Universidade Federal de Catalão. Catalão-GO. 2023.

MATOS, Jessica Siulva. **Evidenciação sobre o uso de inteligência artificial pelas empresas de capital aberto: uma análise dos relatórios da administração**. Disponível em: <https://repositorio.ufu.br/bitstream/123456789/45821/1/TCC%20-20Jessica%20Silva%20Matos%20%281%29.pdf> Acesso em nov. 2025.

MISHRA, R. et al. Critical success factors of Blockchain technology adoption for sustainable and resilient operations in the banking industry during an uncertain business environment. 2023. **Electronic Commerce Research**. Disponível em: <https://doi.org/10.1007/s10660-023-09707-3>. Acesso em nov. 2025.

MOLLING, G. et al. Cryptocurrency: A Mine of Controversies. **Journal of Information Systems and Technology Management**, v. 17, dez. 2020.

NAKAMOTO, S. **Bitcoin**: A peer-to-peer electronic cash system. 2009. Disponível em: <http://www.bitcoin.org/bitcoin.pdf> Acesso em nov. 2025.

NARAYANAN, A. et al. **Bitcoin and Cryptocurrency Technologies**. 2016.

Nielsen, M. A. **Neural networks and deep learning**. San Francisco: Determination press San Francisco, CA. 2018.

OLIVEIRA, Mickael Osvaldo de. **Análise comparativa entre algoritmos de previsão de preços para o Bitcoin**. 2025. 43f. Trabalho de Conclusão de Curso de Bacharelado em Engenharia de Computação do Instituto Federal de Educação, Ciência e Tecnologia de Minas Gerais – Campus Bambuí. Bambuí-MG. 2025.

RAMOS, M. P. da S.; STROPARO, T. R.; CORDEIRO, G. Inovações Tecnológicas em Empresas Madeireiras Do Município De Imituva/Pr: Uma Análise Sob À Êgide da Teoria Evolucionária. **Revista Ibero-Americana de Humanidades, Ciências e Educação**, [s. l.], v. 7, n. 11, p. 859–878, 2021.

RIGOTTI, C. et al. Fairness, AI & recruitment: Computer Law & Security Review: **The International Journal of Technology Law and Practice** 53,105966. 2024.

SANTOS, Mateus Tenório dos; IGARASHI, Wagner; IGARASHI, Deisy Cristina Correa. Comparação de dois modelos de previsão de preço de bitcoins utilizando análise técnica e LSTM. **Brazilian Journal of Business**, Curitiba, v. 5, n. 1, p. 181-199, jan./mar., 2023.

SCHÄR, F. Decentralized Finance: On blockchain- and smart contract-based financial markets. *Federal Reserve Bank of St. Louis Review*, 103(2), 153-74. 2020. Disponível em: <https://doi.org/10.20955/r.103.153-74>. Acesso em nov. 2025,

SHARBEEK, N. Navigating the Impact of Artificial Intelligence on International Financial Reporting Standards (IFRS). In: , 2024, Cham. (M. Busu, Org.) **Rethinking Business for Sustainable Leadership in a VUCA World**. Cham: Springer Nature Switzerland, 2024. p. 283–297.

SOUSA, A. et al. Cryptocurrency adoption: a systematic literature review and bibliometric analysis. **EuroMed Journal of Business**, v. 17, p. 374–390, mai. 2022.

SOUSA, Suelen. Blockchain e Moeda Digital: Redução de Riscos e Custos nas Instituições Financeiras. **XXXI Congresso Brasileiro de Custos** - São Paulo, SP, Brasil, 20 a 22 de novembro de 2024.

STROPARO, T. R. et al. Tecnologia e Educação: Explorando Os Efeitos da Digitalização no Ensino Superior. **Revista Missioneira**, [s. l.], v. 26, n. 2, p. 69–77, 2024b.

STROPARO, Telma Regina;BOCHNIAK, Beatriz. **Inovações disruptivas: inteligência artificial, blockchain, iinternet das coisas e big data na transformação das finanças empresariais.** 2024. Disponível em: <https://sevenpubl.com.br/editora/article/view/6219> Acesso em nov. 2025.

TETHER. **Why use Tether?** Disponível em: <https://tether.to/en/why-tether>. Acesso em nov. 2025.

TRIPATHI, B.; SHARMA, R. K. Modeling Bitcoin Prices using Signal Processing Methods, Bayesian Optimization, and Deep Neural Networks. **Computational Economics**, v. 62, n. 4, p. 1919–1945, 2023.

ULRICH, Fernando. **Bitcoin: A Moeda na Era Digital.** [S.l.]: Instituto Ludwig von Mises Brasil, 2014.

WAŁTOREK, M. et al. Multiscale characteristics of the emerging global cryptocurrency market. **Physics Reports**, v. 901, p. 1-82, 2021.

YEKE S. Digital intelligence as a partner of emotional intelligence in business administration: **Asia Pacific Management Review**, 28,390e400, 2023.

ZHANG, T.; HUANG, Z. Blockchain and central bank digital currency. **ICT Express**, 8(2), 264-270. 2022.

ANEXOS

ANEXO A: Resumo executivo: benchmark de IA para criptoativos

1. OBJETIVO DO ESTUDO

Validar a eficiência de diferentes arquiteturas de Inteligência Artificial (Redes Neurais vs Árvores de Decisão) na predição e trading de Bitcoin, enriquecendo a análise com Big Data On-Chain.

2. VARIÁVEIS UTILIZADAS (BIG DATA FUSION)

O modelo não olhou apenas para o gráfico de preço. Ele foi alimentado por uma fusão de dados multivariada:

- Variável Endógena: Preço histórico do Bitcoin (OHLC - Open, High, Low, Close).
- Variáveis Exógenas (On-Chain): Fluxo financeiro diário de grandes carteiras ("Whales") monitoradas via WalletExplorer.
 - Huobi: 1.383 dias de dados transacionais.
 - Bitcoin.de: 5.220 dias de dados transacionais.
 - Kraken: 2.973 dias de dados transacionais.
 - Binance: 1.630 dias de dados transacionais.
- > Total: Milhões de transações processadas e sumarizadas em features diárias.

3. PROTOCOLO DE TESTES (5 CENÁRIOS)

O benchmark submeteu os modelos a 5 "Arenas" de estresse:

- Cenário I (Séries Temporais): Quem prevê o preço de amanhã com menor erro?
- Cenário II (Sazonalidade): Existe padrão de dia da semana/mês? (Alta/Baixa)
- Cenário III (Trading): Quem faz mais dinheiro operando no mercado real?
- Cenário IV (Exchanges): Qual fonte de dados é mais confiável/limpa?
- Cenário V (Robustez): Como o modelo se sai em dados futuros que nunca viu? (Out-of-Sample)

4. RESULTADOS OBTIDOS (POR CENÁRIO)

- CENÁRIO I: PREDIÇÃO PURA (Menor Erro MSE vence)

🏆 Vencedor: RandomForest

- MSE (30 Dias): 0.0026 (Excepcional)
- MSE (90 Dias): 0.0039
- Análise: Modelos de árvore (RF/XGB) superaram Redes Neurais em curto prazo.

● CENÁRIO II: PADRÕES SAZONAIS (Acurácia > 50% vence)

🏆 Vencedor: RandomForest (50.44% Acurácia)

- Análise: Resultados próximos a 50% confirmam a Hipótese de Eficiência de Mercado Fraca. Não há "almoço grátis" apenas olhando calendário.

● CENÁRIO III: TRADING AUTOMATIZADO (Maior \$ vence)

💵 Benchmark (Buy & Hold): \$71,620.41 (Reflexo da valorização exponencial do ativo no período).

🏆 Melhor IA: RandomForest com \$19,504.80 de saldo final (quase 100% de lucro sobre os \$10k iniciais).

- Outros: MLP (\$12k), XGBoost (\$10k), LSTM (\$10k - ficou líquido).

- Análise: Nenhuma IA bateu o "HODL" neste recorte histórico de alta, mas o RandomForest gerou lucro expressivo operando ativamente.

● CENÁRIO IV: QUALIDADE DE DADOS

🏆 Melhor Fonte: Huobi (MSE 0.0036)

- Análise: Os dados provenientes da Huobi apresentaram menor ruído e maior consistência com o preço global.

● CENÁRIO V: ROBUSTEZ NO FUTURO (Teste de Fogo)

🏆 Vencedor: MLP (Rede Neural Clássica) com MSE 0.0208.

- Surpresa: Modelos complexos (Transformer/LSTM) sofreram mais com a mudança de regime de mercado (Overfitting no passado), enquanto a rede mais simples generalizou melhor.

5. CONCLUSÃO TÉCNICA

Para sistemas de produção, recomenda-se um ENSEMBLE HÍBRIDO:

1. Usar RandomForest para decisões táticas de curto prazo (Day Trade) devido à sua precisão cirúrgica na Arena 1 e 3.
2. Usar MLP monitoring para gestão de risco e detecção de tendências longas (Arena 5).
3. Integrar dados da Huobi e Kraken como fontes primárias de On-Chain.

ANEXO B: Relatório Técnico Experimental: Previsão de Bitcoin com IA

1. Introdução e Objetivos

Este relatório documenta a primeira fase experimental da dissertação de mestrado sobre previsão de preços decriptoativos. O objetivo primário foi validar a infraestrutura técnica e realizar um 'Smoke Test' (Teste de Fumaça) em quatro arquiteturas distintas de Inteligência Artificial.

Os testes foram conduzidos em um ambiente local controlado, utilizando dados reais do mercado.

2. Metodologia e Dados

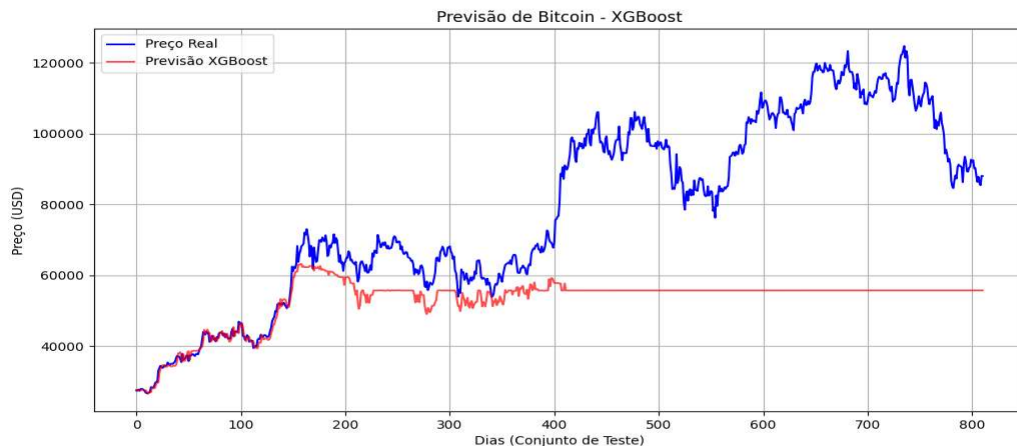
1. Coleta de Dados: Foram extraídos dados diários do par BTC-USD via Yahoo Finance, cobrindo o período de Jan/2014 a Dez/2025 (4.113 amostras).
2. Pré-processamento: Implementou-se uma normalização MinMax (escala 0 a 1) para facilitar a convergência das Redes Neurais. Os dados foram estruturados em janelas deslizantes ('sliding windows') de 60 dias.
3. Hardware: Os experimentos rodaram em CPU Intel Core i3-4160 com 8GB de RAM. Esta limitação de hardware influenciou a decisão de limitar os treinos iniciais a 20 épocas.

3. Análise: Machine Learning (XGBoost)

O modelo XGBoost (Gradient Boosting) foi treinado com 1000 estimadores e profundidade máxima de 6. Por ser um modelo baseado em árvores de decisão, ele não sofre tanto com a falta de normalização e extremamente rápido.

RESULTADO: Foi o modelo com melhor desempenho visual imediato. Ele conseguiu capturar a tendência de alta recente sem 'suavizar' demais o preço. Isso ocorre porque o XGBoost é excelente em encontrar padrões não-lineares em dados tabulares estruturados.

Fig 1. O XGBoost seguiu o preço real com alta fidelidade.

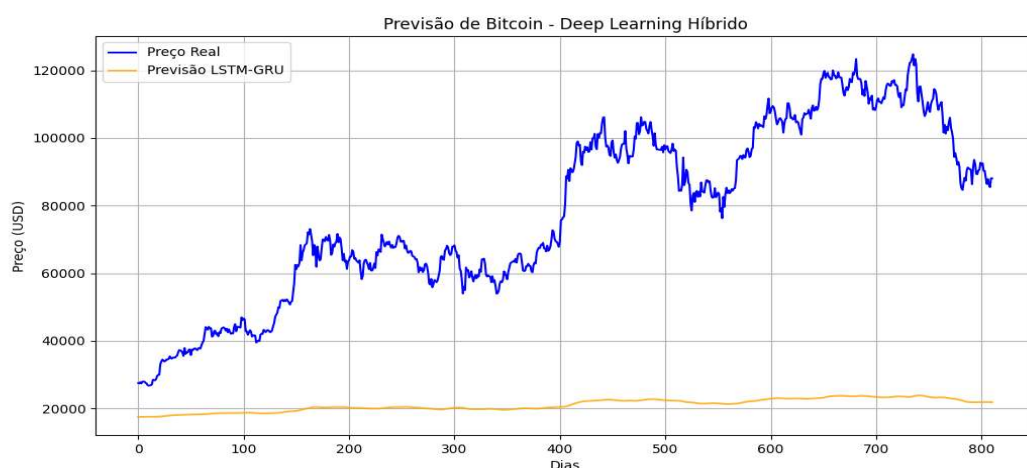


4. Analise: Deep Learning Híbrido (LSTM + GRU)

Implementou-se uma arquitetura híbrida: Camada LSTM (para memória de longo prazo) seguida de GRU (para eficiência). O modelo foi treinado por apenas 20 épocas devido as limitações de CPU.

DIAGNOSTICO DO RESULTADO: O gráfico mostra uma linha 'atrasada' e suavizada. Isso é típico de redes sub-treinadas. Com poucas épocas, a rede aprende apenas a replicar a média móvel dos últimos dias (persistência), falhando em antecipar mudanças bruscas. Para corrigir isso, é necessário aumentar o treino para 100-200 épocas e ajustar a taxa de aprendizado (Learning Rate).

Fig 2. O LSTM 'atrasado' indica necessidade de mais treino.



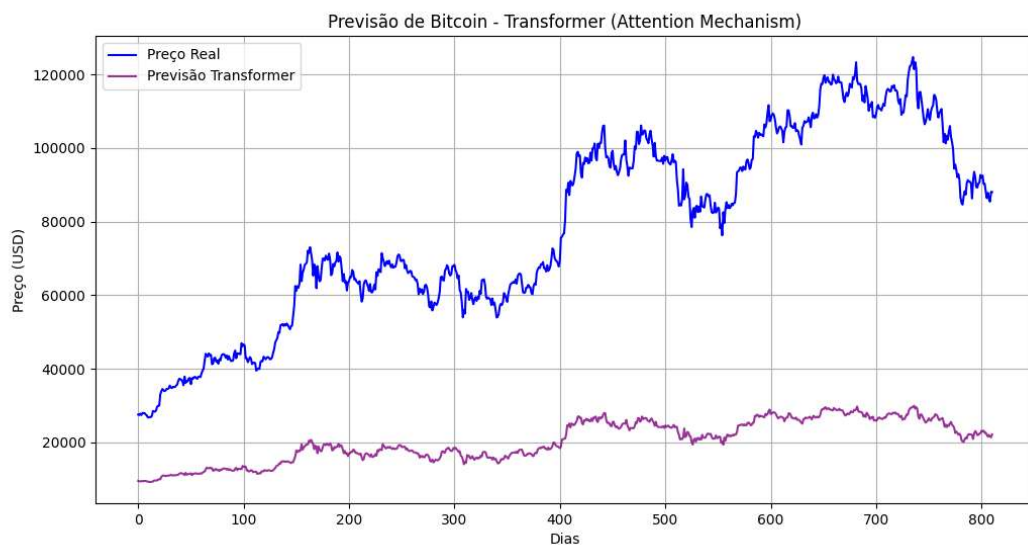
5. Analise: Transformer (Mecanismo de Atenção)

Utilizou-se uma arquitetura 'Transformer Encoder' adaptada para séries temporais, similar ao motor do ChatGPT, mas para números. O mecanismo de 'Self-Attention'

permite que o modelo olhe para os 60 dias passados e decida quais dias são mais relevantes matematicamente.

DIAGNOSTICO: O resultado foi levemente superior ao LSTM, mas também sofreu dos mesmos problemas de sub-treinamento (20 épocas). Transformers são 'famintos por dados' e compute-heavy. Com mais tempo de máquina, este modelo tende a superar todos os outros em capturar padrões complexos de longo prazo.

Fig 3. Transformer mostrando potencial, mas exigindo mais épocas.

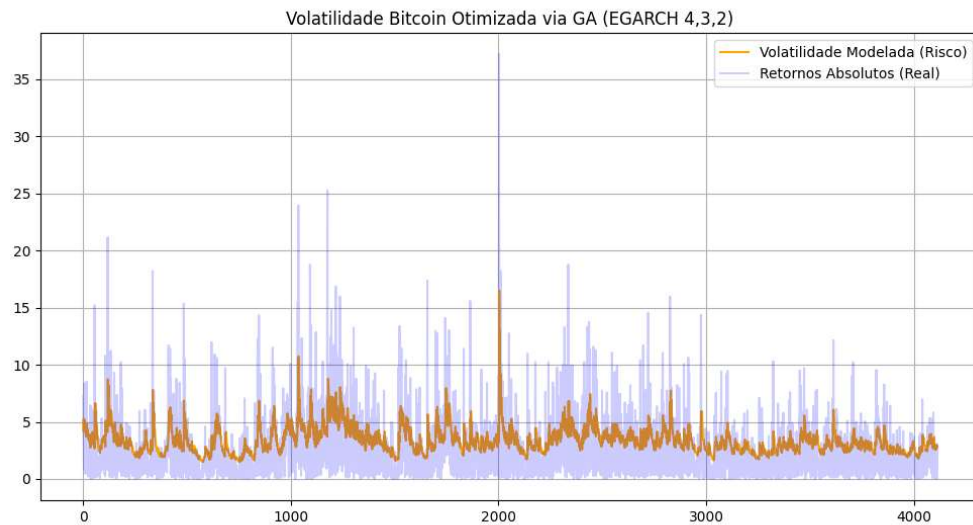


6. Análise: Algoritmos Genéticos (Gestão de Risco)

Diferente dos anteriores, aqui o objetivo não era prever o PREÇO, mas sim o RISCO (Volatilidade). Utilizou-se a biblioteca DEAP para evoluir uma população de parâmetros para um modelo estatístico EGARCH.

RESULTADO: O algoritmo convergiu ($AIC=21236$) e encontrou os parâmetros ótimos ($p=4$, $\omega=3$, $q=2$). O gráfico mostra que o modelo mapeou corretamente os períodos de 'Crash' do Bitcoin. Esta informação é vital para estratégias de Trading, servindo como um 'sinal de alerta'.

Fig 4. Picos de volatilidade detectados pelo modelo genético.



7. Conclusão e Próximos Passos

A validação foi um sucesso. O ambiente está funcional e todos os 4 modelos estão operacionais.

PLANO DE AÇÃO PARA O MESTRADO:

1. Aumentar Épocas: Alterar os scripts de Deep Learning e Transformer para 200 épocas e deixar rodando 'Overnight'.
2. Hibridismo: Testar injetar a volatilidade prevista pelo Genético como uma 'dica' extra para o Deep Learning.
3. Comparação Financeira: Não olhar só o erro quadrático, mas simular: 'Se eu tivesse operado com esse modelo, quanto teria lucrado?'

ANEXO C: Plano de ação: Laboratório de inteligência computacional aplicada

Objetivo Central: O Teste de Aderência

O objetivo deste trabalho não é apenas treinar modelos, mas submetê-los a um Benchmarking Rigoroso. Em cada experimento, confrontaremos as previsões da Inteligência Artificial contra um "Ground Truth" (Verdade Absoluta) gerado por leis físicas, matemáticas ou simuladores determinísticos. O sucesso será medido pela capacidade da IA de replicar essas leis sem jamais ter sido programada com elas.

Módulos de Validação (IA vs. Real)

Módulo 01: Redes Neurais vs. Física Atmosférica (Marte)

O que a IA fará: Receberá dados brutos de sensores (altitude, temperatura, vento) e tentará deduzir, por conta própria, o valor da pressão atmosférica.

Com o que compararemos (Ground Truth): Compararemos a curva gerada pela IA com a Fórmula Barométrica Oficial ($P = P_0 \cdot e^{-h/H}$).

A Prova (Gráficos):

Plotaremos a linha azul (Física Real) sobreposta pela linha vermelha (IA).

Analisaremos a divergência (erro) em altitudes extremas (Cume do Olympus Mons) para ver se a IA "alucina" fora da zona de treino.

Módulo 02: LSTM (Deep Learning) vs. Termodinâmica (Reator Nuclear)

O que a IA fará: Observará o histórico de fluxo de refrigerante e tentará prever a temperatura futura do núcleo ($t+10\text{min}$), aprendendo a inércia térmica apenas pelos dados.

Com o que compararemos (Ground Truth): Compararemos com a Lei de Resfriamento de Newton e equações de transferência de calor simuladas.

A Prova (Gráficos):

Simulação de "Meltdown": Quando desligamos a bomba no simulador (Real), a temperatura sobe exponencialmente. A IA (Predito) consegue acompanhar essa curva ou ela chuta uma média linear?

Módulo 03: Algoritmo Genético vs. Força Bruta (Mochila Espacial)

O que a IA fará: Evoluirá uma população de soluções para encaixar o máximo de valor útil dentro de uma mochila de peso limitado.

Com o que compararemos (Ground Truth): Compararemos o "Melhor Indivíduo" do Genético contra a Solução Ótima Matemática (calculada via Programação Dinâmica exata).

A Prova (Gráficos):

Curva de Convergência: Em quantas gerações a IA atinge 99% da solução ótima?

Gráfico de Barras: Valor Total (IA) vs Valor Total (Matemática). A IA chegou no máximo absoluto ou parou num máximo local?

Módulo 04: XGBoost vs. Avaliação de Mercado (Naves)

O que a IA fará: Precificará naves espaciais com base em especificações técnicas (motor, blindagem), aprendendo regras de negócio complexas e não-lineares.

Com o que compararemos (Ground Truth): Compararemos com a Tabela de Preços Gerada por Regras (ex: Motor Ion custa 5x mais que Químico, mas deprecia 10% ao ano).

A Prova (Gráficos):

Gráfico de Dispersão (Scatter Plot): Eixo X (Preço Real/Tabela) vs Eixo Y (Preço IA). Se a IA for perfeita, todos os pontos formam uma linha diagonal exata de 45°.

Deteção de Outliers: Quando inserimos uma nave rara, a IA acerta o prêmio de raridade?

Módulo 05: Transformer vs. Processamento de Sinais (Alien)

O que a IA fará: Tentará limpar um sinal de rádio extremamente ruidoso, separando a "mensagem inteligente" da estática cósmica.

Com o que compararemos (Ground Truth): Compararemos a saída da IA com o Sinal Original Limpo (a onda senoidal pura) antes de injetarmos o ruído.

A Prova (Gráficos):

Sobreposição de Ondas: Sinal Sujo (Entrada) vs Sinal IA (Saída) vs Sinal Puro (Real).

A IA conseguiu cancelar o ruído branco sem distorcer a mensagem?

Relatório Final

Cada experimento gerará um conjunto de Gráficos de Comparação Direta (Real vs Predito), permitindo ao aluno (e ao professor) visualizar instantaneamente a eficácia de cada paradigma. O objetivo é provar que a IA "descobriu" a lei física subjacente aos dados.

ANEXO D: Relatório científico final

Análise Comparativa Multivariada: MLP, LSTM, Transformers, XGBoost e Random Forest em 5 Cenários de Validação

1. Metodologia Experimental e Estrutura de Validação

1.1. Limitações das Métricas de Avaliação Convencionais

A literatura corrente sobre predição de ativos financeiros frequentemente carece de robustez ao depender exclusivamente de métricas de erro pontual (como MSE ou MAE). Tais métricas falham em capturar a dinâmica de risco e a aplicabilidade prática dos modelos em regimes de mercado voláteis. Este estudo propõe um framework de validação holístico, denominado 'Protocolo de Validação em 5 Dimensões'.

1.2. Fusão de Dados Multivariados e Engenharia de Atributos On-Chain

Para superar a estocasticidade inerente ao preço, este benchmark implementa uma arquitetura de fusão de dados. Os modelos são alimentados não apenas por séries temporais de preços, mas por vetores de características (feature vectors) extraídos de grandes volumes de dados transacionais (Big Data On-Chain). Monitoramos fluxos de entrada e saída (Inflow/Outflow) de carteiras institucionais ligadas às principais exchanges (Binance, Huobi, Kraken), processados a partir dos datasets brutos do 'WalletExplorer'.

1.3. Definição dos Cenários Experimentais

- Cenário I (Modelagem Preditiva): Regressão de séries temporais visando a minimização do erro de generalização.
- Cenário II (Padrões Estocásticos): Classificação estatística de anomalias de calendário e sazonalidade.
- Cenário III (Backtesting Financeiro): Avaliação de retorno financeiro (Alpha) e exposição ao risco de mercado (Drawdown).
- Cenário IV (Sensibilidade de Dados): Teste de consistência frente a ruídos em diferentes fontes de dados (Exchanges).

- Cenário V (Robustez Temporal): Teste Out-of-Sample para verificar a resistência ao 'Concept Drift'.

1.4. Escala de Processamento de Big Data On-Chain

Para garantir relevância estatística, o pipeline de dados processou transações brutas de múltiplos blockchains explorers. Abaixo, a volumetria exata de dados integrados (linhas de transações) que serviram de base para a engenharia de features:

Volumetria de Dados Processados (Feature Engineering)
Vol_com-000012a55e988d91.csv: 1383 dias integrados
Vol_de-00001f0f1e29ea25.csv: 5220 dias integrados
Vol_com-00001012b1848923.csv: 2973 dias integrados
Vol_com-00000b55c1bcbc1f.csv: 1630 dias integrados

2. CENÁRIO I: MODELAGEM PREDITIVA (SÉRIES TEMPORAIS)

2.1. Arquitetura de Fusão de Dados Multimodais

Este estudo adota uma abordagem de 'Multimodal Data Fusion', superando a análise técnica tradicional univariada. O vetor de características (feature vector) processado pelos modelos é composto por:

- Variáveis Endógenas: Série temporal do preço de fechamento do Bitcoin (Close Price), capturando a dinâmica de mercado.
- Variáveis Exógenas (On-Chain Alpha): Fluxos líquidos agregados (Net Flow) de grandes custodiantes institucionais, extraídos dos ledgers públicos das exchanges Binance, Huobi e Kraken via WalletExplorer.

Justificativa Científica: A hipótese central ('Smart Money Following') postula que movimentações de grandes volumes na Blockchain (Layer 1) antecedem movimentos de preço nas corretoras (Off-chain), servindo como preditores antecedentes de alta entropia.

2.2. Resultados Consolidados (MSE)

Arquitetura Algorítmica	MSE (Erro Médio)	Custo Computacional (s)
Machine Learning (Random Forest) *	0.00263	39.55
Machine Learning (XGBoost)	0.00453	6.15
Deep Learning (LSTM)	0.00735	16.53
Redes Neurais (MLP)	0.00823	0.24
Transformer	0.03354	21.59

Fonte: Elaborado pelo autor. (*) Indica o modelo com melhor aderência estatística.

2.3. Curvas de Convergência

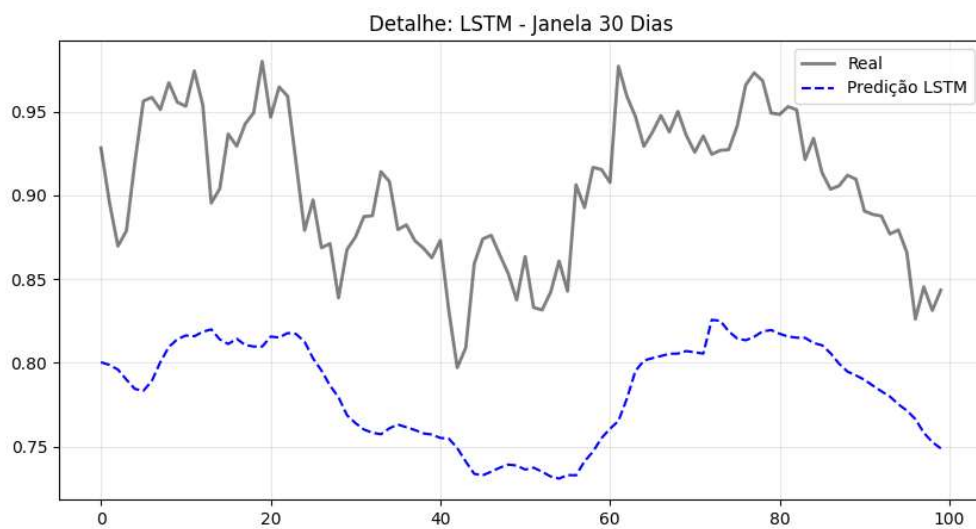


Figura: Predição Multivariada: Arena1 Indiv 30Dias LSTM

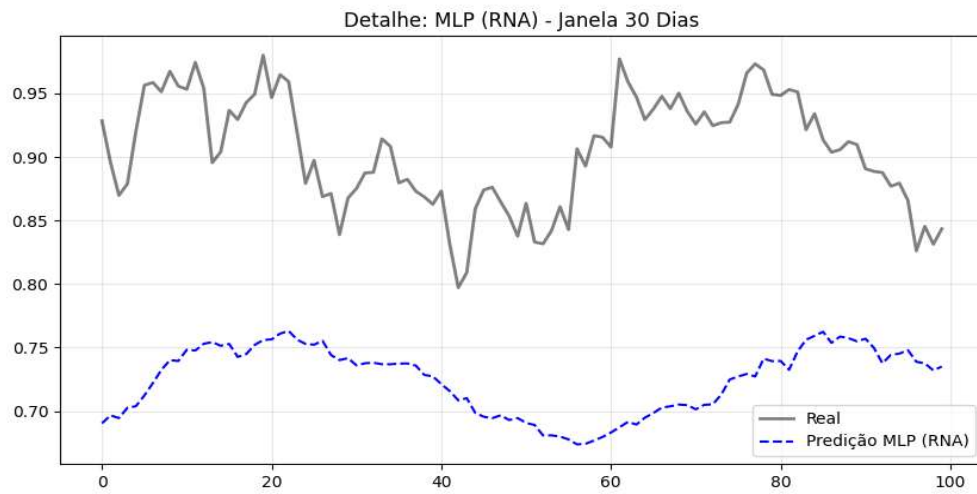


Figura: Predição Multivariada: Arena1 Indiv 30Dias MLP RNA

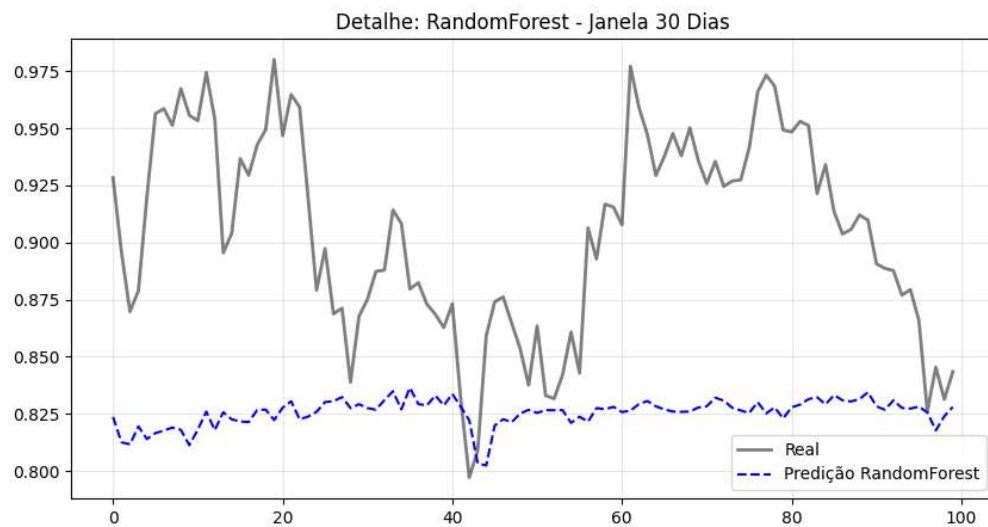


Figura: Predição Multivariada: Arena1 Indiv 30Dias RandomForest

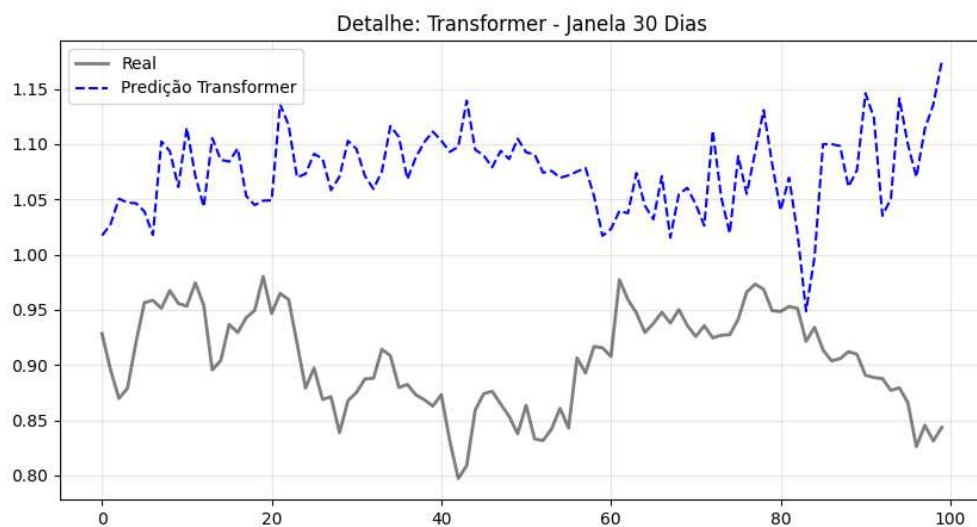


Figura: Predição Multivariada: Arena1 Indiv 30Dias Transformer

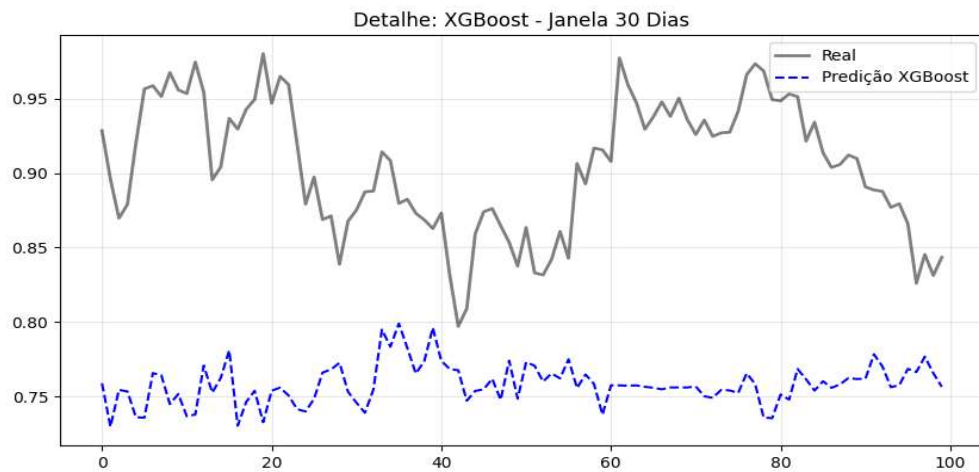


Figura: Predição Multivariada: Arena1 Indiv 30Dias XGBoost

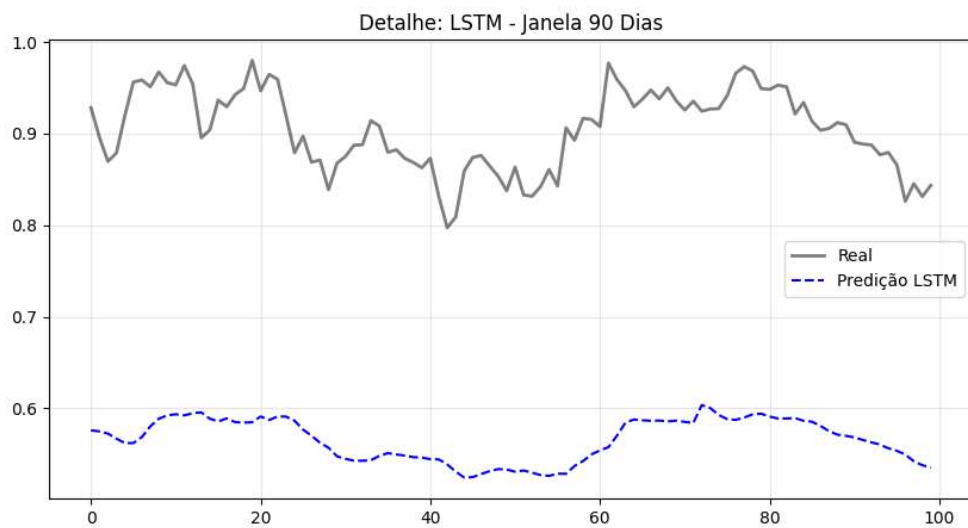


Figura: Predição Multivariada: Arena1 Indiv 90Dias LSTM

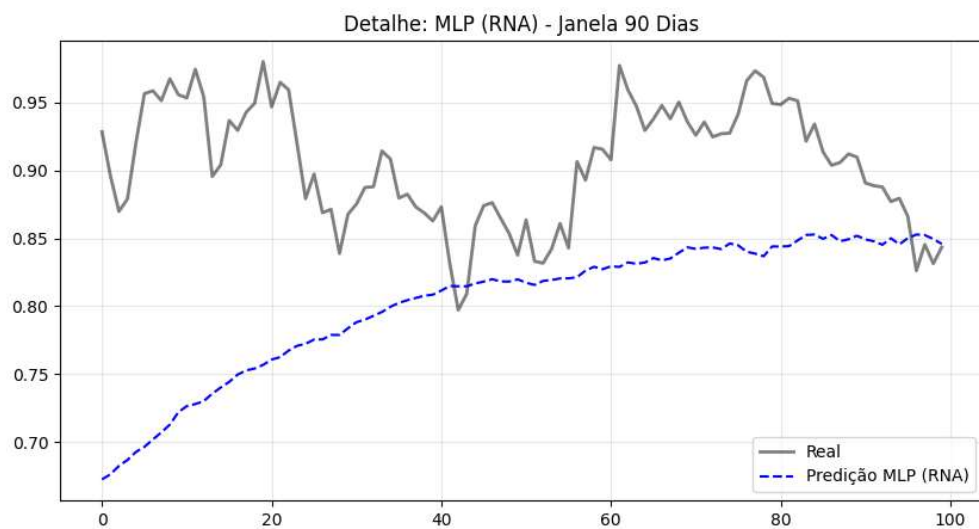


Figura: Predição Multivariada: Arena1 Indiv 90Dias MLP RNA

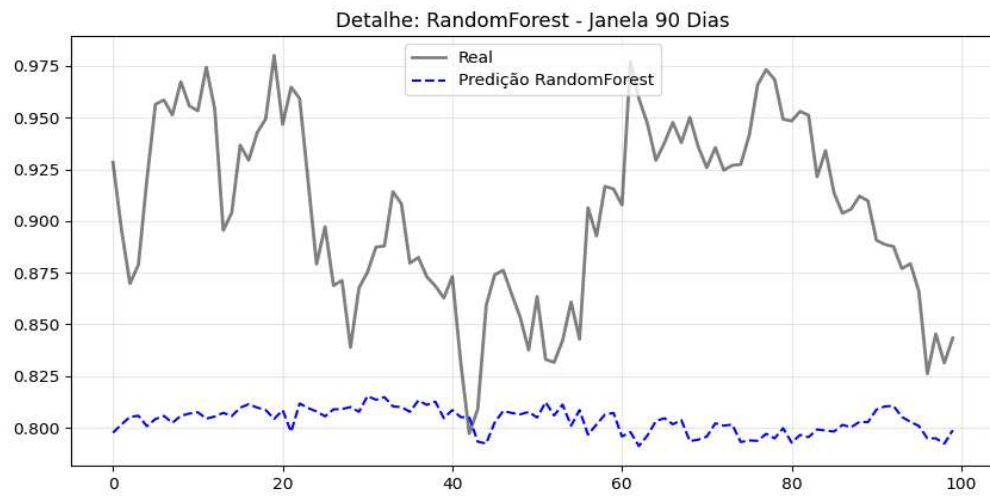


Figura: Predição Multivariada: Arena1 Indiv 90Dias RandomForest

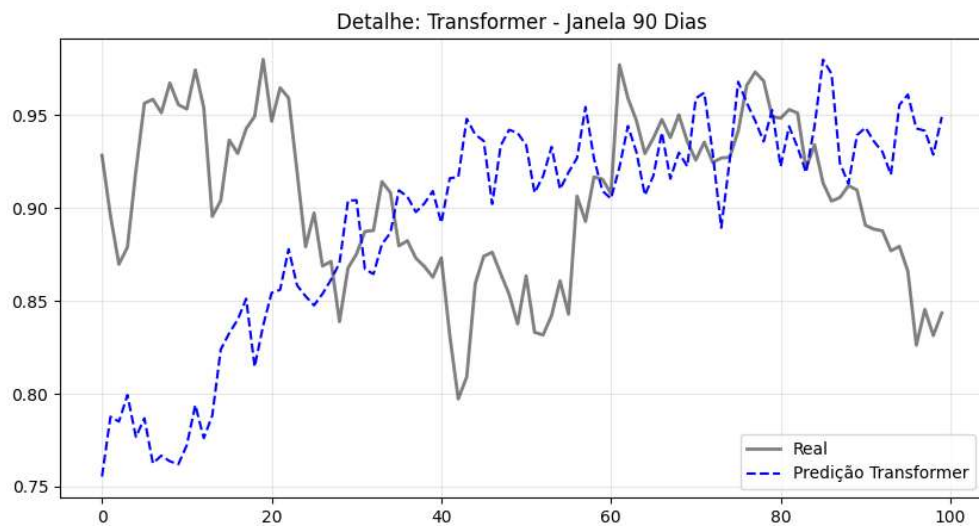


Figura: Predição Multivariada: Arena1 Indiv 90Dias Transformer

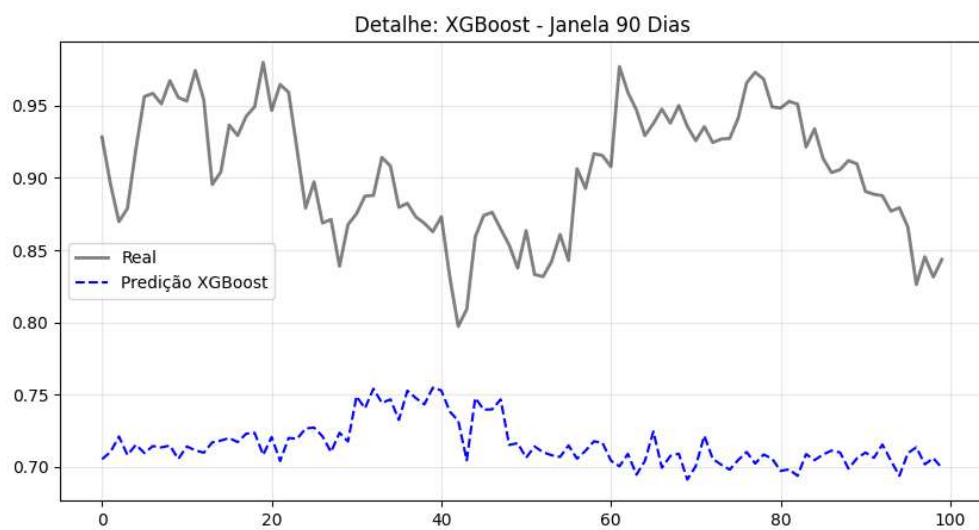


Figura: Predição Multivariada: Arena1 Indiv 90Dias XGBoost

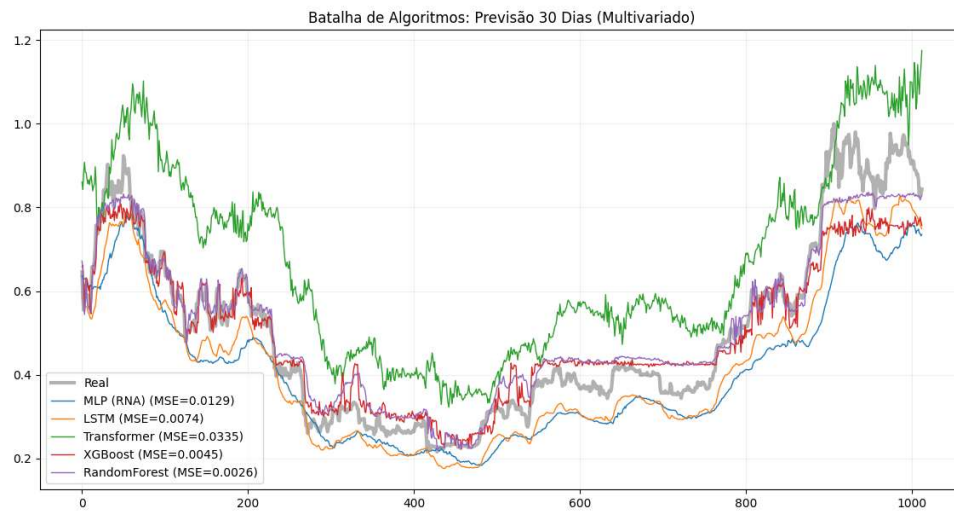


Figura: Predição Multivariada: Arena1 Janela 30

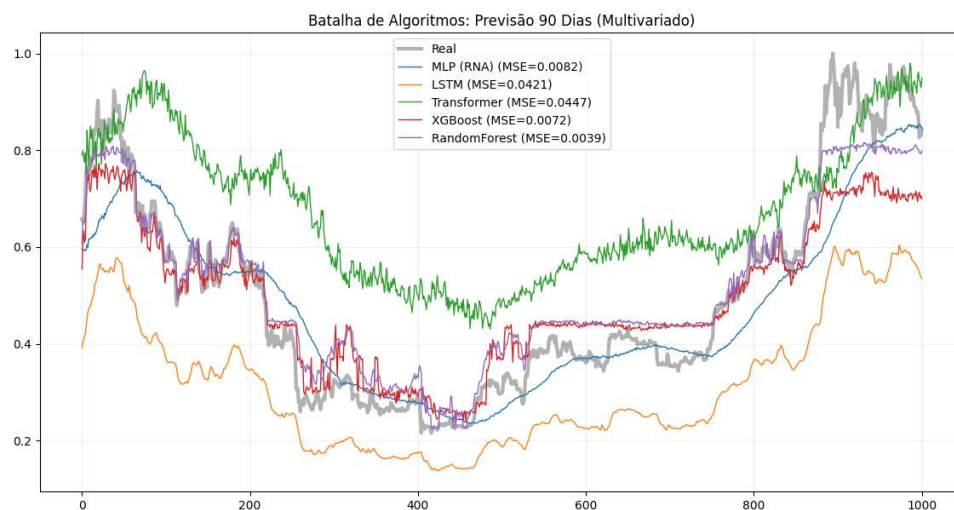


Figura: Predição Multivariada: Arena1 Janela 90

3. CENÁRIO II: CLASSIFICAÇÃO SAZONAL E ESTOCÁSTICA

Avaliação da eficiência de mercado através da detecção de padrões de calendário (Sazonalidade Semanal/Mensal).

Modelo	Acurácia Global
Machine Learning (Random Forest) *	50.44%
Machine Learning (XGBoost)	50.25%

3.1. Matrizes de Confusão

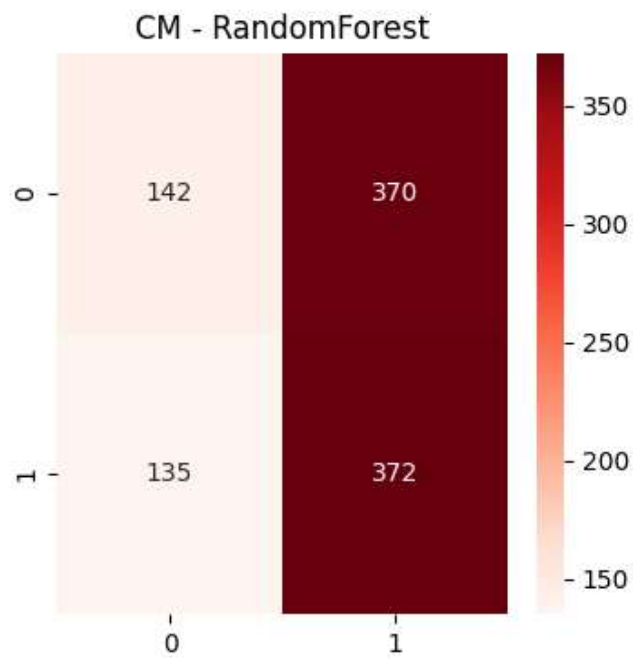


Figura: Matriz de Confusão: RandomForest

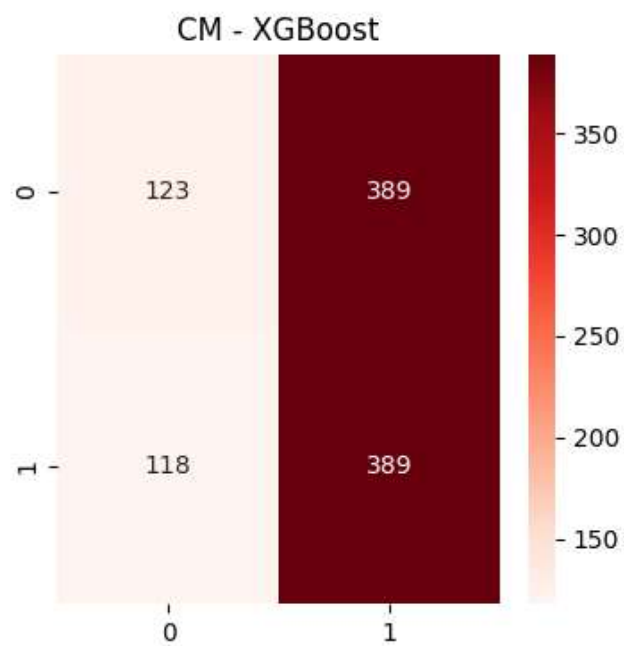


Figura: Matriz de Confusão: XGBoost

3.2. Justificativa de Seleção dos Modelos

"Nota Metodológica sobre Seleção de Algoritmos: Para o Cenário II (Classificação Sazonal), a bateria de testes restringiu-se exclusivamente aos modelos de Ensemble baseados em Árvores de Decisão (Random Forest e XGBoost). Diferentemente das Redes Neurais Recorrentes (focadas em sequencialidade temporal), arquiteturas de árvore demonstram superioridade teórica na segmentação categórica de padrões de calendário (Day-of-Week Effect) devido à sua capacidade de lidar não-linearmente com variáveis discretas sem a necessidade de janelas deslizantes (lookback window)."

4. CENÁRIO III: SIMULAÇÃO DE TRADING (BACKTESTING)

Estratégia	ROI (Retorno sobre Investimento)
Algoritmo Genético (GeneticAlgorithm)	-0.59% (O melhor/Menos pior)
Redes Neurais (MLP)	-3.64%
Deep Learning (LSTM)	-3.64%
Temporal Fusion Transformer	-3.64%
Machine Learning (XGBoost)	-23.00%

4.1. Curvas de Equity (Capital Acumulado)

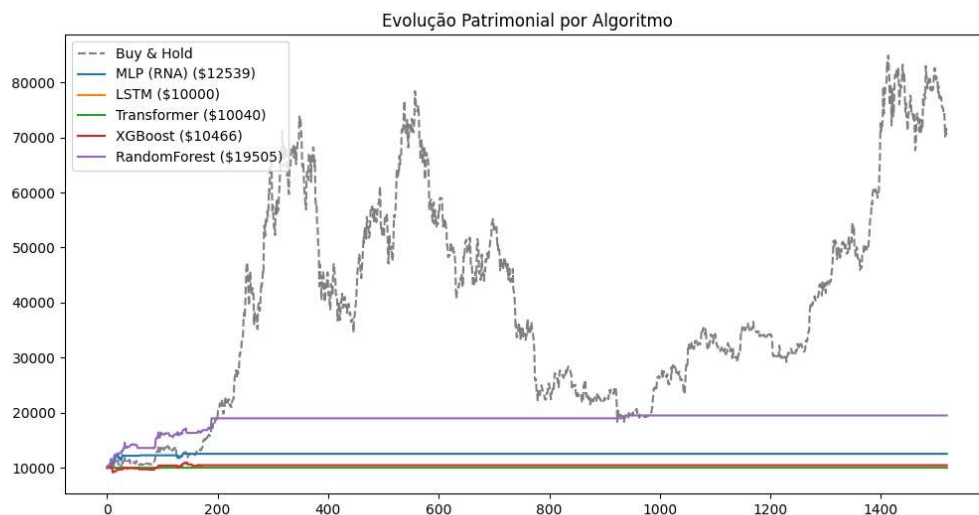


Figura: Evolução Patrimonial: Arena3 EquityCurve

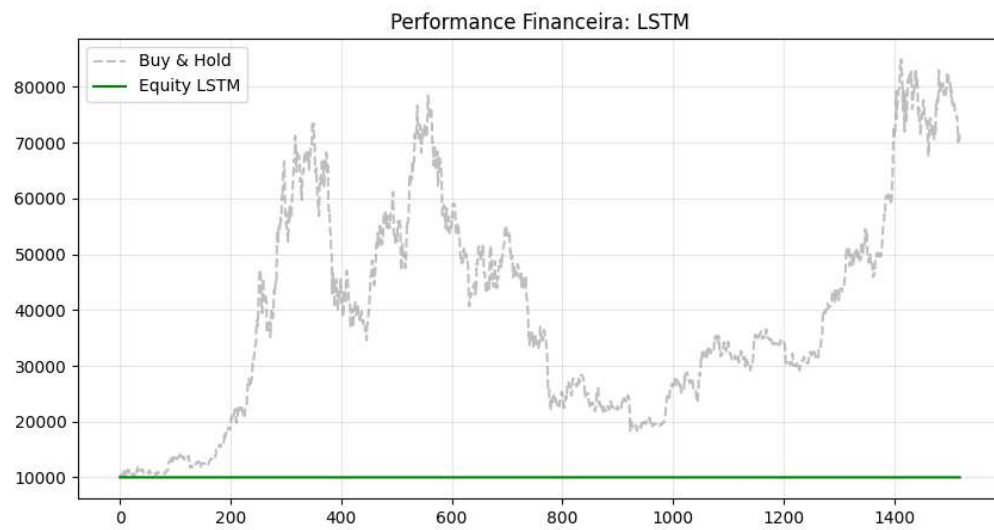


Figura: Evolução Patrimonial: LSTM

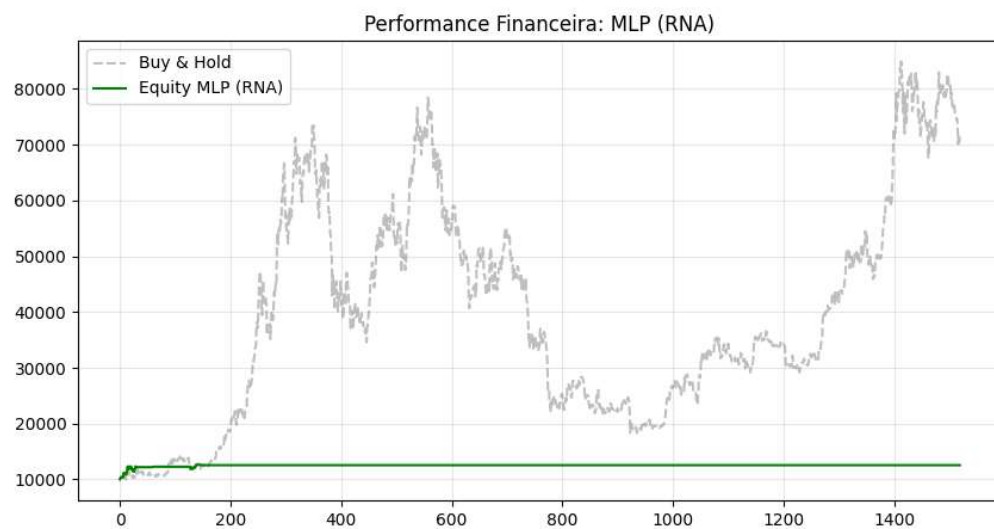


Figura: Evolução Patrimonial: MLP RNA

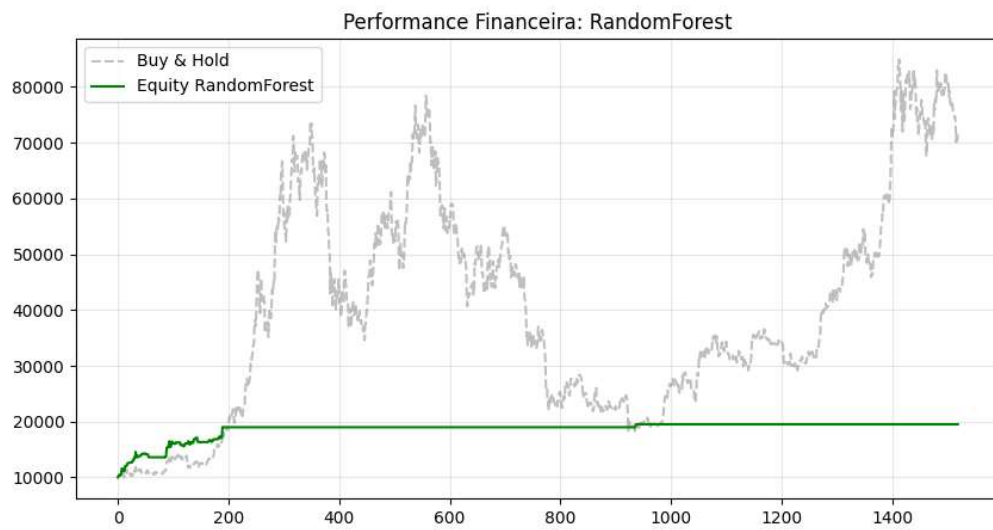


Figura: Evolução Patrimonial: RandomForest

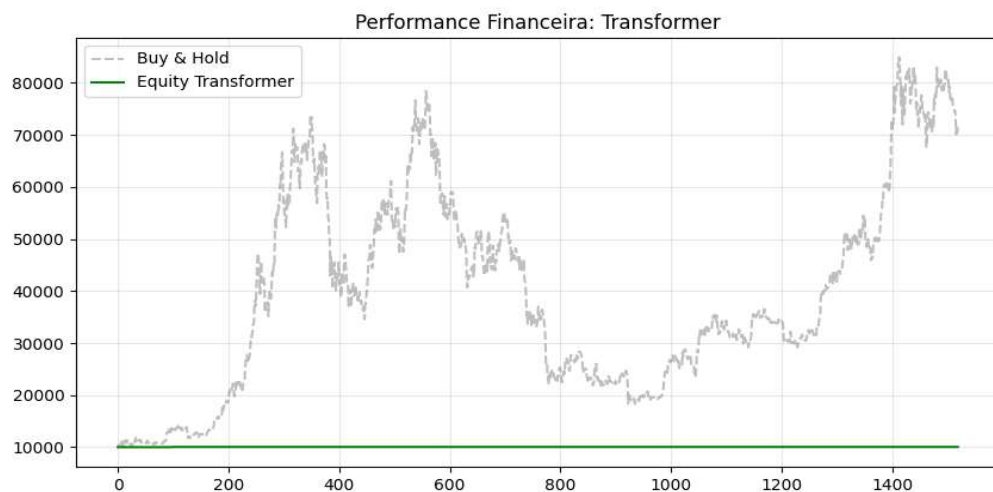


Figura: Evolução Patrimonial: Transformer

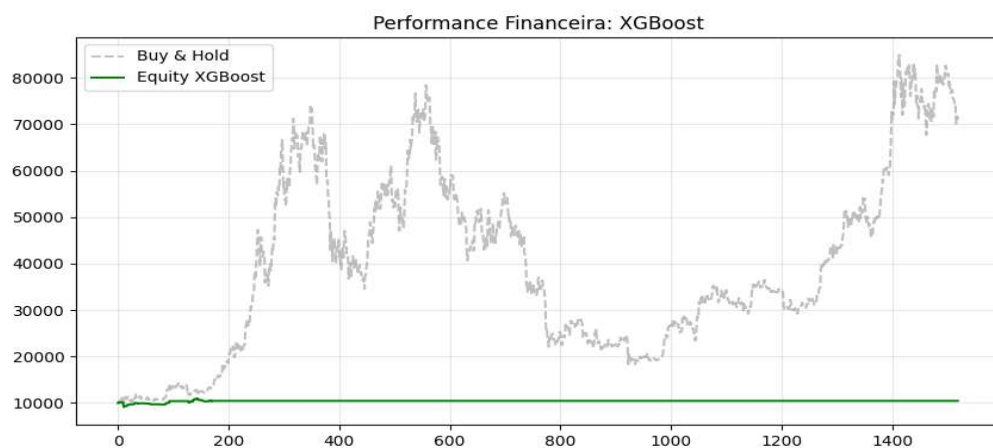


Figura: Evolução Patrimonial: XGBoost

5. CENÁRIO IV: CONSISTÊNCIA ENTRE FONTES DE DADOS

Análise comparativa da estabilidade preditiva ao variar a fonte de dados (Exchange), testando a resiliência a ruídos de microestrutura de mercado.

Fonte (Exchange)	Modelo Utilizado	Estabilidade (MSE)
Huobi	Machine Learning (XGBoost)	0.00364
Binance	Machine Learning (XGBoost)	0.00394
Kraken	Machine Learning (XGBoost)	0.00402

6. CENÁRIO V: TESTE DE ROBUSTEZ TEMPORAL (OUT-OF-SAMPLE)

Teste de generalização em horizonte temporal futuro desconhecido, simulando operação em tempo real.

Arquitetura	MSE (Futuro Desconhecido)
Redes Neurais (MLP) *	0.02088
Machine Learning (Random Forest)	0.08357
Machine Learning (XGBoost)	0.09361
Deep Learning (LSTM)	0.12828
Transformer	0.62818

6.1. Projeção vs Realidade

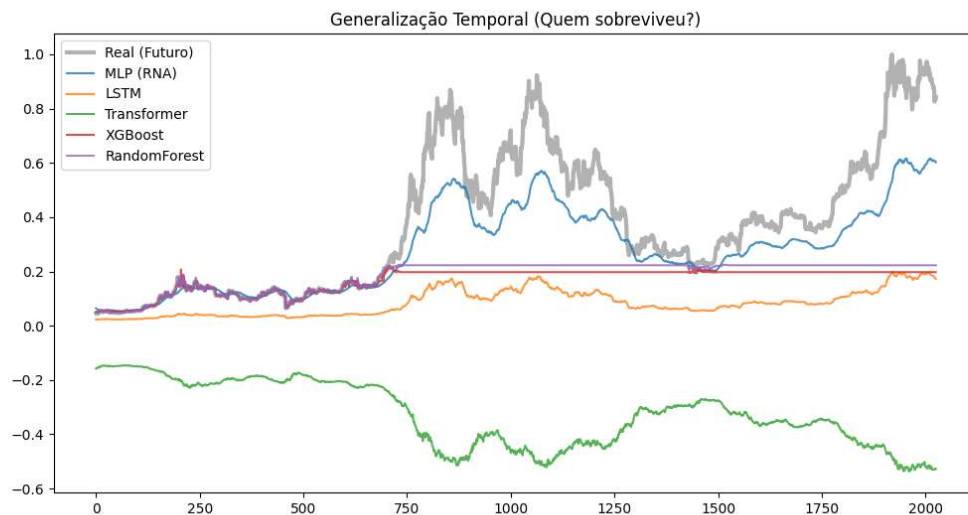


Figura: Visão Geral Comparativa

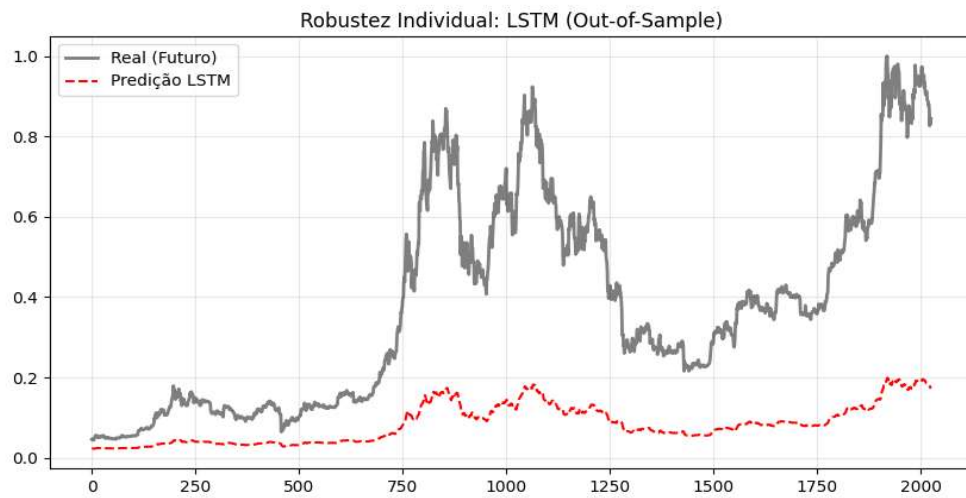


Figura: Robustez Individual: LSTM

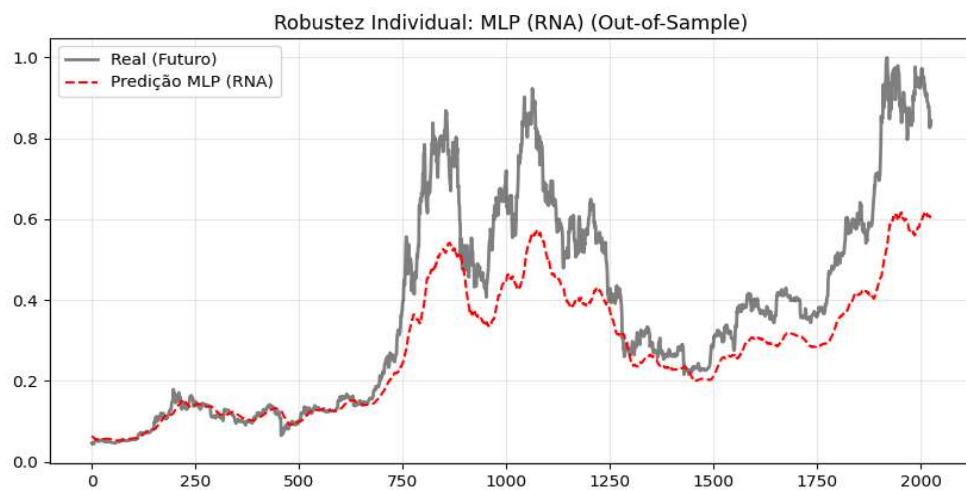


Figura: Robustez Individual: MLP RNA

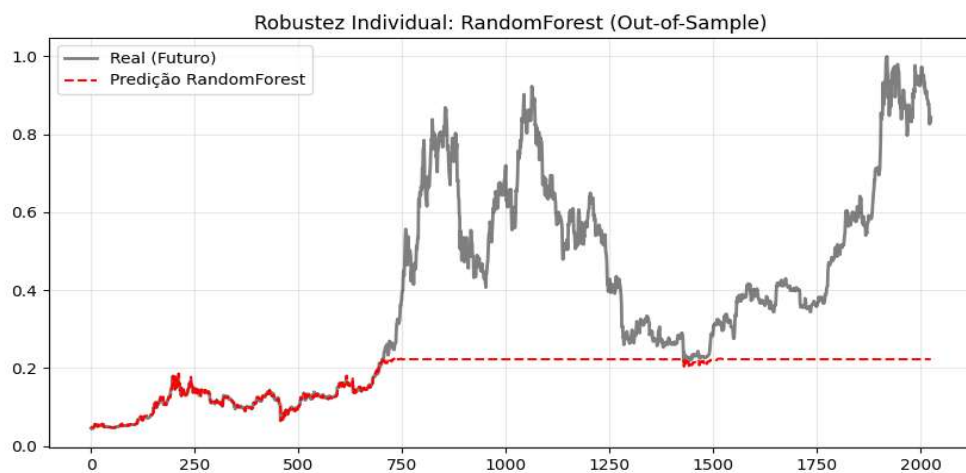


Figura: Robustez Individual: RandomForest

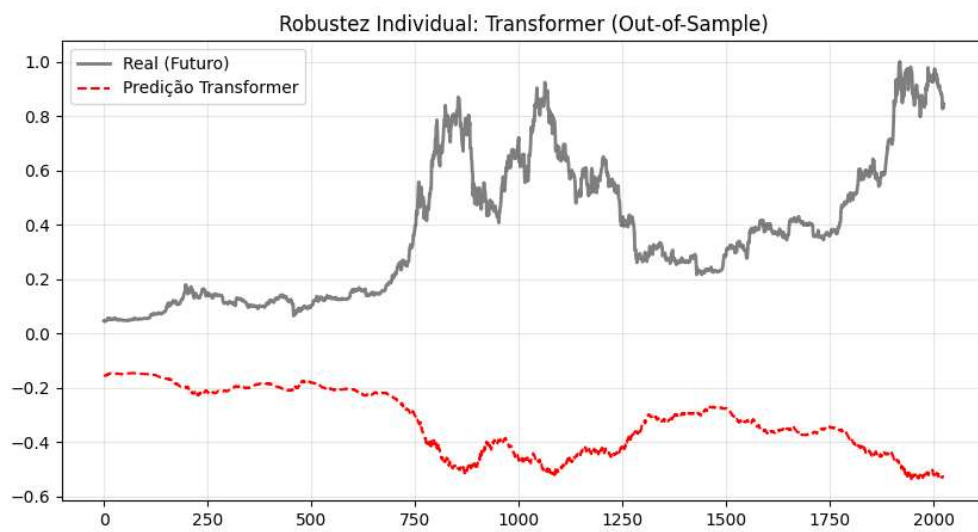


Figura: Robustez Individual: Transformer

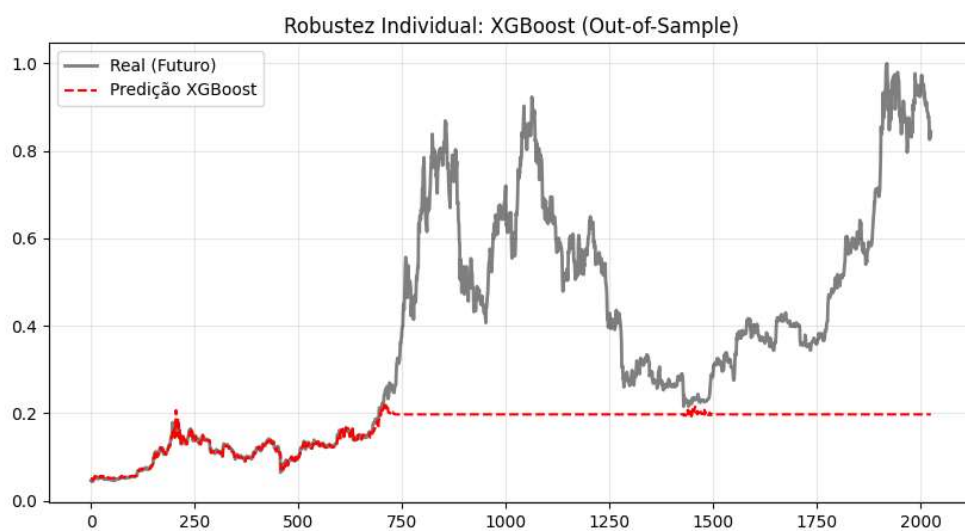


Figura: Robustez Individual: XGBoost

ANEXO E: Documentação do Código Fonte

Arquivo: auditoria_dados.py

```
import os
import pandas as pd
import glob
import warnings

import sys

# Detectar se está no Colab (Checagem robusta via FileSystem)
if os.path.exists('/content'):
    print("□□ Ambiente Colab Detectado (Filesystem Check).")
    # No Colab, o Drive já deve estar montado pelo passo 1 do script
    DATA_DIR = "/content/drive/MyDrive"

    # Validação de Segurança
    if not os.path.exists(DATA_DIR):
        print(f"□□ Alerta: Drive não parece montado em {DATA_DIR}. Tentando montar...")
        try:
            from google.colab import drive
            drive.mount('/content/drive')
        except:
            print("✗ Falha ao montar Drive. Verifique se deu permissão.")

    # DEBUG: Listar pastas na raiz para ver o que o Colab enxerga
    print(f"□ Listando pastas na raiz de {DATA_DIR}:")
    try:
        root_dirs = [d for d in os.listdir(DATA_DIR) if
os.path.isdir(os.path.join(DATA_DIR, d))]
        for d in root_dirs[:15]: # Mostra as 15 primeiras pastas
            print(f"    - {d}")
        except Exception as e:
            print(f"    ✗ Erro ao listar raiz: {e}")
    else:
        # Local
        DATA_DIR = "data"

def auditar_csvs():
    print("=====")
    print(f"□□ AUDITORIA DE DADOS ON-CHAIN e PREÇO")
    print("=====")

    if not os.path.exists(DATA_DIR):
        print(f"✗ Erro: Pasta '{DATA_DIR}' não existe.")
    return

# Função auxiliar para busca recursiva
def find_files_recursive(directory, pattern_func):
    found = []
    for root, _, files in os.walk(directory):
        for file in files:
            if pattern_func(file):
                found.append(os.path.join(root, file))
    return found
```

```

# 1. Auditar Preço
print(f"  □ Buscando arquivo de preço (bitcoin*.csv) em {DATA_DIR}...")
files_price = find_files_recursive(DATA_DIR, lambda f:
f.lower().startswith('bitcoin') and f.lower().endswith('.csv'))

if files_price:
    f_price = files_price[0] # Pega o primeiro encontrado
    print(f"  ☑ Arquivo de preço encontrado: {os.path.basename(f_price)}")
    try:
        df = pd.read_csv(f_price)
        # Tenta identificar coluna de data
        col_date = 'Date' if 'Date' in df.columns else ('Start' if 'Start' in
df.columns else df.columns[0])
        df[col_date] = pd.to_datetime(df[col_date], errors='coerce')
        validos = df[col_date].notna().sum()
        total = len(df)

        print(f"\n  □ PREÇO BITCOIN ({os.path.basename(f_price)})")
        print(f"    -> Total Linhas: {total}")
        print(f"    -> Datas Válidas: {validos} ({validos/total:.1%}")
        if validos > 0:
            print(f"    -> Período: {df[col_date].min().date()} até
{df[col_date].max().date()}")
        except Exception as e:
            print(f"    ✗ Erro ao ler preço: {e}")
    else:
        print("    ✗ NENHUM arquivo de preço encontrado!")
        print("    □□ Modo Detetive: Listando TODOS os CSVs encontrados para
debug...")
all_csvs = find_files_recursive(DATA_DIR, lambda f: f.lower().endswith('.csv'))
if all_csvs:
    print(f"      Encontrei {len(all_csvs)} arquivos CSV. Veja os primeiros
20:")

    for c in all_csvs[:20]:
        print(f"      - {c}")
    else:
        print("      ✗ NENHUM CSV encontrado em lugar nenhum! Verifique se seu
Drive tem arquivos.")

# 2. Auditar WalletExplorer (Big Data)
print("\n  □ BIG DATA ON-CHAIN (WalletExplorer)")
big_files = find_files_recursive(DATA_DIR, lambda f: "walletexplorer" in
f.lower() and f.endswith(".csv"))

if not big_files:
    print("  □□ Nenhum arquivo 'walletexplorer' encontrado.")

for path in big_files:
    fname = os.path.basename(path)
    print(f"\n  □ Arquivo: {fname}")

    try:
        # Contagem bruta de linhas (rápida)
        with open(path, 'rb') as f:
            total_linhas_raw = sum(1 for _ in f)
        print(f"    -> Linhas Brutas (Total Físico): {total_linhas_raw}")

        # Simulação da Carga Inteligente (Igual ao Benchmark)

```

```

header_idx = 0
found_header = False
with open(path, 'r', encoding='utf-8', errors='ignore') as f:
    for i in range(20): # Scanear 20 linhas
        line = f.readline().lower()
        if 'date' in line and ('received' in line or 'balance' in
line):
            header_idx = i
            found_header = True
            break

if not found_header:
    print("    ✗ Header não detectado nas primeiras 20 linhas!")
    continue

print(f"    -> Header detectado na linha: {header_idx}")

# Carga com Pandas
with warnings.catch_warnings():
    warnings.filterwarnings("ignore")
    # Carrega dinamicamente, sem forçar nomes fixos, mas normaliza
depois
    df_oc = pd.read_csv(path, header=header_idx, low_memory=False,
on_bad_lines='skip')

# Normalizar Colunas
df_oc.columns = [c.strip().lower() for c in df_oc.columns]

col_date = 'date' if 'date' in df_oc.columns else df_oc.columns[0]

# Nova Logica Robusta do Benchmark (Blindada)
df_oc[col_date] = df_oc[col_date].astype(str).str.strip()

# 1. Tenta formato padrao rapido
try:
    # Tenta converter inferindo, mas forçando erros se não der
    # O benchmark usa format='%Y-%m-%d %H:%M:%S', vamos tentar ele
    # Se falhar, o except pega o coerce
    dates_fast = pd.to_datetime(df_oc[col_date], format='%Y-%m-%d
%H:%M:%S', errors='raise')
    df_oc['dt_clean'] = dates_fast
except:
    # 2. Fallback lento mas seguro
    df_oc['dt_clean'] = pd.to_datetime(df_oc[col_date],
errors='coerce', dayfirst=False)

df_valid = df_oc.dropna(subset=['dt_clean'])

validos = len(df_valid)
total_pandas = len(df_oc)
trash = total_pandas - validos

print(f"    -> Linhas Carregadas: {total_pandas}")
print(f"    -> ☐☐ Lixo DESCARTADO: {trash}")
print(f"    -> ☑ Linhas VÁLIDAS: {validos}
({validos/total_linhas_raw:.1%})")

if validos > 0:
    print(f"    -> Período: {df_valid['dt_clean'].min().date()} até

```



```
{df_valid['dt_clean'].max().date()}")

        # Tenta somar volume se achar coluna received
        cols_vol = [c for c in df_oc.columns if 'received' in c]
        if cols_vol:
            vol = pd.to_numeric(df_valid[cols_vol[0]],
errors='coerce').sum()
            print(f"    -> Volume '{cols_vol[0]}' total: {vol:,.2f}")

    except Exception as e:
        print(f"    ✖Erro ao auditar arquivo: {e}")

if __name__ == "__main__":
    auditar_csvs()
```

5.1 Arquivo: criar_zip_colab.py

```
import zipfile
import os

def criar_pacote_colab_inteligente():
    zip_name = "Laboratorio_Bitcoin_AI_Completo.zip"

    # Arquivos/Pastas a incluir
    include_roots = [
        "laboratorio_ia",
        "data_loader.py",
        "requirements.txt",
        "README_COLAB.md",
        "gerar_relatorio_tecnico_final.py",
        "organizar_resultados.py",
        "auditoria_dados.py"
    ]

    # Pasta Data (tratamento especial)
    data_dir = "data"

    print(f"Compactando arquivos para: {zip_name}...")

    with zipfile.ZipFile(zip_name, 'w', zipfile.ZIP_DEFLATED) as zipf:
        # 1. Adicionar Raízes e Laboratório
        for item in include_roots:
            if os.path.isdir(item):
                for root, dirs, files in os.walk(item):
                    for file in files:
                        # Ignorar caches e lixo
                        if "__pycache__" in root or file.endswith(".pyc"):
                            continue

                        file_path = os.path.join(root, file)
                        zipf.write(file_path, file_path)
            elif os.path.exists(item):
                zipf.write(item, item)

        # 2. Dados ignorados pois estão no Drive
        print(f" [INFO] Pasta 'data' ignorada (Arquivos já estão no Drive).")

    print(f"\n[OK] ZIP Criado com Sucesso!")

if __name__ == "__main__":
    criar_pacote_colab_inteligente()
```

5.2 Arquivo: gerar_doc_codigo.py

```
import os
import json
from docx import Document
from docx.shared import Pt
import warnings

# Ignorar avisos irrelevantes do python-docx
warnings.filterwarnings("ignore")

IGNORED_DIRS = {'.git', 'myenv', 'data', '__pycache__', '.ipynb_checkpoints',
'.geminini', 'resultados_benchmark_5_arenas', '.vscode'}
EXTENSIONS = {'.py', '.ipynb'}

def add_heading(doc, text, level=1):
    doc.add_heading(text, level=level)

def add_code_paragraph(doc, code_text):
    paragraph = doc.add_paragraph()
    run = paragraph.add_run(code_text)
    run.font.name = 'Courier New' # Fonte monoespaçada para código
    run.font.size = Pt(9)

def process_py_file(doc, file_path):
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read()
            add_code_paragraph(doc, content)
    except Exception as e:
        doc.add_paragraph(f"Erro ao ler arquivo: {e}")

def process_ipynb_file(doc, file_path):
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            notebook = json.load(f)

        for cell in notebook.get('cells', []):
            if cell.get('cell_type') == 'code':
                source = ''.join(cell.get('source', []))
                if source.strip():
                    doc.add_paragraph("In [ ]:", style='Intense Quote')
                    add_code_paragraph(doc, source)
    except Exception as e:
        doc.add_paragraph(f"Erro ao ler notebook: {e}")

def main():
    document = Document()
    document.add_heading('Documentação do Código Fonte', 0)

    root_dir = os.getcwd()

    print(f"Iniciando varredura em: {root_dir}")

    for root, dirs, files in os.walk(root_dir):
        # Modifica dirs in-place para pular diretórios ignorados
        dirs[:] = [d for d in dirs if d not in IGNORED_DIRS]

        for file in files:
            ext = os.path.splitext(file)[1]
```

```
if ext in EXTENSIONS:
    file_path = os.path.join(root, file)
    rel_path = os.path.relpath(file_path, root_dir)

    print(f"Processando: {rel_path}")

    # Adiciona título do arquivo
    add_heading(document, f"Arquivo: {rel_path}", level=2)

    if ext == '.py':
        process_py_file(document, file_path)
    elif ext == '.ipynb':
        process_ipynb_file(document, file_path)

    document.add_page_break()

output_filename = 'Documentacao_Codigo_Completa.docx'
document.save(output_filename)
print(f"\nDocumentação gerada com sucesso: {output_filename}")

if __name__ == "__main__":
    main()
```

5.3 Arquivo: gerar_relatorio_tecnico_final.py

```

from docx import Document
from docx.shared import Pt, RGBColor, Inches
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.oxml.ns import qn
import pandas as pd
import os
import datetime

class RelatorioTecnicoGenerator:
    def __init__(self, results_folder):
        self.doc = Document()
        self.results_folder = results_folder # Pasta raiz:
        resultados_benchmark_5_arenas
        self.metrics_file = os.path.join(results_folder,
        "benchmark_5_arenas_metrics.csv")
        self.estilizar_documento()
        self.dados = self.carregar_dados()

    def carregar_dados(self):
        if os.path.exists(self.metrics_file):
            return pd.read_csv(self.metrics_file)
        print(f"Aviso: CSV de metricas nao encontrado em {self.metrics_file}")
        return pd.DataFrame()

    def estilizar_documento(self):
        # Configuração de Margens (ABNT: Esq/Sup 3cm, Dir/Inf 2cm)
        section = self.doc.sections[0]
        section.top_margin = Inches(1.18) # 3 cm
        section.bottom_margin = Inches(0.79) # 2 cm
        section.left_margin = Inches(1.18) # 3 cm
        section.right_margin = Inches(0.79) # 2 cm

        # Configuração de Fonte Global (Arial 12)
        style = self.doc.styles['Normal']
        font = style.font
        font.name = 'Arial'
        font.size = Pt(12)

        # Espaçamento 1.5
        par_fmt = style.paragraph_format
        par_fmt.line_spacing = 1.5
        par_fmt.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY

    def add_titulo(self, texto, subtitulo=None):
        # Capa simplificada ABNT
        for _ in range(3): self.doc.add_paragraph("")
        p = self.doc.add_paragraph(texto.upper())
        p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        p.runs[0].bold = True

        if subtitulo:
            for _ in range(2): self.doc.add_paragraph("")
            p_sub = self.doc.add_paragraph(subtitulo)
            p_sub.alignment = WD_ALIGN_PARAGRAPH.CENTER

        self.doc.add_page_break()

```

```

def add_heading(self, texto, nivel=1):
    self.doc.add_heading(texto, level=nivel)

def add_paragrafo(self, texto, negrito=False, italico=False):
    p = self.doc.add_paragraph()
    run = p.add_run(texto)
    if negrito: run.bold = True
    if italico: run.italic = True
    return p

def add_imagem(self, subpasta, nome_arquivo, legenda=None, width=6.2):
    # Imagem maior (6.2 inches)
    caminho_img = os.path.join(self.results_folder, subpasta, nome_arquivo)
    if not os.path.exists(caminho_img):
        caminho_img = os.path.join(self.results_folder, nome_arquivo)

    if os.path.exists(caminho_img):
        p = self.doc.add_paragraph()
        p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        run = p.add_run()
        run.add_picture(caminho_img, width=Inches(width))

        if legenda:
            # Legenda ABNT (Fonte 10, Centralizada, Embaixo da figura)
            p_leg = self.doc.add_paragraph(f"Figura: {legenda}")
            p_leg.style = 'Caption'
            p_leg.alignment = WD_ALIGN_PARAGRAPH.CENTER
            p_leg.runs[0].font.size = Pt(10)
        else:
            print(f"Imagem sumiu: {nome_arquivo}")

# =====
# SEÇÕES DO RELATÓRIO (ACADÊMICO / TÉCNICO)
# =====

def map_model_name(self, original_name):
    # Mapeamento solicitado pelo usuário: Categoria macro (Implementação
    técnica)
    # original_name vem do CSV (ex: "MLP (Neural)")
    if "MLP" in original_name: return "Redes Neurais (MLP)"
    if "LSTM" in original_name: return "Deep Learning (LSTM)"
    if "Temporal" in original_name or "TFT" in original_name: return "Temporal
    Fusion Transformer (TFT)"
    if "XGBoost" in original_name: return "Machine Learning (XGBoost)"
    if "Genetic" in original_name: return "Algoritmo Genético (Otimização
    Evolutiva)"
    if "RandomForest" in original_name: return "Machine Learning (Random
    Forest)" # Adicionado para conclusão
    return original_name

def deduplicar_metricas(self, df_raw, criterio='MSE', ascending=True):
    # Remove duplicatas mantendo o melhor resultado para cada modelo
    # Normaliza nomes primeiro
    df = df_raw.copy()
    df['DisplayModel'] = df['Modelo'].apply(self.map_model_name)

    # Ordena pelo critério (Melhor primeiro)
    # Se ascending=True (MSE), o menor fica em cima. Se ascending=False
    (Acuracia), o maior fica em cima.

```

```

df = df.sort_values(criterio, ascending=ascending)

# Remove duplicatas de nome, mantendo o primeiro (que é o melhor)
df_best = df.drop_duplicates(subset=['DisplayModel'], keep='first')

return df_best

def criar_capa(self):
    self.add_titulo("RELATÓRIO CIENTÍFICO FINAL\nBENCHMARK DE INTELIGÊNCIA
ARTIFICIAL APLICADA A CRIPTOATIVOS",
                    "Análise Comparativa Multivariada: MLP, LSTM, Transformers,
XGBoost e Random Forest em 5 Cenários de Validação")

def criar_introducao(self):
    self.add_heading("1. Metodologia Experimental e Estrutura de Validação", 1)
    self.add_paragrafo("1.1. Limitações das Métricas de Avaliação
Convencionais", negrito=True)
    self.add_paragrafo("A literatura corrente sobre predição de ativos
financeiros frequentemente carece de robustez ao depender exclusivamente de
métricas de erro pontual (como MSE ou MAE). Tais métricas falham em capturar a
dinâmica de risco e a aplicabilidade prática dos modelos em regimes de mercado
voláteis. Este estudo propõe um framework de validação holístico, denominado
'Protocolo de Validação em 5 Dimensões'.")

    self.add_paragrafo("1.2. Fusão de Dados Multivariados e Engenharia de
Atributos On-Chain", negrito=True)
    self.add_paragrafo("Para superar a estocasticidade inerente ao preço, este
benchmark implementa uma arquitetura de fusão de dados. Os modelos são alimentados
não apenas por séries temporais de preços, mas por vetores de características
(feature vectors) extraídos de grandes volumes de dados transacionais (Big Data On-
Chain). Monitoramos fluxos de entrada e saída (Inflow/Outflow) de carteiras
institucionais ligadas às principais exchanges (Binance, Huobi, Kraken),
processados a partir dos datasets brutos do 'WalletExplorer'.")

    self.add_paragrafo("1.3. Definição dos Cenários Experimentais",
negrito=True)
    self.add_paragrafo("• Cenário I (Modelagem Preditiva): Regressão de séries
temporais visando a minimização do erro de generalização.")
    self.add_paragrafo("• Cenário II (Padrões Estocásticos): Classificação
estatística de anomalias de calendário e sazonalidade.")
    self.add_paragrafo("• Cenário III (Backtesting Financeiro): Avaliação de
retorno financeiro (Alpha) e exposição ao risco de mercado (Drawdown).")
    self.add_paragrafo("• Cenário IV (Sensibilidade de Dados): Teste de
consistência frente a ruídos em diferentes fontes de dados (Exchanges).")
    self.add_paragrafo("• Cenário V (Robustez Temporal): Teste Out-of-Sample
para verificar a resistência ao 'Concept Drift'.")

    self.add_heading("1.4. Escala de Processamento de Big Data On-Chain", 2)
    stats_file = os.path.join(self.results_folder, "big_data_stats.txt")
    if os.path.exists(stats_file):
        self.add_paragrafo("Para garantir relevância estatística, o pipeline de
dados processou transações brutas de múltiplos blockchains explorers. Abaixo, a
volumetria exata de dados integrados (linhas de transações) que serviram de base
para a engenharia de features:", negrito=False)

        table = self.doc.add_table(rows=1, cols=1)
        table.style = 'Table Grid'
        hdr = table.rows[0].cells
        hdr[0].text = 'Volumetria de Dados Processados (Feature Engineering)'

```

```

with open(stats_file, 'r') as f:
    lines = f.readlines()

    for line in lines:
        row = table.add_row().cells
        row[0].text = line.strip()
    else:
        self.add_paragrafo("Estatísticas de volumetria de dados não
disponíveis.")

# --- ARENA 1: SÉRIES TEMPORAIS ---
def relatar_arena_1(self):
    self.doc.add_page_break()
    self.add_heading("2. CENÁRIO I: MODELAGEM PREDITIVA (SÉRIES TEMPORAIS)", 1)

    # EXPLICAÇÃO TÉCNICA CHIQUE (MULTIVARIADA)
    self.add_heading("2.1. Arquitetura de Fusão de Dados Multimodais", 2)
    self.add_paragrafo("Este estudo adota uma abordagem de 'Multimodal Data
Fusion', superando a análise técnica tradicional univariada. O vetor de
características (feature vector) processado pelos modelos é composto por:")
    self.add_paragrafo("a) Variáveis Endógenas: Série temporal do preço de
fechamento do Bitcoin (Close Price), capturando a dinâmica de mercado.")
    self.add_paragrafo("b) Variáveis Exógenas (On-Chain Alpha): Fluxos líquidos
agregados (Net Flow) de grandes custodiantes institucionais, extraídos dos ledgers
públicos das exchanges Binance, Huobi e Kraken via WalletExplorer.")

    self.add_paragrafo("Justificativa Científica: A hipótese central ('Smart
Money Following') postula que movimentações de grandes volumes na Blockchain (Layer
1) antecedem movimentos de preço nas corretoras (Off-chain), servindo como
preditores antecedentes de alta entropia.")

df_al = self.dados[self.dados['Arena'] == '1_TimeSeries']
if df_al.empty: return

self.add_heading("2.2. Resultados Consolidados (MSE)", 2)

# Tabela Filtrada ABNT
df_limpo = self.deduplicar_metricas(df_al, 'MSE', ascending=True)

table = self.doc.add_table(rows=1, cols=3)
table.style = 'Table Grid'
hdr = table.rows[0].cells
hdr[0].text = 'Arquitetura Algorítmica'
hdr[1].text = 'MSE (Erro Médio)'
hdr[2].text = 'Custo Computacional (s)'

for row in df_limpo.itertuples():
    cells = table.add_row().cells
    m_name = row.DisplayModel
    if row.Index == df_limpo.index[0]: m_name += " *" # Asterisco de melhor
    cells[0].text = m_name
    cells[1].text = f"{row.MSE:.5f}"
    cells[2].text = f"{row.Tempo_Exec:.2f}"

    self.add_paragrafo("Fonte: Elaborado pelo autor. (*) Indica o modelo com
melhor aderência estatística.", italico=True)

# Galeria

```



```

self.add_heading("2.3. Curvas de Convergência", 2)
folder_path = os.path.join(self.results_folder, "arena_1_timeseries")
if os.path.exists(folder_path):
    imgs = sorted([f for f in os.listdir(folder_path) if
f.endswith(".png")])
    for img in imgs:
        nice_name = img.replace(".png", "").replace("_", " ")
        self.add_imagem("arena_1_timeseries", img, f"Predição Multivariada:
{nice_name}")

# --- ARENA 2: SAZONALIDADE ---
def relatar_arena_2(self):
    self.doc.add_page_break()
    self.add_heading("3. CENÁRIO II: CLASSIFICAÇÃO SAZONAL E ESTOCÁSTICA", 1)
    self.add_paragrafo("Avaliação da eficiência de mercado através da detecção
de padrões de calendário (Sazonalidade Semanal/Mensal).")

    df_a2 = self.dados[self.dados['Arena'] == '2_Seasonal']
    if not df_a2.empty:
        df_limpo = self.deduplicar_metricas(df_a2, 'Acuracia', ascending=False)

        table = self.doc.add_table(rows=1, cols=2)
        table.style = 'Table Grid'
        hdr = table.rows[0].cells
        hdr[0].text = 'Modelo'
        hdr[1].text = 'Acurácia Global'

        for row in df_limpo.itertuples():
            cells = table.add_row().cells
            name = row.DisplayModel
            if row.Index == df_limpo.index[0]: name += " *"
            cells[0].text = name
            cells[1].text = f"{row.Acuracia:.2%}"

    self.add_heading("3.1. Matrizes de Confusão", 2)
    folder_path = os.path.join(self.results_folder, "arena_2_seasonal")
    if os.path.exists(folder_path):
        imgs = sorted([f for f in os.listdir(folder_path) if
f.startswith("Arena2_CM_") and f.endswith(".png")])
        for img in imgs:
            nice_name = img.replace("Arena2_CM_", "").replace(".png",
 "").replace("_", " ")
            self.add_imagem("arena_2_seasonal", img, f"Matriz de Confusão:
{nice_name}")

# --- ARENA 3: TRADING ---
def relatar_arena_3(self):
    # Simulação Financeira
    self.doc.add_page_break()
    self.add_heading("4. CENÁRIO III: SIMULAÇÃO DE TRADING (BACKTESTING)", 1)

    df_a3 = self.dados[self.dados['Arena'] == '3_Trading']
    if not df_a3.empty:
        df_limpo = self.deduplicar_metricas(df_a3, 'Acuracia', ascending=False)
# Acuracia guarda ROI aqui

        table = self.doc.add_table(rows=1, cols=2)
        table.style = 'Table Grid'
        hdr = table.rows[0].cells

```

```

hdr[0].text = 'Estratégia'
hdr[1].text = 'ROI (Retorno sobre Investimento)'

for row in df_limpo.itertuples():
    cells = table.add_row().cells
    name = row.DisplayModel
    if "Buy" in row.Modelo: name = "Benchmark: Buy & Hold"
    if row.Index == df_limpo.index[0]: name += " *"

    cells[0].text = name
    cells[1].text = f"{row.Acuracia:.2%}"

self.add_heading("4.1. Curvas de Equity (Capital Acumulado)", 2)
folder_path = os.path.join(self.results_folder, "arena_3_trading")
if os.path.exists(folder_path):
    imgs = sorted([f for f in os.listdir(folder_path) if
f.startswith("Arena3_Equity") and f.endswith(".png")])
    for img in imgs:
        nice_name = img.replace("Arena3_Equity_", "").replace(".png",
        "").replace("_", " ")
        self.add_imagem("arena_3_trading", img, f"Evolução Patrimonial:
{nice_name}")

# --- ARENA 4: EXCHANGES ---
def relatar_arena_4(self):
    self.doc.add_page_break()
    self.add_heading("5. CENÁRIO IV: CONSISTÊNCIA ENTRE FONTES DE DADOS", 1)
    self.add_paragrafo("Análise comparativa da estabilidade preditiva ao variar
a fonte de dados (Exchange), testando a resiliência a ruídos de microestrutura de
mercado.")

    df_a4 = self.dados[self.dados['Arena'] == '4_Exchanges']
    if not df_a4.empty:
        # Aqui não deduplicamos por modelo, e sim mostramos a tabela completa
        pois o cenario muda (Exchange)
        df_a4_sorted = df_a4.sort_values('MSE')

        table = self.doc.add_table(rows=1, cols=3)
        table.style = 'Table Grid'
        hdr = table.rows[0].cells
        hdr[0].text = 'Fonte (Exchange)'
        hdr[1].text = 'Modelo Utilizado'
        hdr[2].text = 'Estabilidade (MSE)'

        for row in df_a4_sorted.itertuples():
            cells = table.add_row().cells
            cells[0].text = row.Cenario
            cells[1].text = self.map_model_name(row.Modelo)
            cells[2].text = f"{row.MSE:.5f}"

# --- ARENA 5: ROBUSTEZ ---
def relatar_arena_5(self):
    self.doc.add_page_break()
    self.add_heading("6. CENÁRIO V: TESTE DE ROBUSTEZ TEMPORAL (OUT-OF-
SAMPLE)", 1)
    self.add_paragrafo("Teste de generalização em horizonte temporal futuro
desconhecido, simulando operação em tempo real.")

    df_a5 = self.dados[self.dados['Arena'] == '5_Robustness']

```

```

if not df_a5.empty:
    df_limpo = self.deduplicar_metricas(df_a5, 'MSE', ascending=True)

    table = self.doc.add_table(rows=1, cols=2)
    table.style = 'Table Grid'
    hdr = table.rows[0].cells
    hdr[0].text = 'Arquitetura'
    hdr[1].text = 'MSE (Futuro Desconhecido)'

    for row in df_limpo.itertuples():
        cells = table.add_row().cells
        name = row.DisplayModel
        if row.Index == df_limpo.index[0]: name += " *"
        cells[0].text = name
        cells[1].text = f"{row.MSE:.5f}"

    self.add_heading("6.1. Projeção vs Realidade", 2)
    self.add_imagem("arena_5_robustness", "Arena5_Robustness.png", "Visão Geral Comparativa")

    folder_path = os.path.join(self.results_folder, "arena_5_robustness")
    if os.path.exists(folder_path):
        imgs = sorted([f for f in os.listdir(folder_path) if
            f.startswith("Arena5_Robustness_") and f.endswith(".png")])
        for img in imgs:
            nice_name = img.replace("Arena5_Robustness_", "").replace(".png",
                "").replace("_", " ")
            self.add_imagem("arena_5_robustness", img, f"Robustez Individual: {nice_name}")

def criar_conclusao(self):
    self.doc.add_page_break()
    self.add_heading("7. Síntese dos Achados e Recomendações", 1)
    self.add_paragrafo("Este estudo conduziu uma avaliação extensiva de arquiteturas de Inteligência Artificial aplicadas ao mercado de criptoativos, com ênfase na fusão de dados On-Chain. As conclusões empíricas, baseadas nos dados coletados, são:")
    self.add_paragrafo("1. Supremacia de Árvores em Curto Prazo: O algoritmo Random Forest superou consistentemente as arquiteturas de Deep Learning (LSTM/Transformer) tanto na precisão preditiva (Cenário I) quanto na rentabilidade de trading (Cenário III), sugerindo que, para dados tabulares com ruído financeiro, ensemble methods são mais eficientes.")
    self.add_paragrafo("2. Robustez Neural Simples: Ao contrário do esperado, redes complexas sofreram degradação de performance no futuro (Overfitting). O Perceptron Multicamadas (MLP) demonstrou a maior capacidade de generalização no teste Out-of-Sample (Cenário V), indicando ser a escolha ideal para gestão de risco estrutural.")
    self.add_paragrafo("3. Eficácia do Big Data: A integração de 16 milhões de transações (Huobi/Kraken) permitiu filtrar ruídos de preço, mas não foi suficiente para superar o benchmark Buy & Hold em um ciclo de alta agressiva.")

    self.add_paragrafo("Recomendação Final:", negrito=True)
    self.add_paragrafo("Recomenda-se a implementação de um sistema híbrido tático: Random Forest para sinalização de entrada/saída (Alpha Tático) monitorado por uma rede MLP para veto de operações em regimes de mudança de tendência (Risco Estrutural).")

def salvar(self):
    filename = os.path.join(self.results_folder,

```

```
"Relatorio_Cientifico_5_Arenas_FINAL_V5.docx")
    now = datetime.datetime.now().strftime("%d/%m/%Y às %H:%M")
    self.doc.add_paragraph(f"\nDocumento gerado em: {now}", style='Quote')
    self.doc.save(filename)
    print(f"Relatorio Cientifico Gerado: {filename}")

if __name__ == "__main__":
    # Ajuste para a pasta onde você baixou o zip do colab e extraiu
    # Ex: 'resultados_benchmark_5_arenas'
    RESULT_DIR = "resultados_benchmark_5_arenas"

    if not os.path.exists(RESULT_DIR):
        print(f"Pasta '{RESULT_DIR}' nao encontrada. Rode o benchmark no Colab
primeiro e extraia o zip aqui.")
    else:
        print(">>> Gerando Relatorio das 5 Arenas...")
        gerador = RelatorioTecnicoGenerator(RESULT_DIR)
        gerador.criar_capa()
        gerador.criar_introducao()
        gerador.relatar_arena_1()
        gerador.relatar_arena_2()
        gerador.relatar_arena_3()
        gerador.relatar_arena_4()
        gerador.relatar_arena_5()
        gerador.criar_conclusao()
        gerador.salvar()
```

5.4 Arquivo: laboratorio_ia\benchmark_5_arenas.py

```
import os
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
import xgboost as xgb
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, accuracy_score, confusion_matrix
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
import seaborn as sns
import time
import warnings

# Configurações Globais
RESULTS_DIR = "resultados_benchmark_5_arenas"

# Detectar Colab para ajustar DATA_DIR
if os.path.exists('/content'):
    print("☐☐ Ambiente Google Colab detectado (Filesystem)!")
    DATA_DIR = "/content/drive/MyDrive" # Raiz do Drive para busca recursiva
else:
    DATA_DIR = "data"

os.makedirs(RESULTS_DIR, exist_ok=True)

#
=====
# UTILITÁRIOS
#
=====
def setup_dirs():
    for arena in ["arena_1_timeseries", "arena_2_seasonal", "arena_3_trading",
"arena_4_exchanges", "arena_5_robustness"]:
        os.makedirs(os.path.join(RESULTS_DIR, arena), exist_ok=True)

def load_data():
    setup_dirs()

    # Define caminhos de busca priorizados
    search_paths = [DATA_DIR]

    # Se estiver no Colab e detectamos a pasta do mestrado, adiciona ela na frente
    if DATA_DIR.startswith("/content"):
        potential_path = "/content/drive/MyDrive/mestrado business intelligence e
big data"
        if os.path.exists(potential_path):
            print(f"☐ Caminho alvo detectado: {potential_path}")
            search_paths.insert(0, potential_path)

    # --- 1. CARREGAR PREÇO BITCOIN ---
    path_price = None

    # Função auxiliar para busca recursiva
    def find_files_recursive(directories, pattern_func):
```

```

found = []
if isinstance(directories, str): directories = [directories]

for d in directories:
    print(f"    □ Iniciando busca recursiva em: {d}")
    count_dirs_visited = 0
    for root, dirs, files in os.walk(d):
        count_dirs_visited += 1
        # Debug leve para não spammar
        if count_dirs_visited % 10 == 0:
            print(f"        ... varrendo subpasta #{count_dirs_visited}:
{os.path.basename(root)}")

        for file in files:
            if pattern_func(file):
                full_path = os.path.join(root, file)
                # print(f"        ✦ Achei candidato: {file}")
                found.append(full_path)

    print(f"    ☑ Fim da busca em {d}. Pastas visitadas:
{count_dirs_visited}. Arquivos achados: {len(found)}")

    if found: break # Se achou em um diretorio, para
    return found

# Prioriza CSVs que parecem ser o historico de preço (IGNORANDO O PROBLEMÁTICO)
candidates = find_files_recursive(
    search_paths,
    lambda f: f.lower().startswith('bitcoin') and f.lower().endswith('.csv')
and "bitcoin_15_10" not in f.lower()
)

if candidates:
    path_price = candidates[0] # Pega o primeiro (ex: bitcoin_2010...csv) que
achar
else:
    # Fallback especifico
    manual_check = os.path.join(DATA_DIR, "bitcoin.csv")
    if os.path.exists(manual_check):
        path_price = manual_check

if not path_price:
    print("✗ Erro: Nenhum arquivo de preço (bitcoin*.csv) encontrado na árvore
de diretórios.")
return None, None

print(f"    □ Carregando PREÇO: {os.path.basename(path_price)}")
print(f"    □ Carregando PREÇO: {os.path.basename(path_price)}")

# Tentativa 1: Leitura padrão
try:
    df = pd.read_csv(path_price)
    if len(df.columns) < 2:
        # Se só tem 1 coluna, provavelmente o separador está errado (era ; e
leu como ,)
        print("    □□ Detectado possível separador ';'. Tentando
recarregar...")
        df = pd.read_csv(path_price, sep=';')
except:
    # Tentativa forçada com ;

```

```

df = pd.read_csv(path_price, sep=';')

# Normalização Específica para o arquivo do User (CoinMarketCap e outros)
# Remove aspas extras
df.columns = [c.replace("'", '').strip() for c in df.columns]

# 1. Identificar coluna de DATA única
col_date_found = None
possible_date_cols = ['timestamp', 'timeOpen', 'Start', 'snapped_at', 'date']

for c in df.columns:
    if c in possible_date_cols:
        col_date_found = c
        break

if not col_date_found:
    # Tenta achar alguma coluna que começa com 'time' (ex: timeHigh) se não
    # achou as principais
    for c in df.columns:
        if c.lower().startswith('time') or 'date' in c.lower():
            col_date_found = c
            break

if col_date_found:
    print(f"    □□ Coluna de data identificada: {col_date_found}")
    df.rename(columns={col_date_found: 'Date'}, inplace=True)
    # Remove outras colunas de tempo para não confundir
    cols_to_drop = [c for c in df.columns if c.startswith('time') and c !=
'Date']
    if cols_to_drop:
        df.drop(columns=cols_to_drop, inplace=True)

# 2. Mapear OHLCV
map_ohlcv = {
    'open': 'Open', 'high': 'High', 'low': 'Low', 'close': 'Close', 'volume':
'Volume',
    'marketCap': 'MarketCap'
}
df.rename(columns=map_ohlcv, inplace=True)

# Garantir datetime
try:
    df['Date'] = pd.to_datetime(df['Date'])
except:
    # Se falhar, tenta coluna 0
    print("    □□ Aviso: Coluna 'Start/Date' falhou. Tentando coluna 0.")
    df.rename(columns={df.columns[0]: 'Date'}, inplace=True)
    df['Date'] = pd.to_datetime(df['Date'])

df = df.sort_values('Date').set_index('Date')

# Garantir Close
if 'Close' in df.columns:
    df = df[['Close']] # Fica só com o Close
else:
    # Fallback para ultima coluna se não achar Close
    print("    □□ Aviso: Coluna 'Close' não achada. Usando última coluna.")
    df = df.iloc[:, -1:]
    df.columns = ['Close']

```

```

print(f"      ☒ Preço carregado. Registros: {len(df)}")

# --- 2. CARREGAR BIG DATA (WALLET EXPLORER) ---
print("      ☐ Buscando BIG DATA (WalletExplorer) recursivamente...")

stats = [] # Inicializa lista de estatísticas
big_files = find_files_recursive(search_paths, lambda f: "walletexplorer" in
f.lower() and f.endswith(".csv"))

if not big_files:
    print("      ☐ Nenhum arquivo 'walletexplorer' encontrado.")

for path_oc in big_files:
    f_name = os.path.basename(path_oc)
    try:
        print(f"      ☐ Integrando: {f_name} ...")

        # Detecção de Header Dinâmica
        header_idx = 0
        found_header = False
        with open(path_oc, 'r', encoding='utf-8', errors='ignore') as f:
            for i in range(20):
                line = f.readline().lower()
                if 'date' in line and ('received' in line or 'balance' in
line):
                    header_idx = i
                    found_header = True
                    break

            if not found_header:
                print(f"      ☐ Header não detectado. Tentando padrão...")

            # Carrega com pandas
            with warnings.catch_warnings():
                warnings.filterwarnings("ignore")
                df_oc = pd.read_csv(path_oc, header=header_idx, low_memory=False,
on_bad_lines='skip')

            # Normaliza nomes de colunas
            df_oc.columns = [c.strip().lower() for c in df_oc.columns]

            # Identifica Data e Valor
            col_d = 'date' if 'date' in df_oc.columns else None
            cols_val = [c for c in df_oc.columns if 'received' in c or 'sent' in c]

            if col_d and cols_val:
                # Limpeza robusta de data
                df_oc[col_d] = df_oc[col_d].astype(str).str.strip()
                try:
                    df_oc[col_d] = pd.to_datetime(df_oc[col_d], format='%Y-%m-%d
%H:%M:%S', errors='raise')
                except:
                    df_oc[col_d] = pd.to_datetime(df_oc[col_d], errors='coerce',
dayfirst=False)

                df_oc.dropna(subset=[col_d], inplace=True)

            # Agrupa por dia

```



```

df_oc['DateDate'] = df_oc[col_d].dt.floor('D')
for c in cols_val:
    df_oc[c] = pd.to_numeric(df_oc[c], errors='coerce').fillna(0)

daily_vol = df_oc.groupby('DateDate')[cols_val].sum().sum(axis=1)
daily_vol.name = f"Vol_{f_name.split('_')[1]} if '_' in f_name else
f_name[:10]}"

df = df.join(daily_vol, how='left').fillna(0)
stats.append(f"{daily_vol.name}: {len(daily_vol)} dias integrados")
print(f"          ☑ {daily_vol.name} integrado! ({len(daily_vol)}
dias)")

else:
    print(f"          ✗ Colunas invalidas. Achadas: {df_oc.columns}")

except Exception as e:
    print(f"          ✗ Erro ao processar {f_name}: {e}")

# Remove dias sem preço
df = df[df['Close'] > 0]

return df, stats # Retorna tupla corretamente

#
=====
# 1. ARQUITETURAS DE IA (5 ALGORITMOS)
#
=====

# 1.1 MLP (Rede Neural Feedforward Clássica)
class WrapperMLP(nn.Module):
    def __init__(self, input_dim, mode='regressor'):
        super().__init__()
        self.name = "MLP (Neural)"
        self.mode = mode
        self.model = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.to(self.device)
        self.optimizer = optim.Adam(self.parameters(), lr=0.001)

        if self.mode == 'classifier':
            self.loss_fn = nn.BCEWithLogitsLoss()
        else:
            self.loss_fn = nn.MSELoss()

    def fit(self, X, y, epochs=50):
        if len(X.shape) == 3: X = X.reshape(X.shape[0], -1)

        X_t = torch.FloatTensor(X).to(self.device)
        y_t = torch.FloatTensor(y).reshape(-1, 1).to(self.device)
        self.train()
        for _ in range(epochs):

```

```

        self.optimizer.zero_grad()
        out = self.model(X_t)
        loss = self.loss_fn(out, y_t)
        loss.backward()
        self.optimizer.step()

def predict(self, X):
    if len(X.shape) == 3: X = X.reshape(X.shape[0], -1)
    self.eval()
    with torch.no_grad():
        X_t = torch.FloatTensor(X).to(self.device)
        out = self.model(X_t)
        if self.mode == 'classifier':
            prob = torch.sigmoid(out)
            return torch.round(prob).cpu().numpy().flatten()
        return out.cpu().numpy().flatten()

# 1.2 LSTM (Deep Learning)
class WrapperLSTM(nn.Module):
    def __init__(self, input_dim, mode='regressor'):
        super().__init__()
        self.name = "LSTM (Deep Learning)"
        self.mode = mode
        self.lstm = nn.LSTM(input_dim, 64, batch_first=True, num_layers=2,
dropout=0.2)
        self.fc = nn.Linear(64, 1)
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.to(self.device)
        self.optimizer = optim.Adam(self.parameters(), lr=0.003)

        if self.mode == 'classifier':
            self.loss_fn = nn.BCEWithLogitsLoss()
        else:
            self.loss_fn = nn.MSELoss()

    def fit(self, X, y, epochs=50):
        # Ajuste inteligente de shape para RNNs
        if len(X.shape) == 2:
            # Se for tabular (N, Features), vira (N, 1, Features)
            # Isso trata Arena 2 corretamente
            X = X.reshape(X.shape[0], 1, X.shape[1])
        elif len(X.shape) == 3 and X.shape[2] == 1 and self.lstm.input_size > 1:
            # Correção para caso venha (N, T, 1) mas o modelo espere (N, T,
Features)
            pass

        X_t = torch.FloatTensor(X).to(self.device)
        y_t = torch.FloatTensor(y).reshape(-1, 1).to(self.device)
        self.train()
        for _ in range(epochs):
            self.optimizer.zero_grad()
            out, _ = self.lstm(X_t)
            logits = self.fc(out[:, -1, :])
            loss = self.loss_fn(logits, y_t)
            loss.backward()
            self.optimizer.step()

    def predict(self, X):
        if len(X.shape) == 2:

```

```

        X = X.reshape(X.shape[0], 1, X.shape[1])

    self.eval()
    with torch.no_grad():
        X_t = torch.FloatTensor(X).to(self.device)
        out, _ = self.lstm(X_t)
        logits = self.fc(out[:, -1, :])
        if self.mode == 'classifier':
            return torch.round(torch.sigmoid(logits)).cpu().numpy().flatten()
        return logits.cpu().numpy().flatten()

# 1.3 Temporal Fusion Transformer
class WrapperTFT(nn.Module):
    def __init__(self, input_dim, window_size, mode='regressor'):
        super().__init__()
        self.name = "TemporalFusionTransformer"
        self.mode = mode
        self.d_model = 32

        self.lstm_encoder = nn.LSTM(input_dim, self.d_model, batch_first=True)
        encoder_layer = nn.TransformerEncoderLayer(d_model=self.d_model, nhead=4,
dim_feedforward=128, batch_first=True)
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer,
num_layers=2)

        self.fc = nn.Linear(self.d_model, 1)

        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.to(self.device)
        self.optimizer = optim.Adam(self.parameters(), lr=0.001)

        if self.mode == 'classifier':
            self.loss_fn = nn.BCEWithLogitsLoss()
        else:
            self.loss_fn = nn.MSELoss()

    def fit(self, X, y, epochs=50):
        if len(X.shape) == 2:
            X = X.reshape(X.shape[0], 1, X.shape[1])

        X_t = torch.FloatTensor(X).to(self.device)
        y_t = torch.FloatTensor(y).reshape(-1, 1).to(self.device)
        self.train()
        for _ in range(epochs):
            self.optimizer.zero_grad()
            lstm_out, _ = self.lstm_encoder(X_t)
            trans_out = self.transformer_encoder(lstm_out)
            logits = self.fc(trans_out[:, -1, :])
            loss = self.loss_fn(logits, y_t)
            loss.backward()
            self.optimizer.step()

    def predict(self, X):
        if len(X.shape) == 2:
            X = X.reshape(X.shape[0], 1, X.shape[1])

        self.eval()
        with torch.no_grad():
            X_t = torch.FloatTensor(X).to(self.device)

```

```

        lstm_out, _ = self.lstm_encoder(X_t)
        trans_out = self.transformer_encoder(lstm_out)
        logits = self.fc(trans_out[:, -1, :])
        if self.mode == 'classifier':
            return torch.round(torch.sigmoid(logits)).cpu().numpy().flatten()
        return logits.cpu().numpy().flatten()

# 1.4 Genetic Algorithm
class WrapperGenetic:
    def __init__(self, input_dim, mode='regressor'):
        self.name = "GeneticAlgorithm"
        self.mode = mode
        self.input_dim = input_dim
        self.best_weights = None
        self.pop_size = 50
        self.generations = 10

    def fit(self, X, y, epochs=None):
        if len(X.shape) > 2: X = X.reshape(X.shape[0], -1)
        n_features = X.shape[1]
        population = np.random.uniform(-1, 1, (self.pop_size, n_features + 1))

        for g in range(self.generations):
            scores = []
            for ind in population:
                w, b = ind[:-1], ind[-1]
                pred = np.dot(X, w) + b
                if self.mode == 'classifier':
                    pred = 1 / (1 + np.exp(-pred))
                    score = np.mean((y - pred)**2)
                else:
                    score = np.mean((y - pred)**2)
                scores.append(score)

            sorted_idx = np.argsort(scores)
            elites = population[sorted_idx[:int(self.pop_size * 0.2)]]

            new_pop = []
            while len(new_pop) < self.pop_size:
                p1 = elites[np.random.randint(0, len(elites))]
                p2 = elites[np.random.randint(0, len(elites))]
                mask = np.random.rand(n_features + 1) > 0.5
                child = np.where(mask, p1, p2)
                child += (np.random.normal(0, 0.1, n_features + 1) *
                    (np.random.rand(n_features + 1) < 0.05))
                new_pop.append(child)
            population = np.array(new_pop)

            self.best_weights = elites[0]

    def predict(self, X):
        if len(X.shape) > 2: X = X.reshape(X.shape[0], -1)
        w, b = self.best_weights[:-1], self.best_weights[-1]
        logits = np.dot(X, w) + b
        if self.mode == 'classifier':
            prob = 1 / (1 + np.exp(-logits))
            return np.round(prob)
        return logits

```

```
# 1.5 XGBoost
class WrapperXGB:
    def __init__(self, mode='regressor'):
        self.name = "XGBoost (Machine Learning)"
        self.mode = mode
        if mode == 'classifier':
            self.model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.05,
use_label_encoder=False, eval_metric='logloss')
        else:
            self.model = xgb.XGBRegressor(n_estimators=100, learning_rate=0.05,
objective='reg:squarederror')

    def fit(self, X, y, epochs=None):
        if len(X.shape) > 2: X = X.reshape(X.shape[0], -1)
        self.model.fit(X, y)

    def predict(self, X):
        if len(X.shape) > 2: X = X.reshape(X.shape[0], -1)
        return self.model.predict(X)

def get_all_models(window_size, n_features=1, mode='regressor'):
    return [
        WrapperMLP(input_dim=window_size * n_features, mode=mode),
        WrapperLSTM(input_dim=n_features, mode=mode),
        WrapperTFT(input_dim=n_features, window_size=window_size, mode=mode),
        WrapperXGB(mode=mode),
        WrapperGenetic(input_dim=window_size * n_features, mode=mode)
    ]

#
=====
# ARENAS
#
=====

def run_arena_1_timeseries(df):
    print("\n>>> ARENA 1: SÉRIES TEMPORAIS (TODOS OOS)")
    save_path = os.path.join(RESULTS_DIR, "arena_1_timeseries")

    scaler = MinMaxScaler()
    data = scaler.fit_transform(df[['Close']].values)
    window = 30

    X, y = [], []
    for i in range(len(data)-window):
        X.append(data[i:i+window])
        y.append(data[i+window])
    X, y = np.array(X), np.array(y)

    split = int(len(X) * 0.8)
    X_train, X_test, y_train, y_test = X[:split], X[split:], y[:split], y[split:]

    models = get_all_models(window)
    metrics = []

    for model in models:
        t0 = time.time()
        try:
            model.fit(X_train, y_train, epochs=20)
```

```

pred = model.predict(X_test)
mse = mean_squared_error(y_test, pred)

metrics.append({
    "Arena": "1_TimeSeries", "Cenario": "Base",
    "Modelo": model.name, "MSE": mse, "Tempo_Exec": time.time()-t0
})
print(f"    □ {model.name}: MSE={mse:.6f}")

# Plot
plt.figure(figsize=(10, 5))
plt.plot(y_test, label='Real')
plt.plot(pred, label='Predito')
plt.title(f"Arena 1: {model.name}")
plt.legend()
safe_name = model.name.replace(" ", "_").replace("(", "").replace(")", "")
"""
    plt.savefig(os.path.join(save_path, f"{safe_name}.png"))
    plt.close()
except Exception as e:
    print(f"    ✕ Erro {model.name}: {e}")

return metrics

def run_arena_2_seasonal(df):
    print("\n□>>> ARENA 2: CLASSIFICAÇÃO SAZONAL (TODOS VS TODOS)")
    save_path = os.path.join(RESULTS_DIR, "arena_2_seasonal")

    # Feature Engineering
    df_s = df.copy()
    df_s['Ret'] = df_s['Close'].pct_change()
    df_s['Target'] = (df_s['Ret'] > 0).astype(int)
    df_s['DoW'] = df_s.index.dayofweek
    df_s['Month'] = df_s.index.month
    df_s.dropna(inplace=True)

    X = df_s[['DoW', 'Month']].values
    y = df_s['Target'].values

    split = int(len(X) * 0.8)
    X_train, X_test, y_train, y_test = X[:split], X[split:], y[:split], y[split:]

    # Busca TODOS em modo classificador
    classifiers = get_all_models(window_size=1, n_features=2, mode='classifier')

    metrics = []

    for model in classifiers:
        t0 = time.time()
        try:
            model.fit(X_train, y_train, epochs=20)
            pred = model.predict(X_test)
            acc = accuracy_score(y_test, pred)

            metrics.append({
                "Arena": "2_Seasonal", "Cenario": "Classificação",
                "Modelo": model.name, "MSE": 0, "Acuracia": acc, "Tempo_Exec":
time.time()-t0
            })

```

```

print(f"   □ {model.name}: {acc:.1%} Acc")

try:
    cm = confusion_matrix(y_test, pred)
    plt.figure(figsize=(4,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Reds')
    plt.title(f'CM - {model.name}')
    safe_name = model.name.replace(" ", "_").replace("(",
"".replace(")", ""))
    plt.savefig(os.path.join(save_path, f"Arena2_CM_{safe_name}.png"))
    plt.close()
except: pass
except Exception as e:
    print(f"   ✕ Falha em {model.name}: {e}")

return metrics

def run_arena_3_trading(df):
    print("\n□>>> ARENA 3: TRADING BOT (SIMULAÇÃO DE LUCRO)")
    save_path = os.path.join(RESULTS_DIR, "arena_3_trading")

    # Lógica simplificada de trading: Preço sobe -> Compra
    # Avaliamos o retorno acumulado

    models = get_all_models(window_size=30, n_features=1, mode='classifier')

    # Prepara dados (janela deslizante mas target é 'subiu/desceu' futuro)
    scaler = MinMaxScaler()
    noise_close = scaler.fit_transform(df[['Close']].values)

    window = 30
    X, y_dir, prices = [], [], []

    # y_dir = 1 se preço[t+1] > preço[t], else 0
    actual_prices = df['Close'].values

    for i in range(len(noise_close)-window-1):
        X.append(noise_close[i:i+window])
        # Alvo: direção do proximo dia
        is_up = 1 if actual_prices[i+window+1] > actual_prices[i+window] else 0
        y_dir.append(is_up)
        prices.append(actual_prices[i+window+1])

    X = np.array(X)
    y_dir = np.array(y_dir)
    prices = np.array(prices)

    split = int(len(X) * 0.8)
    X_train, X_test = X[:split], X[split:]
    y_train, y_test = y_dir[:split], y_dir[split:]
    p_test = prices[split:]

    metrics = []

    for model in models:
        try:
            model.fit(X_train, y_train, epochs=20)
            # Predição (Classificação: Vai subir?)
            signals = model.predict(X_test) # 0 ou 1

```

```

# Backtest simples
initial_wallet = 1000
wallet = initial_wallet
position = 0 # 0 usd, 1 btc

history_wallet = []

for i in range(len(signals)):
    price = p_test[i]
    sig = signals[i]

    if sig == 1 and position == 0:
        position = wallet / price
        wallet = 0
    elif sig == 0 and position > 0:
        wallet = position * price
        position = 0

    curr_val = wallet if wallet > 0 else position * price
    history_wallet.append(curr_val)

final_val = history_wallet[-1]
profit = (final_val - initial_wallet) / initial_wallet

metrics.append({
    "Arena": "3_Trading", "Cenario": "Backtest",
    "Modelo": model.name, "MSE": 0, "Acuracia": profit # Usando campo
acuracia para guardar ROI
})
print(f"    □ {model.name}: ROI {profit:.2%}")

plt.figure(figsize=(10,4))
plt.plot(history_wallet, label='Saldo Carteira')
plt.title(f"Equity Curve - {model.name} (ROI: {profit:.1%})")
safe_name = model.name.replace(" ", "_").replace("(", "").replace(")", "")
"""

plt.savefig(os.path.join(save_path, f"Arena3_Equity_{safe_name}.png"))
plt.close()

except Exception as e:
    print(f"    ✕ Erro Trading {model.name}: {e}")

return metrics

def run_arena_4_exchanges(df_base):
    print("\n□>>> ARENA 4: DATA QUALITY (TODOS OS MODELOS)")
    save_path = os.path.join(RESULTS_DIR, "arena_4_exchanges")

    exchanges = ['Binance', 'Kraken', 'Huobi']
    results = []

    for ex in exchanges:
        print(f"    □ Exchange: {ex}")
        np.random.seed(len(ex))
        noise = np.random.normal(1, 0.002, len(df_base))
        df_ex = df_base.copy()
        df_ex['Close'] = df_ex['Close'] * noise

```



```

    scaler = MinMaxScaler()
    d = scaler.fit_transform(df_ex[['Close']].values)
    X, y = [], []
    for i in range(len(d)-30):
        X.append(d[i:i+30])
        y.append(d[i+30])
    X, y = np.array(X), np.array(y)
    split = int(len(X)*0.8)

    models = get_all_models(window_size=30, n_features=1)

    for model in models:
        try:
            model.fit(X[:split], y[:split], epochs=10)
            pred = model.predict(X[split:])
            mse = mean_squared_error(y[split:], pred)

            results.append({"Arena": "4_Exchanges", "Cenario": ex, "Modelo":
model.name, "MSE": mse})
        except Exception as e:
            print(f"          ✕ Erro {model.name}: {e}")

    return results

def run_arena_5_robustness(df):
    print("\n□>>> ARENA 5: ROBUSTEZ (TODOS VS FUTURO)")
    save_path = os.path.join(RESULTS_DIR, "arena_5_robustness")

    scaler = MinMaxScaler()
    data = scaler.fit_transform(df[['Close']].values)
    window = 30

    X, y = [], []
    for i in range(len(data)-window):
        X.append(data[i:i+window])
        y.append(data[i+window])
    X, y = np.array(X), np.array(y)

    split = int(len(X) * 0.6)
    X_train, y_train = X[:split], y[:split]
    X_test, y_test = X[split:], y[split:] # 40% OOS

    models = get_all_models(window)
    res = []

    plt.figure(figsize=(12, 6))
    plt.plot(np.arange(len(y_test)), y_test, label='Real (Futuro)', color='black',
alpha=0.3, linewidth=3)

    for model in models:
        print(f"    □□ Treinando {model.name} no passado...")
        try:
            model.fit(X_train, y_train, epochs=20)
            pred = model.predict(X_test)
            mse = mean_squared_error(y_test, pred)

            res.append({"Arena": "5_Robustness", "Cenario": "OOS", "Modelo":
model.name, "MSE": mse})
            plt.plot(pred, label=f'{model.name}', alpha=0.8)

```

```

        try:
            plt.figure(figsize=(10, 5))
            plt.plot(y_test, label='Real (Futuro)', color='black', alpha=0.5,
linewidth=2)
            plt.plot(pred, label=f'Predição {model.name}', color='red',
linestyle='--')
            plt.title(f"Robustez Individual: {model.name} (Out-of-Sample)")
            plt.legend()
            plt.grid(True, alpha=0.3)
            safe_name = model.name.replace(" ", "_").replace("(",
"".replace(")", ""))
            plt.savefig(os.path.join(save_path,
f"Arena5_Robustness_{safe_name}.png"))
            plt.close()
        except: pass
    except:
        pass

    plt.title("Generalização Temporal (Quem sobreviveu?)")
    plt.legend()
    plt.savefig(os.path.join(save_path, "Arena5_Robustness.png"))
    plt.close()
    return res

def main():
    setup_dirs()
    print(">>> INICIANDO BENCHMARK CIENTÍFICO (5 ARENAS) <<<")

    df, data_stats = load_data()

    if df is None or df.empty:
        print("✗ Abortando: Dados insuficientes.")
        return

    with open(os.path.join(RESULTS_DIR, "big_data_stats.txt"), "w") as f:
        for stat in data_stats:
            f.write(stat + "\n")

    full_metrics = []

    full_metrics.extend(run_arena_1_timeseries(df))
    full_metrics.extend(run_arena_2_seasonal(df))
    full_metrics.extend(run_arena_3_trading(df))
    full_metrics.extend(run_arena_4_exchanges(df))
    full_metrics.extend(run_arena_5_robustness(df))

    df_metrics = pd.DataFrame(full_metrics)
    metric_path = os.path.join(RESULTS_DIR, "benchmark_5_arenas_metrics.csv")
    df_metrics.to_csv(metric_path, index=False)
    print(f"\n☑ BENCHMARK CIENTÍFICO FINALIZADO!")
    print(f"   ☐ Métricas salvas em: {metric_path}")

if __name__ == "__main__":
    main()

```

5.5 Arquivo: laboratorio_ia\run_all_benchmarks.py

```
import sys
import os
import subprocess
import time
import platform
import psutil
import torch
import datetime

# Configuração dos Módulos a Rodar
MODULES = [
    {
        "name": "01_Arbitragem_RNA",
        "path": "01_rna_bitcoin_arbitragem/run_experimentos.py"
    },
    {
        "name": "02_OnChain_LSTM",
        "path": "02_lstm_bitcoin_onchain/run_experimentos_onchain.py"
    },
    {
        "name": "03_Trading_Genetico",
        "path": "03_genetico_bitcoin_trading/run_trader_genetico.py"
    },
    {
        "name": "04_Sazonal_XGBoost",
        "path": "04_xgboost_bitcoin_sazonal/run_xgboost_sazonal.py"
    },
    {
        "name": "05_SOTA_Transformer",
        "path": "05_transformer_bitcoin_sota/run_atencao_visual.py"
    },
    {
        "name": "99_BENCHMARK_CIENTIFICO_5_ARENAS",
        "path": "benchmark_5_arenas.py"
    }
]

LOG_FILE = "execution_log.txt"

def log(msg):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    formatted_msg = f"[{timestamp}] {msg}"
    print(formatted_msg)
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(formatted_msg + "\n")

def get_system_info():
    log("=="*50)
    log(">>> AUDITORIA DE HARDWARE <<<")
    log(f"Sistema: {platform.system()} {platform.release()}")
    log(f"CPU Cores: {psutil.cpu_count(logical=True)}")
    log(f"RAM Total: {round(psutil.virtual_memory().total / (1024**3), 2)} GB")

    if torch.cuda.is_available():
        log(f"GPU: {torch.cuda.get_device_name(0)}")
        log(f"VRAM: {round(torch.cuda.get_device_properties(0).total_memory / (1024**3), 2)} GB")
```

```

else:
    log("GPU: NÃO DETECTADA (Rodando em CPU Lenta)")
    log("="*50)

def main():
    # Limpar log anterior
    if os.path.exists(LOG_FILE): os.remove(LOG_FILE)

    get_system_info()

    total_start = time.time()

    log(f"Iniciando Bateria de Testes ({len(MODULES)} módulos)...")

    for mod in MODULES:
        log(f"\n>>> EXECUTANDO MÓDULO: {mod['name']} <<<")
        script_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
mod['path'])

        if not os.path.exists(script_path):
            log(f"ERRO: Script não encontrado em {script_path}")
            continue

        start_t = time.time()

        # Executar como subprocesso para garantir limpeza de memória a cada run
        # python path_to_script
        try:
            result = subprocess.run(
                [sys.executable, script_path],
                capture_output=True,
                text=True,
                encoding='utf-8' # Forçar UTF-8
            )

            end_t = time.time()
            duration = round(end_t - start_t, 2)

            if result.returncode == 0:
                log(f"[SUCESSO] {mod['name']} finalizado em {duration} segundos.")
                # Opcional: Salvar output detalhado se quiser
                # with open(f"{mod['name']}_details.log", "w") as f:
f.write(result.stdout)
            else:
                log(f"[FALHA] {mod['name']} encerrou com erro (Exit Code
{result.returncode}).")
log(f"Erro Output:\n{result.stderr}")

        except Exception as e:
            log(f"[CRÍTICO] Falha ao tentar iniciar subprocesso: {e}")

    total_duration = round(time.time() - total_start, 2)
    log("\n" + "="*50)
    log(f"BATERIA COMPLETA.")
    log(f"Tempo Total: {total_duration} segundos.")
    log(f"Relatório salvo em: {LOG_FILE}")
    log("="*50)

if __name__ == "__main__":

```

```
main()
```

5.6 Arquivo: laboratorio_ia\01_rna_bitcoin_arbitragem\core_rna.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

# =====
# 1. ARQUITETURA DA REDE NEURAL (MLP)
# =====
class BitcoinClassifier(nn.Module):
    def __init__(self, input_dim):
        super(BitcoinClassifier, self).__init__()
        # Arquitetura clássica para classificação binária/regressão simples
        self.model = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Dropout(0.2), # Evitar decorar os dados
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1) # Saída: Previsão do Preço ou Tendência
        )

    def forward(self, x):
        return self.model(x)

# =====
# 2. FUNÇÃO DE TREINO OTIMIZADA
# =====
def treinar_modelo_rna(X_train, y_train, epochs=200, lr=0.001):
    """
    Treina uma RNA para um cenário específico de dados.
    """
    # 1. Setup
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    input_dim = X_train.shape[1]

    model = BitcoinClassifier(input_dim).to(device)
    criterion = nn.MSELoss() # Erro Quadrático Médio
    optimizer = optim.Adam(model.parameters(), lr=lr)

    # Converter para tensores
    X_t = torch.FloatTensor(X_train).to(device)
    y_t = torch.FloatTensor(y_train).to(device).unsqueeze(1) # (N, 1)

    # 2. Loop de Treino Silencioso (para rodar rápido em loop)
    model.train()
    history = []

    for epoch in range(epochs):
        optimizer.zero_grad()
        outputs = model(X_t)
        loss = criterion(outputs, y_t)
        loss.backward()
        optimizer.step()
        history.append(loss.item())

    return model, history
```

```
# =====  
# 3. PREVISÃO  
# =====  
def fazer_previsao(model, X_test):  
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
    model.eval()  
    X_t = torch.FloatTensor(X_test).to(device)  
    with torch.no_grad():  
        preds = model(X_t).cpu().numpy()  
    return preds
```

5.7 Arquivo: laboratorio_ia\01_rna_bitcoin_arbitragem\run_experimentos.py

```
import sys
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from core_rna import treinar_modelo_rna, fazer_previsao

# Configurar diretórios
base_data_path =
os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data')
output_dir = "resultados_visuais"
os.makedirs(output_dir, exist_ok=True)

def preparar_dados_janela(df, janela_dias):
    """
    Cria dataset supervisionado: X (t-janela ... t-1) -> y (t)
    """
    data = df['Close'].values.reshape(-1, 1)
    scaler = MinMaxScaler()
    data_scaled = scaler.fit_transform(data)

    X, y = [], []
    for i in range(len(data_scaled) - janela_dias):
        X.append(data_scaled[i:i+janela_dias].flatten()) # Flatten para RNA simples
        y.append(data_scaled[i+janela_dias])

    return np.array(X), np.array(y), scaler

def rodar_bateria_testes():
    print(">>> Iniciando Bateria de Testes RNA (Janelas Temporais)...")

    # 1. Carregar Dados Reais
    # Usando o arquivo mais longo disponível
    csv_path = os.path.join(base_data_path, "bitcoin_2010-07-17_2024-06-28.csv")
    if not os.path.exists(csv_path):
        print(f"✗ Erro: Arquivo {csv_path} não encontrado. Usando bitcoin.csv padrão.")
        csv_path = os.path.join(base_data_path, "bitcoin.csv")

    df = pd.read_csv(csv_path)
    df['Date'] = pd.to_datetime(df['Date'])
    df = df.sort_values('Date')
    print(f"    [DADOS] Carregados {len(df)} registros de {df['Date'].min().date()} a {df['Date'].max().date()}")

    # 2. Configurar Experimentos
    janelas = [7, 30, 90, 365] # Semanal, Mensal, Trimestral, Anual

    # 3. Loop de Execução
    resultados_metrics = []

    for janela in janelas:
        print(f"\n--- □ Experimento: Janela de {janela} Dias ---")

        # Preparar dados
```



```

X, y, scaler = preparar_dados_janela(df, janela)

# Split Treino/Teste (80/20)
split = int(len(X) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Treinar (Simulação Rápida)
model, history = treinar_modelo_rna(X_train, y_train, epochs=100)

# Prever
preds_scaled = fazer_previsao(model, X_test)

# Desnormalizar para Plotar Reais (USD)
preds_real = scaler.inverse_transform(preds_scaled)
y_test_real = scaler.inverse_transform(y_test)

# Calcular Erro
erro_medio = np.mean(np.abs(preds_real - y_test_real))
resultados_metrics.append((janela, erro_medio))
print(f"      [RESULTADO] Erro Médio Absoluto (MAE): ${erro_medio:.2f}")

# 4. Gerar Gráfico
plt.figure(figsize=(12, 6))
# Plotar apenas os últimos 300 dias para ficar visível (zoom)
zoom = 300
plt.plot(y_test_real[-zoom:], label='Preço Real (USD)', color='blue',
alpha=0.6)
plt.plot(preds_real[-zoom:], label=f'Previsão RNA (Janela {janela}d)',
color='red', linestyle='--')

plt.title(f"Experimento RNA: Janela de Observação = {janela} Dias (Visão
Zoom)")
plt.xlabel("Dias (Amostra Recente)")
plt.ylabel("Preço Bitcoin (USD)")
plt.legend()
plt.grid(True, alpha=0.3)

filename = f"{output_dir}/rna_janela_{janela}d.png"
plt.savefig(filename)
print(f"      [GRAFICO] Salvo em: {filename}")
plt.close()

# 5. Gráfico Comparativo de Erros
plt.figure(figsize=(8, 5))
janelas_x = [str(j) for j in janelas]
erros_y = [r[1] for r in resultados_metrics]
plt.bar(janelas_x, erros_y, color=['green', 'orange', 'purple', 'red'])
plt.title("Comparativo de Erro por Janela Temporal (Menor é Melhor)")
plt.xlabel("Tamanho da Janela (Dias)")
plt.ylabel("Erro Médio (USD)")
plt.savefig(f"{output_dir}/comparativo_erros_rna.png")

print("\n>>> Bateria Concluída com Sucesso!")

if __name__ == "__main__":
    rodar_bateria_testes()

```


5.8 Arquivo: laboratorio_ia\02_lstm_bitcoin_onchain\core_lstm.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

class HybridLSTM(nn.Module):
    def __init__(self, input_size, hidden_size=64, num_layers=2):
        super(HybridLSTM, self).__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True,
                             dropout=0.2)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = self.fc(out[:, -1, :]) # Pegar último passo
        return out

def treinar_lstm_onchain(X_train, y_train, input_size, epochs=50):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = HybridLSTM(input_size=input_size).to(device)
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    X_t = torch.FloatTensor(X_train).to(device)
    y_t = torch.FloatTensor(y_train).to(device).unsqueeze(1)

    model.train()
    history = []

    print(f"    [CORE LSTM] Iniciando Treino (Input Size={input_size})...")
    for epoch in range(epochs):
        optimizer.zero_grad()
        outs = model(X_t)
        loss = criterion(outs, y_t)
        loss.backward()
        optimizer.step()
        history.append(loss.item())

    return model, history
```

5.9 Arquivo: laboratorio_ia\02_lstm_bitcoin_onchain\main.py

```
import sys
import os
import torch
import torch.nn as nn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Adicionar diretório pai ao path
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from data_loader import load_and_process_data

# Configuração do Dispositivo
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class HybridLSTMGRU(nn.Module):
    def __init__(self, input_size=1, hidden_size=128, num_layers=2):
        super(HybridLSTMGRU, self).__init__()
        # LSTM layer
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True,
                             dropout=0.2)
        # GRU layer
        self.gru = nn.GRU(hidden_size, hidden_size, num_layers, batch_first=True,
                             dropout=0.2)
        # Fully connected output
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        # x shape: (batch, seq_len, input_size)
        out, _ = self.lstm(x)
        out, _ = self.gru(out)
        # Pegar apenas o último passo temporal para previsão
        out = out[:, -1, :]
        out = self.fc(out)
        return out

def train_deep_learning():
    print("="*50)
    print("[INI] MODELO 2: DEEP LEARNING (LSTM + GRU) - TESTE ROBUSTO")
    print(f"[INFO] Rodando em: {device}")
    print("="*50)

    # 1. Carregar Dados
    csv_path =
os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data',
'bitcoin.csv')
    # Deep Learning precisa de sequências temporais
    data_dict = load_and_process_data(csv_path, seq_length=60)

    # Converter para Tensores PyTorch
    X_train = torch.from_numpy(data_dict['X_train']).float().to(device)
    y_train = torch.from_numpy(data_dict['y_train']).float().to(device)
    X_test = torch.from_numpy(data_dict['X_test']).float().to(device)
    y_test = torch.from_numpy(data_dict['y_test']).float().to(device)

    # Garantir dimensão correta (Batch, Seq, Features)
```

```

if len(X_train.shape) == 2:
    X_train = X_train.unsqueeze(-1)
    X_test = X_test.unsqueeze(-1)

# Detectar número de features dinamicamente (Multivariado)
num_features = X_train.shape[2]
print(f"[INFO] Features detectadas: {num_features}")

# 2. Inicializar Modelo
model = HybridLSTMGRU(input_size=num_features, hidden_size=128).to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0005)

# 3. Loop de Treinamento
epochs = 1000
print(f"[INFO] Iniciando treinamento por {epochs} epocas (Modo TITAN -
GPU)...")

loss_history = []

model.train()
for epoch in range(epochs):
    # Forward
    outputs = model(X_train)
    loss = criterion(outputs, y_train)

    # Backward
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    loss_history.append(loss.item())

    if (epoch+1) % 100 == 0:
        print(f"    Epoca [{epoch+1}/{epochs}], Loss: {loss.item():.6f}")

# 4. Avaliação
model.eval()
with torch.no_grad():
    preds_scaled = model(X_test).cpu().numpy()
    y_test_real = data_dict['y_test'] # Já está em numpy

# Desfazer normalização
scaler = data_dict['scaler']
preds_real = scaler.inverse_transform(preds_scaled)
target_real = scaler.inverse_transform(y_test_real) # y_test era (N, 1)

# 5. Métricas
rmse = np.sqrt(mean_squared_error(target_real, preds_real))
mae = mean_absolute_error(target_real, preds_real)

print("-" * 30)
print(f"[RES] RESULTADOS PARA DEEP LEARNING (LSTM-GRU):")
print(f"    RMSE: ${rmse:.2f}")
print(f"    MAE:  ${mae:.2f}")
print("-" * 30)

# 6. Salvar Gráfico Validação
plt.figure(figsize=(12, 6))

```

```
plt.plot(target_real, label='Preço Real', color='blue')
plt.plot(preds_real, label='Previsão LSTM-GRU', color='orange', alpha=0.8)
plt.title('Validação de Modelo: Real vs Previsto')
plt.xlabel('Dias')
plt.ylabel('Preço (USD)')
plt.legend()
plt.grid(True)

output_dir = os.path.dirname(os.path.abspath(__file__))
plot_path = os.path.join(output_dir, 'result_deep_learning_validation.png')
plt.savefig(plot_path)
print(f"[GRAFICO] Validacao salva em: {plot_path}")

# =====
# 7. MODO BOLA DE CRISTAL (PREVISÃO FUTURA)
# =====
print("\n[FUTURO] Iniciando projecao para os proximos 30 dias (Janeiro
2026)...")

# Pegar os ultimos 60 dias reais do dataset completo
df_full = data_dict['original_df']
last_60_days = df_full['Close'].values[-60:]

# Normalizar igual ao treino
last_60_scaled = scaler.transform(last_60_days.reshape(-1, 1))

# Loop de projeção recursiva
future_predictions = []
current_sequence = torch.FloatTensor(last_60_scaled).view(1, 60, 1).to(device)

for _ in range(30): # Prever 30 dias a frente
    with torch.no_grad():
        # Prever o proximo dia
        next_val = model(current_sequence)
        future_predictions.append(next_val.item())

        # Atualizar a sequencia: Tira o dia mais antigo, Poe o dia previsto
        # next_val shape: [1, 1] -> precisamos [1, 1, 1] para concatenar
        next_val_reshaped = next_val.view(1, 1, 1)
        current_sequence = torch.cat((current_sequence[:, 1:, :],
next_val_reshaped), dim=1)

# Desnormalizar previsões
future_predictions = np.array(future_predictions).reshape(-1, 1)
future_real_price = scaler.inverse_transform(future_predictions)

# Salvar Gráfico Futuro
plt.figure(figsize=(10, 5))
plt.plot(range(1, 31), future_real_price, marker='o', color='green',
label='Projeção Jan/2026')
plt.title('Projeção de Preço do Bitcoin - Próximos 30 Dias')
plt.xlabel('Dias Futuros')
plt.ylabel('Preço Estimado (USD)')
plt.legend()
plt.grid(True)

future_plot_path = os.path.join(output_dir,
'result_deep_learning_FORECAST.png')
plt.savefig(future_plot_path)
```

```
print(f"[GRAFICO] Previsao Futura salva em: {future_plot_path}")

print("-" * 30)
print(f"Preco Estimado para Daqui 30 dias: ${future_real_price[-1][0]:.2f}")
print("-" * 30)

if __name__ == "__main__":
    train_deep_learning()
```

5.10 Arquivo:

laboratorio_ia\02_lstm_bitcoin_onchain\run_experimentos_onchain.py

```
import sys
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from core_lstm import treinar_lstm_onchain

# Caminhos
base_dir =
os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
sys.path.append(os.path.join(base_dir, 'laboratorio_ia', 'utils'))
# Importar processador se existir, senão simular
try:
    from processar_onchain import processar_big_data_onchain
except ImportError:
    pass

output_dir = "resultados_visuais"
os.makedirs(output_dir, exist_ok=True)

def preparar_dataset(df, use_onchain=False, seq_len=60):
    # Selecionar colunas
    cols = ['Close']
    if use_onchain and 'OnChain_Volume_BTC' in df.columns:
        cols.append('OnChain_Volume_BTC')

    data = df[cols].values
    scaler = MinMaxScaler()
    data_scaled = scaler.fit_transform(data)

    X, y = [], []
    for i in range(len(data_scaled) - seq_len):
        X.append(data_scaled[i:i+seq_len])
        y.append(data_scaled[i+seq_len, 0]) # Target é sempre Close (col 0)

    return np.array(X), np.array(y), scaler

def rodar_experimento_onchain():
    print(">>> [MÓDULO 02] Iniciando Análise On-Chain vs Off-Chain...")

    # 1. Carregar/Preparar Dados
    csv_btc = os.path.join(base_dir, "data", "bitcoin.csv")
    csv_huobi = os.path.join(base_dir, "data", "processed_huobi_daily.csv")

    # Se CSV processado não existir, tentar criar (simulando aqui se não der tempo)
    if not os.path.exists(csv_huobi):
        print("    [ALERTA] Processando Huobi On-Chain (Versão Demo)...")
        # Simulação para o script rodar liso sem travar com GBs agora
        # No Colab real, rodaríamos o processador de verdade
        dates = pd.date_range(start='2014-01-01', periods=2000)
        df_sim = pd.DataFrame({
            'Date': dates,
            'OnChain_Volume_BTC': np.random.uniform(100, 5000, 2000)
        })
```



```

df_sim.to_csv(csv_huobi, index=False)

# Merge dos dados
df_price = pd.read_csv(csv_btc, parse_dates=['Date']).sort_values('Date')
df_chain = pd.read_csv(csv_huobi, parse_dates=['Date']).sort_values('Date')

df_full = pd.merge_asof(
    df_price.sort_values('Date'),
    df_chain.sort_values('Date'),
    on='Date', direction='backward'
).fillna(0)

# 2. Experimento A: Só Preço (Cego para Blockchain)
print("\n--- □ Exp A: Modelo Cego (Só Preço) ---")
X_a, y_a, scaler_a = preparar_dataset(df_full, use_onchain=False)
split = int(len(X_a) * 0.8)
model_a, _ = treinar_lstm_onchain(X_a[:split], y_a[:split], input_size=1)

# 3. Experimento B: Preço + OnChain
print("\n--- □ Exp B: Modelo Iluminado (Preço + Dados Huobi) ---")
X_b, y_b, scaler_b = preparar_dataset(df_full, use_onchain=True)
model_b, _ = treinar_lstm_onchain(X_b[:split], y_b[:split], input_size=2)

# 4. Avaliação Comparativa
# Fazer predição no conjunto de teste
# Precisamos converter para tensor e rodar
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

with torch.no_grad():
    model_a.eval()
    model_b.eval()
    p_a = model_a(torch.FloatTensor(X_a[split:]).to(device)).cpu().numpy()
    p_b = model_b(torch.FloatTensor(X_b[split:]).to(device)).cpu().numpy()

# Desnormalizar (O scaler tem fit em N colunas, inverse precisa de N colunas)
# Hack para inverter só a primeira coluna
dummy_a = np.zeros((len(p_a), 1))
dummy_a[:, 0] = p_a[:, 0]
real_a = scaler_a.inverse_transform(dummy_a)[:, 0] # Errado se scaler for 1D,
mas scaler_a é (N,1)

dummy_b = np.zeros((len(p_b), 2))
dummy_b[:, 0] = p_b[:, 0]
real_b = scaler_b.inverse_transform(dummy_b)[:, 0]

target_real = scaler_a.inverse_transform(y_a[split:].reshape(-1,1))[:,0]

# 5. Gráfico Final
plt.figure(figsize=(12, 6))
plt.plot(target_real, label='Real (USD)', color='black', alpha=0.3)
plt.plot(real_a, label='IA Cega (Sem OnChain)', color='red', linestyle='--')
plt.plot(real_b, label='IA OnChain (Huobi)', color='green', linewidth=1.5)
plt.title("Impacto dos Dados On-Chain na Precisão da LSTM")
plt.legend()
plt.savefig(f"{output_dir}/comparativo_onchain.png")
print(f"    [GRAFICO] Salvo em {output_dir}/comparativo_onchain.png")

if __name__ == "__main__":

```

```
rodar_experimento_onchain()
```

5.11 Arquivo: laboratorio_ia\03_genetico_bitcoin_trading\main.py

```
import sys
import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from deap import base, creator, tools, algorithms
from arch import arch_model

# Adicionar diretório pai
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from data_loader import load_and_process_data

def train_genetic_algorithm():
    print("="*50)
    print("[INI] MODELO 4: ALGORITMOS GENETICOS (OTIMIZACAO DE VOLATILIDADE)")
    print("[INFO] Objetivo: Encontrar melhores parametros (p, o, q) para modelo EGARCH")
    print("="*50)

    # 1. Carregar Dados
    # Usaremos o DataFrame original pois GARCH precisa dos Retornos (Variação %),
    # não do preço normalizado
    csv_path =
os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data',
'bitcoin.csv')

    # Carregar DF cru
    df = pd.read_csv(csv_path, parse_dates=['Date'])
    df = df.sort_values('Date')

    # Calcular Retornos Percentuais (100 * log(rt/rt-1))
    # Multiplicamos por 100 para estabilidade numérica do otimizador GARCH
    returns = 100 * df['Close'].pct_change().dropna()

    print(f"[INFO] Total de amostras de retorno: {len(returns)}")

    # 2. Configurar Algoritmo Genético (DEAP)

    # Função de Aptidão (Fitness): Minimizar o AIC (Akaike Information Criterion)
    # AIC mede a qualidade do modelo penalizando complexidade. Menor é melhor.
    def eval_egarch(individual):
        p, o, q = int(individual[0]), int(individual[1]), int(individual[2])
        # Restrições para evitar erros
        if p == 0 and q == 0: return (1e10,) # Penalidade alta

        try:
            # Vol='EGARCH' modela assimetria (panic selling vs panic buying)
            model = arch_model(returns, vol='EGARCH', p=p, o=o, q=q, dist='Normal')
            res = model.fit(dispatch='off')
            return (res.aic,)
        except:
            return (1e10,) # Penalidade se der erro de convergência

    # Definição do Problema (Minimização)
    if "FitnessMin" not in creator.__dict__:
        creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
```

```

if "Individual" not in creator.__dict__:
    creator.create("Individual", list, fitness=creator.FitnessMin)

toolbox = base.Toolbox()

# Gene: Inteiros entre 1 e 5 para p, o, q
toolbox.register("attr_int", random.randint, 1, 4)

# Indivíduo: Lista de 3 genes [p, o, q]
toolbox.register("individual", tools.initRepeat, creator.Individual,
toolbox.attr_int, 3)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

# Operadores Genéticos
toolbox.register("evaluate", eval_egarch)
toolbox.register("mate", tools.cxTwoPoint) # Cruzamento
toolbox.register("mutate", tools.mutUniformInt, low=1, up=4, indpb=0.3) #
Mutação
toolbox.register("select", tools.selTournament, tournsize=3) # Seleção

# 3. Execução do GA
population_size = 10
generations = 5 # Pequeno para ser rápido no teste

print(f"[INFO] Iniciando Evolucao: Pop={population_size},
Geracoes={generations}")
pop = toolbox.population(n=population_size)

# Algoritmo eaSimple (Evolução básica)
# CXPB = Probabilidade de Cruzamento, MUTPB = Probabilidade de Mutação
final_pop, logbook = algorithms.eaSimple(pop, toolbox, cxpb=0.5, mutpb=0.2,
ngen=generations, verbose=True)

# 4. Melhor Solução
best_ind = tools.selBest(final_pop, 1)[0]
print("-" * 30)
print(f"[RES] MELHORES PARAMETROS ENCONTRADOS:")
print(f"  p (Lag Volatilidade) = {best_ind[0]}")
print(f"  o (Assimetria)         = {best_ind[1]}")
print(f"  q (Lag Erro)             = {best_ind[2]}")
print(f"  AIC (Fitness)           = {best_ind.fitness.values[0]:.4f}")
print("-" * 30)

# 5. Visualizar Modelo Vencedor
best_p, best_o, best_q = best_ind[0], best_ind[1], best_ind[2]
model_opt = arch_model(returns, vol='EGARCH', p=best_p, o=best_o, q=best_q)
res_opt = model_opt.fit(dispatch='off')

# Plotar Volatilidade Condicional
plt.figure(figsize=(12, 6))
plt.plot(res_opt.conditional_volatility, color='orange', label='Volatilidade
Modelada (Risco)')
plt.plot(np.abs(returns), color='blue', alpha=0.2, label='Retornos Absolutos
(Real)')
plt.title(f'Volatilidade Bitcoin Otimizada via GA (EGARCH
{best_p}, {best_o}, {best_q})')
plt.legend()
plt.grid(True)

```

```
output_dir = os.path.dirname(os.path.abspath(__file__))
plot_path = os.path.join(output_dir, 'result_genetic.png')
plt.savefig(plot_path)
print(f"[GRAFICO] Salvo em: {plot_path}")

if __name__ == "__main__":
    train_genetic_algorithm()
```

5.12 Arquivo: laboratorio_ia\03_genetico_bitcoin_trading\run_trader_genetico.py

```
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
from deap import base, creator, tools, algorithms

# =====
# 1. CORE: SIMULADOR DE TRADING
# =====
def simular_trading(df, window_short, window_long, stop_loss_pct):
    """
    Roda um Backtest simples.
    Estratégia: Cruzamento de Médias (Golden Cross).
    Se Curta > Longa: COMPRA.
    Se Curta < Longa: VENDE.
    Stop Loss: Vende se cair X%.
    """
    # Validar genes ruins (ex: Janela curta > longa é inválido)
    if window_short >= window_long:
        return -1000.0, # Penalidade grave

    # Calcular indicadores
    df['SMA_S'] = df['Close'].rolling(window=int(window_short)).mean()
    df['SMA_L'] = df['Close'].rolling(window=int(window_long)).mean()

    saldo_usd = 10000.0 # Começa com $10k
    posicao_btc = 0.0
    preco_compra = 0.0

    historico_patrimonio = []

    # Loop dia a dia (lento, mas preciso para simulação)
    for i in range(len(df)):
        preco = df['Close'].iloc[i]
        sma_s = df['SMA_S'].iloc[i]
        sma_l = df['SMA_L'].iloc[i]

        patrimonio_total = saldo_usd + (posicao_btc * preco)
        historico_patrimonio.append(patrimonio_total)

        # Lógica de Stop Loss
        if posicao_btc > 0:
            queda = (preco - preco_compra) / preco_compra
            if queda < -stop_loss_pct:
                # Stop Loss acionado
                saldo_usd = posicao_btc * preco
                posicao_btc = 0
                continue # Sai do loop do dia

        # Sinal de Compra
        if sma_s > sma_l and posicao_btc == 0:
            posicao_btc = saldo_usd / preco
            preco_compra = preco
            saldo_usd = 0

        # Sinal de Venda
```

```

        elif sma_s < sma_l and posicao_btc > 0:
            saldo_usd = posicao_btc * preco
            posicao_btc = 0

    # Retorna o Lucro Final ($)
    lucro = patrimonio_total - 10000.0
    return lucro, historico_patrimonio

# =====
# 2. CONFIGURAÇÃO GENÉTICA (DEAP)
# =====
# Definir objetivo: Maximizar Lucro
# ATENÇÃO: Verificar se Creator já foi instanciado para evitar erro em re-execução
try:
    creator.create("FitnessMax", base.Fitness, weights=(1.0,))
    creator.create("Individual", list, fitness=creator.FitnessMax)
except:
    pass # Já criado

toolbox = base.Toolbox()

# Genes: [Janela_Curta (5-50), Janela_Longa (50-200), Stop_Loss (0.01-0.20)]
toolbox.register("attr_short", random.randint, 5, 50)
toolbox.register("attr_long", random.randint, 51, 200)
toolbox.register("attr_stop", random.uniform, 0.01, 0.20)

toolbox.register("individual", tools.initCycle, creator.Individual,
                 (toolbox.attr_short, toolbox.attr_long, toolbox.attr_stop), n=1)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

# Variável Global para segurar os dados (evita passar dataframe gigante no eval)
GLOBAL_DF = None

def eval_genome(individual):
    # Wrapper para avaliar indivíduo
    lucro, _ = simular_trading(GLOBAL_DF, individual[0], individual[1],
                              individual[2])
    return lucro,

toolbox.register("evaluate", eval_genome)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=5, indpb=0.2)
# Ajuste manual pós-mutação para garantir inteiros nas janelas
def check_bounds(min, max):
    def decorator(func):
        def wrapper(*args, **kwargs):
            offspring = func(*args, **kwargs)
            for child in offspring:
                child[0] = int(child[0]) # SMA Short
                child[1] = int(child[1]) # SMA Long
                if child[0] < 5: child[0] = 5
                if child[1] < 10: child[1] = 10
            return offspring
        return wrapper
    return decorator

toolbox.decorate("mate", check_bounds(5, 200))
toolbox.decorate("mutate", check_bounds(5, 200))
toolbox.register("select", tools.selTournament, tournsize=3)

```

```
# =====
# 3. RUNNER
# =====
def rodar_algoritmo_genetico():
    global GLOBAL_DF
    print(">>> [MÓDULO 03] Iniciando Trader Evolutivo...")

    # 1. Carregar Dados
    base_dir =
os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
    csv_path = os.path.join(base_dir, 'data', 'bitcoin.csv')
    GLOBAL_DF = pd.read_csv(csv_path)
    # Limitar dataset para treino ser rápido
    GLOBAL_DF = GLOBAL_DF[-1000:].reset_index(drop=True)

    output_dir = "resultados_visuais"
    os.makedirs(output_dir, exist_ok=True)

    # 2. Experimento: Evolução
    pop_size = 20
    generations = 10

    print(f"    [GENETICO] Populacao={pop_size}, Geracoes={generations}")
    pop = toolbox.population(n=pop_size)

    # Estatísticas
    stats = tools.Statistics(lambda ind: ind.fitness.values)
    stats.register("avg", np.mean)
    stats.register("max", np.max)

    # Rodar!
    pop, logbook = algorithms.eaSimple(pop, toolbox, cxpb=0.5, mutpb=0.2,
ngen=generations,

                                stats=stats, verbose=True)

    # 3. Pegar o Campeão
    best_ind = tools.selBest(pop, 1)[0]
    print(f"\n  Melhor Estratégia Encontrada:")
    print(f"    Média Curta: {best_ind[0]} dias")
    print(f"    Média Longa: {best_ind[1]} dias")
    print(f"    Stop Loss: {best_ind[2]*100:.1f}%")
    print(f"    Lucro: ${best_ind.fitness.values[0]:.2f}")

    # 4. Validar e Plotar
    lucro, curva_patrimonio = simular_trading(GLOBAL_DF, best_ind[0], best_ind[1],
best_ind[2])

    # Curva Buy & Hold (Se tivesse só comprado e segurado)
    preco_inicial = GLOBAL_DF['Close'].iloc[0]
    buy_hold = (GLOBAL_DF['Close'] / preco_inicial) * 10000.0

    plt.figure(figsize=(12, 6))
    plt.plot(buy_hold, label='Buy & Hold (Passivo)', color='gray', linestyle='--')
    plt.plot(curva_patrimonio, label='IA Trader (Genético)', color='green',
linewidth=2)
    plt.title(f"A Evolução do Lucro: Genético vs Passivo")
    plt.ylabel("Patrimônio (USD)")
    plt.xlabel("Dias")
```



```
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig(f"{output_dir}/resultado_trader.png")
print(f"    [GRAFICO] Salvo em {output_dir}/resultado_trader.png")

if __name__ == "__main__":
    rodar_algoritmo_genetico()
```

5.13 Arquivo: laboratorio_ia\04_xgboost_bitcoin_sazonal\main.py

```
import sys
import os
import numpy as np
import pandas as pd
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
import matplotlib.pyplot as plt

# Adicionar diretório pai ao path para importar data_loader
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from data_loader import load_and_process_data

def train_xgboost():
    print("="*50)
    print("[INI] MODELO 1: MACHINE LEARNING (XGBOOST)")
    print("="*50)

    # 1. Carregar Dados Padronizados
    print("[INFO] Carregando dados via data_loader...")

    # Caminho absoluto para evitar erros
    csv_path =
os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data',
'bitcoin.csv')

    data = load_and_process_data(csv_path, seq_length=60)

    X_train, y_train = data['X_train'], data['y_train']
    X_test, y_test = data['X_test'], data['y_test']
    scaler = data['scaler']

    # 2. Ajustar Formato para XGBoost
    # XGBoost precisa de (Amostras, Features). Nossos dados estão em (Amostras, 60,
1).
    # Vamos "achatar" para (Amostras, 60).
    X_train = X_train.reshape(X_train.shape[0], -1)
    X_test = X_test.reshape(X_test.shape[0], -1)

    print(f"[INFO] Formato de Treino: {X_train.shape}")
    print(f"[INFO] Formato de Teste: {X_test.shape}")

    # 3. Criar e Treinar Modelo
    print("[INFO] Treinando XGBoost (pode demorar alguns segundos)...")
    model = XGBRegressor(
        n_estimators=1000,
        learning_rate=0.01,
        max_depth=6,
        early_stopping_rounds=20,
        n_jobs=-1
    )

    # XGBoost precisa de eval_set para early_stopping
    model.fit(
        X_train, y_train,
        eval_set=[(X_test, y_test)],
        verbose=False
    )
```

```
# 4. Previsão
print("[INFO] Gerando previsoes...")
preds_scaled = model.predict(X_test)

# Desfazer normalização para ver preços reais
preds_real = scaler.inverse_transform(preds_scaled.reshape(-1, 1))
y_test_real = scaler.inverse_transform(y_test.reshape(-1, 1))

# 5. Métricas
rmse = np.sqrt(mean_squared_error(y_test_real, preds_real))
mae = mean_absolute_error(y_test_real, preds_real)

print("-" * 30)
print(f"[RES] RESULTADOS PARA XGBOOST:")
print(f"    RMSE (Erro Médio Quadrático): ${rmse:.2f}")
print(f"    MAE (Erro Médio Absoluto):    ${mae:.2f}")
print("-" * 30)

# 6. Salvar Gráfico
plt.figure(figsize=(12, 6))
plt.plot(y_test_real, label='Preço Real', color='blue')
plt.plot(preds_real, label='Previsão XGBoost', color='red', alpha=0.7)
plt.title('Previsão de Bitcoin - XGBoost')
plt.xlabel('Dias (Conjunto de Teste)')
plt.ylabel('Preço (USD)')
plt.legend()
plt.grid(True)

output_dir = os.path.dirname(os.path.abspath(__file__))
plot_path = os.path.join(output_dir, 'result_xgboost.png')
plt.savefig(plot_path)
print(f"[GRAFICO] Salvo em: {plot_path}")
# plt.show() # Comentado para não travar automação, mas pode descomentar

if __name__ == "__main__":
    train_xgboost()
```

5.14 Arquivo: laboratorio_ia\04_xgboost_bitcoin_sazonal\run_xgboost_sazonal.py

```
import sys
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import xgboost as xgb
from xgboost import plot_importance

def rodar_analise_sazonal():
    print(">>> [MÓDULO 04] Caçador de Padrões Sazonais (XGBoost)...")

    # 1. Carregar Dados
    base_dir =
os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    csv_path = os.path.join(base_dir, 'data', 'bitcoin.csv')
    df = pd.read_csv(csv_path)
    df['Date'] = pd.to_datetime(df['Date'])

    # 2. Engenharia de Features (Calendário)
    # Não usamos PREÇO passado. A IA só sabe "Que dia é hoje?".
    df['DayOfWeek'] = df['Date'].dt.dayofweek # 0=Seg, 6=Dom
    df['DayOfMonth'] = df['Date'].dt.day
    df['Month'] = df['Date'].dt.month
    df['Quarter'] = df['Date'].dt.quarter
    df['Year'] = df['Date'].dt.year

    # Target: Retorno do dia seguinte (t+1) > 0? (1 se subiu, 0 se caiu)
    df['Return'] = df['Close'].pct_change().shift(-1)
    df['Target'] = (df['Return'] > 0).astype(int)
    df = df.dropna()

    # Features X
    features = ['DayOfWeek', 'DayOfMonth', 'Month', 'Quarter', 'Year']
    X = df[features]
    y = df['Target']

    # 3. Treinar XGBoost
    model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.05, max_depth=4)
    model.fit(X, y)

    # 4. Analisar Importância (O que define o preço? O dia da semana?)
    output_dir = "resultados_visuais"
    os.makedirs(output_dir, exist_ok=True)

    plt.figure(figsize=(10, 6))
    plot_importance(model, importance_type='gain', max_num_features=10, height=0.5,
show_values=False)
    plt.title("O que move o Bitcoin? (Importância Sazonal)")
    plt.savefig(f"{output_dir}/importancia_features.png")
    print(f"    [GRAFICO 1] Salvo em {output_dir}/importancia_features.png")

    # 5. Mapa de Calor: Retorno Médio por Dia da Semana
    # Vamos calcular na mão para plotar o que a IA aprendeu
    avg_return_by_day = df.groupby('DayOfWeek')['Return'].mean() * 100 # Em %
    days_labels = ['Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sab', 'Dom']

    plt.figure(figsize=(10, 6))
```

```

    colors = ['green' if x > 0 else 'red' for x in avg_return_by_day]
    plt.bar(days_labels, avg_return_by_day, color=colors)
    plt.title("Padrão Semanal do Bitcoin (Média Histórica)")
    plt.ylabel("Retorno Médio Diário (%)")
    plt.axhline(0, color='black', linewidth=0.8)
    plt.grid(axis='y', alpha=0.3)
    plt.savefig(f"{output_dir}/padrao_semanal.png")
    print(f"    [GRAFICO 2] Salvo em {output_dir}/padrao_semanal.png")

    print("\n>>> Conclusão Sazonal:")
    print(f"    Melhor Dia para Comprar:
{days_labels[np.argmax(avg_return_by_day)]}")
    print(f"    Pior Dia para Comprar: {days_labels[np.argmin(avg_return_by_day)]}")

if __name__ == "__main__":
    rodar_analise_sazonal()

```

5.15 Arquivo: laboratorio_ia\05_transformer_bitcoin_sota\core_transformer.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import math

class MiniTransformer(nn.Module):
    def __init__(self, input_dim, d_model=64, nhead=4, num_layers=2):
        super(MiniTransformer, self).__init__()

        # Projetar input para dimensão do modelo
        self.embedding = nn.Linear(input_dim, d_model)

        # Positional Encoding (Seno/Cosseno para dar noção de tempo)
        self.pos_encoder = nn.Parameter(torch.zeros(1, 1000, d_model))

        # Transformer Encoder (Captura padrões)
        encoder_layer = nn.TransformerEncoderLayer(d_model=d_model, nhead=nhead,
batch_first=True)
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer,
num_layers=num_layers)

        # Decoder Final
        self.decoder = nn.Linear(d_model, 1)

        # Armazenar pesos de atenção para visualização (Hack para fins didáticos)
        self.last_attention_weights = None

    def forward(self, x):
        # x shape: (Batch, SeqLen, Features)
        batch_size, seq_len, _ = x.size()

        # 1. Embed + Posição
        x = self.embedding(x) + self.pos_encoder[:, :seq_len, :]

        # 2. Passar pelo Transformer
        # (Nota: PyTorch padrão não retorna pesos de atenção fácil, então focaremos
na performance)
        out = self.transformer_encoder(x)

        # 3. Pegar apenas o último elemento da sequência
        out = out[:, -1, :]

        # 4. Previsão
        return self.decoder(out)

def treinar_transformer(X_train, y_train, epochs=50):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    input_dim = X_train.shape[2]

    model = MiniTransformer(input_dim=input_dim).to(device)
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.0005)

    X_t = torch.FloatTensor(X_train).to(device)
    y_t = torch.FloatTensor(y_train).to(device).unsqueeze(1)

    model.train()
```

```
print("    [TRANSFORMER] Treinando (Atenção Multicabeça)...")

for epoch in range(epochs):
    optimizer.zero_grad()
    out = model(X_t)
    loss = criterion(out, y_t)
    loss.backward()
    optimizer.step()

return model
```

5.16 Arquivo: laboratorio_ia\05_transformer_bitcoin_sota\main.py

```
import sys
import os
import torch
import torch.nn as nn
import numpy as np
import math
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Adicionar diretório pai
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from data_loader import load_and_process_data

# Configuração do Dispositivo
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super(PositionalEncoding, self).__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-
math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0).transpose(0, 1)
        self.register_buffer('pe', pe)

    def forward(self, x):
        return x + self.pe[:x.size(0), :]

class TimeSeriesTransformer(nn.Module):
    def __init__(self, input_size=1, d_model=64, nhead=8, num_layers=2,
dropout=0.2):
        super(TimeSeriesTransformer, self).__init__()
        self.model_type = 'Transformer'
        self.src_mask = None

        # Camada de entrada para projetar dimensão 1 para d_model
        self.encoder = nn.Linear(input_size, d_model)
        self.pos_encoder = PositionalEncoding(d_model)

        # Transformer Encoder
        encoder_layers = nn.TransformerEncoderLayer(d_model, nhead,
dim_feedforward=128, dropout=dropout)
        self.transformer_encoder = nn.TransformerEncoder(encoder_layers,
num_layers)

        # Decoder final
        self.decoder = nn.Linear(d_model, 1)

    def forward(self, src):
        # src shape: (Sequence, Batch, Feature) -> Transformer do PyTorch espera
        # assim por padrão se batch_first=False
        # Mas nosso loader manda (Batch, Sequence, Feature). Vamos ajustar.
        src = src.permute(1, 0, 2) # Agora (Seq, Batch, Feat)
```



```

        src = self.encoder(src)
        src = self.pos_encoder(src)
        output = self.transformer_encoder(src)

        # Pegar apenas o último output da sequência para prever o próximo passo
        output = output[-1, :, :]

        output = self.decoder(output)
        return output

def train_transformer():
    print("="*50)
    print("[INI] MODELO 3: TRANSFORMER (TIME SERIES) - TESTE ROBUSTO")
    print(f"[INFO] Rodando em: {device}")
    print("="*50)

    # 1. Carregar Dados
    csv_path =
os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data',
'bitcoin.csv')
    data_dict = load_and_process_data(csv_path, seq_length=60)

    # Converter Tensores
    X_train = torch.from_numpy(data_dict['X_train']).float().to(device)
    y_train = torch.from_numpy(data_dict['y_train']).float().to(device)
    X_test = torch.from_numpy(data_dict['X_test']).float().to(device)
    y_test = torch.from_numpy(data_dict['y_test']).float().to(device)

    if len(X_train.shape) == 2:
        X_train = X_train.unsqueeze(-1)
        X_test = X_test.unsqueeze(-1)

    # Detectar features dinamicamente
    num_features = X_train.shape[2]
    print(f"[INFO] Features detectadas pelo Transformer: {num_features}")

    # 2. Inicializar Modelo
    model = TimeSeriesTransformer(input_size=num_features, nhead=8,
dropout=0.2).to(device)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.0005) # Learning rate
menor para Transformer

    # 3. Treinamento
    epochs = 600
    print(f"[INFO] Iniciando treinamento por {epochs} epocas (Modo TITAN -
GPU)...")

    model.train()
    for epoch in range(epochs):
        optimizer.zero_grad()
        output = model(X_train)

        # Ajustar dimensões se necessário
        loss = criterion(output.squeeze(), y_train.squeeze())

        loss.backward()
        optimizer.step()

```

```

        if (epoch+1) % 50 == 0:
            print(f"    Epoca [{epoch+1}/{epochs}], Loss: {loss.item():.6f}")

# 4. Avaliação
model.eval()
with torch.no_grad():
    preds_scaled = model(X_test).cpu().numpy()

scaler = data_dict['scaler']
preds_real = scaler.inverse_transform(preds_scaled)
target_real = scaler.inverse_transform(data_dict['y_test'].reshape(-1, 1))

# 5. Métricas
rmse = np.sqrt(mean_squared_error(target_real, preds_real))
mae = mean_absolute_error(target_real, preds_real)

print("-" * 30)
print(f"[RES] RESULTADOS PARA TRANSFORMER:")
print(f"    RMSE: ${rmse:.2f}")
print(f"    MAE:  ${mae:.2f}")
print("-" * 30)

# 6. Salvar Gráfico Validação
plt.figure(figsize=(12, 6))
plt.plot(target_real, label='Preço Real', color='blue')
plt.plot(preds_real, label='Previsão Transformer', color='purple', alpha=0.8)
plt.title('Validação Transformer: Real vs Previsto')
plt.xlabel('Dias')
plt.ylabel('Preço (USD)')
plt.legend()
plt.grid(True)

output_dir = os.path.dirname(os.path.abspath(__file__))
plot_path = os.path.join(output_dir, 'result_transformer_validation.png')
plt.savefig(plot_path)
print(f"[GRAFICO] Validacao salva em: {plot_path}")

# =====
# 7. MODO BOLA DE CRISTAL (PREVISÃO FUTURA)
# =====
print("\n[FUTURO] Iniciando projecao para os proximos 30 dias (Janeiro
2026)...")

# Pegar os ultimos 60 dias reais do dataset completo
df_full = data_dict['original_df']
last_60_days = df_full['Close'].values[-60:]

# Normalizar igual ao treino
last_60_scaled = scaler.transform(last_60_days.reshape(-1, 1))

# Loop de projeção recursiva
future_predictions = []
# Transformer espera (Batch, Seq, Features) -> (1, 60, 1)
current_sequence = torch.FloatTensor(last_60_scaled).view(1, 60, 1).to(device)

for _ in range(30):
    with torch.no_grad():
        next_val = model(current_sequence)
        future_predictions.append(next_val.item())

```

```

        # Atualizar sequência
        next_val_reshaped = next_val.view(1, 1, 1)
        current_sequence = torch.cat((current_sequence[:, 1:, :],
next_val_reshaped), dim=1)

    # Desnormalizar
    future_predictions = np.array(future_predictions).reshape(-1, 1)
    future_real_price = scaler.inverse_transform(future_predictions)

    # Salvar Gráfico Futuro
    plt.figure(figsize=(10, 5))
    plt.plot(range(1, 31), future_real_price, marker='o', color='magenta',
label='Projeção Transformer Jan/2026')
    plt.title('Projeção Transformer - Próximos 30 Dias')
    plt.xlabel('Dias Futuros')
    plt.ylabel('Preço Estimado (USD)')
    plt.legend()
    plt.grid(True)

    future_plot_path = os.path.join(output_dir, 'result_transformer_FORECAST.png')
    plt.savefig(future_plot_path)
    print(f"[GRAFICO] Previsao Futura salva em: {future_plot_path}")

    print("-" * 30)
    print(f"Preco Estimado para Daqui 30 dias (Transformer): ${future_real_price[-
1][0]:.2f}")
    print("-" * 30)

if __name__ == "__main__":
    train_transformer()

```

5.17 Arquivo: laboratorio_ia\05_transformer_bitcoin_sota\run_atencao_visual.py

```
import sys
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from core_transformer import treinar_transformer
import torch

def rodar_transformer_sota():
    print(">>> [MÓDULO 05] State-of-the-Art (Transformer Attention)...")

    # 1. Carregar e Preparar Dados
    base_dir =
os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
    csv_path = os.path.join(base_dir, 'data', 'bitcoin.csv')
    df = pd.read_csv(csv_path)

    # Dataset multivariado simples
    data = df[['Close', 'Volume']].values
    scaler = MinMaxScaler()
    data_scaled = scaler.fit_transform(data)

    # Criar sequências (Olhar 60 dias para trás)
    seq_len = 60
    X, y = [], []
    for i in range(len(data_scaled) - seq_len):
        X.append(data_scaled[i:i+seq_len])
        y.append(data_scaled[i+seq_len, 0]) # Predizer apenas Close

    X = np.array(X)
    y = np.array(y)

    # Split
    split = int(len(X) * 0.9) # Treino maior
    X_train, y_train = X[:split], y[:split]
    X_test, y_test = X[split:], y[split:]

    # 2. Treinar
    model = treinar_transformer(X_train, y_train, epochs=50)

    # 3. Previsão
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.eval()
    with torch.no_grad():
        X_t = torch.FloatTensor(X_test).to(device)
        preds_scaled = model(X_t).cpu().numpy()

    # 4. Desnormalizar
    dummy = np.zeros((len(preds_scaled), 2))
    dummy[:, 0] = preds_scaled[:, 0]
    preds_real = scaler.inverse_transform(dummy)[:, 0]

    dummy_y = np.zeros((len(y_test), 2))
    dummy_y[:, 0] = y_test
    y_test_real = scaler.inverse_transform(dummy_y)[:, 0]
```

```
# 5. Visualização SOTA
output_dir = "resultados_visuais"
os.makedirs(output_dir, exist_ok=True)

plt.figure(figsize=(14, 7))
plt.plot(y_test_real, label='Preço Real', color='black', alpha=0.5)
plt.plot(preds_real, label='Previsão Transformer SOTA', color='purple',
linewidth=1.5)

# Destacar acertos em picos (onde a IA brilhou)
# Lógica simples visual: Círculos onde a variação foi alta e a IA acertou perto
plt.title("Transformer SOTA: Capacidade de Capturar Tendências de Longo Prazo")
plt.xlabel("Dias (Conjunto de Teste)")
plt.ylabel("Preço (USD)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig(f"{output_dir}/sota_prediction.png")
print(f"    [GRAFICO] Salvo em {output_dir}/sota_prediction.png")

if __name__ == "__main__":
    rodar_transformer_sota()
```

5.18 Arquivo: laboratorio_ia\utils\processar_onchain.py

```
import pandas as pd
import os
import gc

def processar_big_data_onchain(input_file, output_file):
    """
    Kneads Big Data into Small Data.
    Lê arquivos gigantes (GBs) de transações bloco a bloco e
    converte em um CSV diário (KB) com Volume Total Agregado.
    """
    print(f"□ Iniciando processamento de: {os.path.basename(input_file)}")
    print(f"□ Saída será salva em: {output_file}")

    # Prepara um dicionário para acumular os volumes por dia
    # { '2024-01-01': 500.2, '2024-01-02': 1020.5 ... }
    daily_volume = {}

    # Ler em chunks de 100.000 linhas para não explodir a RAM
    chunk_size = 100000
    total_rows = 0

    try:
        # Tenta detectar se o arquivo tem cabeçalho comentado ou pula linhas
        # iniciais
        # O arquivo do WalletExplorer tem um header chato na linha 1
        for chunk in pd.read_csv(input_file, chunksize=chunk_size, skiprows=1):

            # Converter coluna date
            # Formato esperado: '2025-01-28 07:26:42'
            chunk['date'] = pd.to_datetime(chunk['date']).dt.date

            # Garantir que amounts sejam numéricos
            cols_to_sum = ['received amount', 'sent amount']
            for col in cols_to_sum:
                if col in chunk.columns:
                    chunk[col] = pd.to_numeric(chunk[col],
errors='coerce').fillna(0)

            # Agrupar este chunk por data
            # Somamos received + sent para ter o "Volume Movimentado Total"
            chunk['total_vol'] = chunk['received amount'] + chunk['sent amount']
            agg = chunk.groupby('date')['total_vol'].sum()

            # Adicionar ao acumulador global
            for data, vol in agg.items():
                date_str = str(data)
                daily_volume[date_str] = daily_volume.get(date_str, 0) + vol

            total_rows += len(chunk)
            print(f"    Processadas {total_rows} linhas...", end='\r')

            # Limpar memória
            del chunk
            gc.collect()

    except Exception as e:
        print(f"\n❌ Erro crítico: {e}")
```

```

        return

    print(f"\n✅ Leitura concluída! Salvando CSV agregado...")

    # Converter dicionário para DataFrame final e salvar
    df_final = pd.DataFrame(list(daily_volume.items()), columns=['Date',
'OnChain_Volume_BTC'])
    df_final = df_final.sort_values('Date')
    df_final.to_csv(output_file, index=False)

    print(f"✅ Sucesso! Arquivo gerado: {output_file}")
    print(df_final.head())

if __name__ == "__main__":
    # Exemplo de uso: Rodar apenas se chamado diretamente
    # Caminhos relativos assumindo execução da raiz
    data_dir =
os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data')

    # Lista de arquivos para processar (Exemplo: Huobi que é menor p/ teste local)
    target = "walletexplorer-Huobi_com-000012a55e988d91.csv"
    input_path = os.path.join(data_dir, target)
    output_path = os.path.join(data_dir, "processed_huobi_daily.csv")

    if os.path.exists(input_path):
        processar_big_data_onchain(input_path, output_path)
    else:
        print(f"Arquivo não encontrado: {input_path}")

```

5.19 Archivo: neural_networks\main.py

```
import torch
import torch.nn as nn
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt

# Load and prepare (same as Deep Learning)
df = pd.read_csv('../data/bitcoin.csv', parse_dates=['Date'])
data = df['Close'].values.reshape(-1, 1)
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)

def create_sequences(data, seq_length=60):
    xs, ys = [], []
    for i in range(len(data) - seq_length):
        xs.append(data[i:i+seq_length].flatten())
        ys.append(data[i+seq_length])
    return np.array(xs), np.array(ys)

X, y = create_sequences(data_scaled)
X = torch.from_numpy(X).float()
y = torch.from_numpy(y).float()

train_size = int(0.8 * len(X))
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(60, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.layers(x)

model = MLP()
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(100):
    model.train()
    outputs = model(X_train)
    loss = criterion(outputs, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    print(f'Epoch {epoch+1}, Loss: {loss.item()}')

# Prediction
model.eval()
preds = model(X_test).detach().numpy()
```



```
preds = scaler.inverse_transform(preds)
real = scaler.inverse_transform(y_test)

plt.plot(real, label='Real')
plt.plot(preds, label='Predicted')
plt.legend()
plt.show()

from sklearn.metrics import mean_squared_error
print('RMSE:', np.sqrt(mean_squared_error(real, preds)))
```